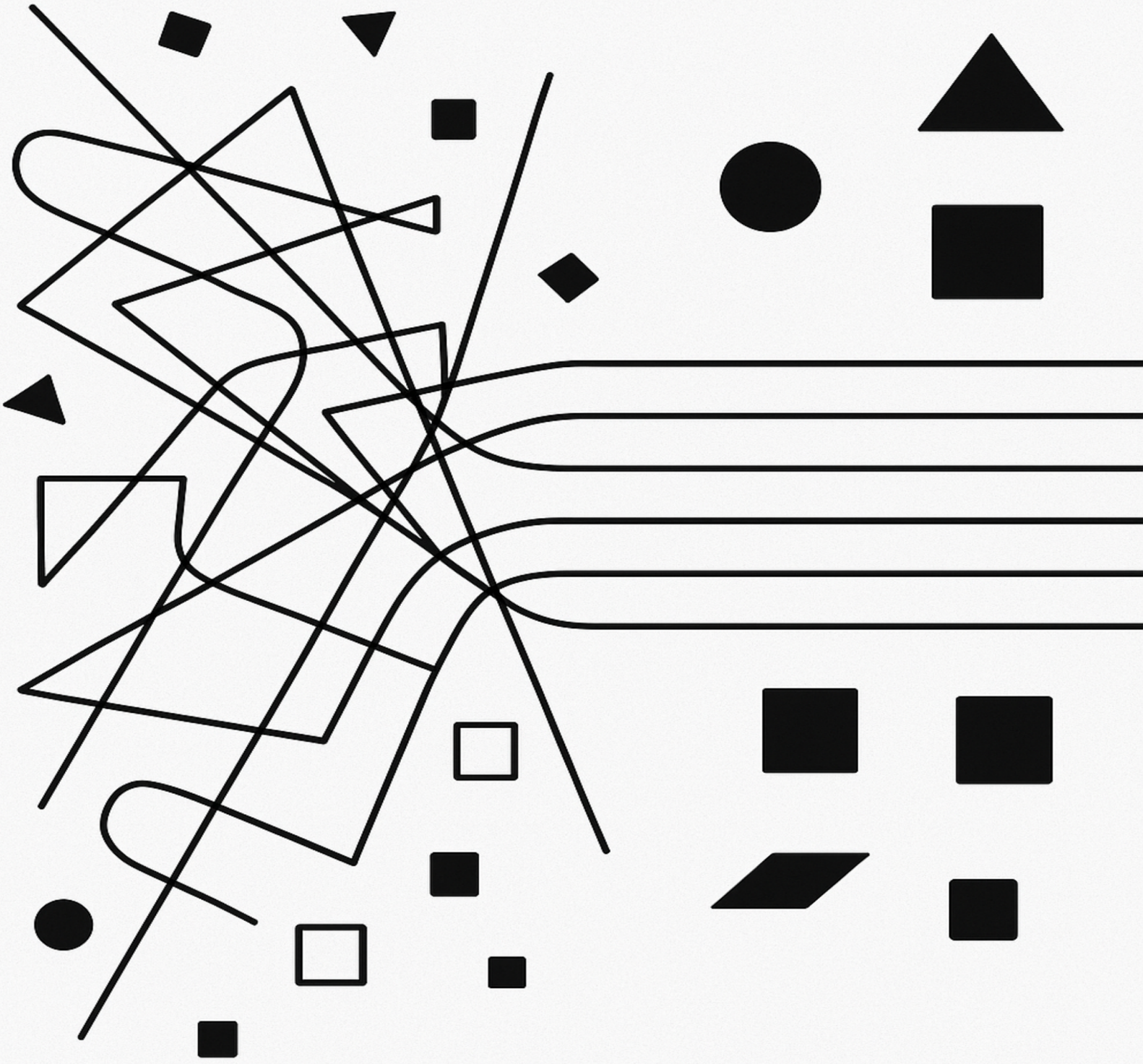
Java Backend Coding Technology

Unified Code Through Functional Composition



Sergiy Yevtushenko

Java Backend Coding Technology

Unified Code Through Functional Composition

Sergiy Yevtushenko

2025

Contents

Chapter 1: Introduction - Code Unification **1**
- What You'll Learn . . . 1
- Code in a New Era . . . 1
- The JBCT Approach . . . 2
- The Five Evaluation Criteria . . . 3
  - Example: Applying the Criteria . . . 3
- Foundational Concepts . . . 4
  - What Are Side Effects? . . . 4
  - What Is Composition? . . . 4
  - What Are Monads (Smart Wrappers)? . . . 5
  - Pragmatic, Not Pure . . . 6
  - Immutability . . . 6
- Who Should Use JBCT? . . . 7
- Key Takeaways . . . 7
- Exercises . . . 7
- What's Next . . . 7

Chapter 2: The Four Return Types **8**
- What You'll Learn . . . 8
- Overview . . . 8
- T - Synchronous, Cannot Fail, Always Present . . . 8
- `Option<T>` - Synchronous, Cannot Fail, May Be Missing . . . 9
- `Result<T>` - Synchronous, Can Fail, Business/Validation Errors . . . 9
- `Promise<T>` - Asynchronous, Can Fail . . . 10
- Why Exactly Four? . . . 11
- Return Type Matrix . . . 12
  - Allowed Return Types . . . 12
  - Discouraged . . . 12
  - Forbidden (Double-Monad Nesting) . . . 12
- Why Not Java's Built-in Types? . . . 13
- Type Conversions . . . 13
  - Lifting: Moving Between Types . . . 13
  - Forbidden Nesting: `Promise<Result<T>>` . . . 14
  - Allowed Nesting: `Result<Option<T>>` . . . 14
- API Quick Reference . . . 14
  - Type Conversions . . . 14
  - Creating Instances . . . 15
- Key Takeaways . . . 15
- Exercises . . . 15
- What's Next . . . 16

Chapter 3: Pragmatica Lite Essentials **17**
- Why a Custom Library? . . . 17
  - Problems with Java's Built-in Types . . . 17
  - What JBCT Needs . . . 17

Installation . . . 18
The Four Core Types . . . 18
`Option<T>` - Value May Be Absent . . . 18
`Result<T>` - Operation May Fail . . . 18
`Promise<T>` - Async Operation May Fail . . . 18
Cause - Typed Error . . . 19
Design Philosophy . . . 19
Consistent Method Names . . . 19
Prefer Cause Methods Over Factory Methods . . . 19
Lift to Higher Types, Not Lower . . . 20
Type Transformations . . . 20
Option Conversions . . . 20
Result Conversions . . . 20
Promise Conversions . . . 21
Common Operations . . . 21
map - Transform Success Value . . . 21
flatMap - Chain Dependent Operations . . . 21
filter - Validate with Predicate . . . 22
recover - Handle Specific Failures . . . 22
or / orElse - Provide Fallback . . . 22
Aggregation . . . 22
Result.all - Validate Multiple Fields . . . 22
Promise.all - Parallel Execution . . . 22
Result.allOf / Promise.allOf - Collection Aggregation . . . 23
any - First Success Wins . . . 23
Exception Handling with lift . . . 23
Basic lift - Wrap Throwing Code . . . 23
lift with Parameters . . . 23
Promise.lift - Async Exception Wrapping . . . 24
Validation Utilities . . . 24
Verify.Is Predicates . . . 24
Combining Validations . . . 25
Parse Utilities - JDK Wrappers . . . 25
Callback Methods . . . 25
Success/Failure Handlers . . . 25
Result Handler (Both Cases) . . . 25
fold - Transform Both Cases . . . 26
Unit Type . . . 26
Thread Safety . . . 26
Promise Resolution . . . 26
Transformation Thread Safety . . . 27
Quick Reference . . . 27
Creating Instances . . . 27
Type Lifting . . . 27
Common Transformations . . . 27
Exercises . . . 28
Summary . . . 28
What’s Next . . . 28

Chapter 4: Parse, Don’t Validate **29**
What You’ll Learn . . . 29
The Principle . . . 29
The Traditional (Wrong) Approach . . . 29
The Parse-Don’t-Validate Approach . . . 30

Naming Conventions . . . 31
Optional Fields with Validation . . . 31
Normalization in Factories . . . 32
Where Validation Belongs . . . 32
Cross-Field Validation with Result.all() . . . 33
Real-World Validation Scenarios . . . 34
Date Range Validation . . . 34
Business Rule Validation . . . 34
Collecting Multiple Errors . . . 35
Pragmatica Lite Validation Utilities . . . 35
Adopting Incrementally . . . 36
Key Takeaways . . . 36
Exercises . . . 36
What's Next . . . 37

Chapter 5: Error Handling & Composition **38**
What You'll Learn . . . 38
No Business Exceptions . . . 38
Why Result: Error Handling Philosophy . . . 38
The Traditional (Wrong) Approach . . . 39
The Result-Based Approach . . . 40
Defining Typed Errors . . . 40
Error Accumulation with Result.all() . . . 41
Programming Errors vs Business Errors . . . 41
Business Errors → Result/Promise with Typed Cause . . . 41
Programming Errors → Exceptions (Fail Fast) . . . 41
Adapter Boundaries → Map to Domain Types . . . 42
Adapter Exceptions: The Boundary . . . 42
Monadic Composition Rules . . . 43
map vs flatMap . . . 43
Lifting Between Types . . . 43
Forbidden: `Promise<Result<T>>` . . . 43
Allowed: `Result<Option<T>>` . . . 43
Testing Errors . . . 44
Key Takeaways . . . 44
Exercises . . . 44
What's Next . . . 45

Chapter 6: Null Policy & Error Recovery **46**
What You'll Learn . . . 46
Null Policy . . . 46
Never Return Null . . . 46
When Null IS Acceptable . . . 47
1. Wrapping External APIs . . . 47
2. Writing to Nullable Database Columns . . . 48
3. Testing Validation . . . 48
When Null is NOT Acceptable . . . 49
Never Pass Null Between JBCT Components . . . 49
Never Use Null for "Unknown" vs "Absent" . . . 49
Never Return Null from Business Logic . . . 50
Null Policy Summary . . . 50
Error Recovery Patterns . . . 51
Providing Fallback Values . . . 51
Fallback to Alternative Operations . . . 52

Recovering from Specific Failures . . . 52
Graceful Degradation Pattern . . . 53
Real-World Example: Configuration with Fallbacks . . . 53
Recovery Patterns Summary . . . 54
When NOT to Recover . . . 54
Key Takeaways . . . 54
Exercises . . . 55
What's Next . . . 55

Chapter 7: Basic Patterns & Structure **56**
What You'll Learn . . . 56
Single Pattern Per Function . . . 56
The Pain of Mixed Patterns . . . 56
Example Violation . . . 57
Corrected . . . 58
Mechanical Refactoring . . . 58
Single Level of Abstraction . . . 58
Anti-Pattern . . . 59
Correct Approach . . . 59
Allowed in Lambdas . . . 60
Forbidden in Lambdas . . . 60
The Three-Zone Framework . . . 61
The Stepdown Rule Test . . . 61
Pattern: Leaf . . . 62
Business Leaves . . . 62
Adapter Leaves . . . 63
Thread Safety . . . 63
Placement Rules . . . 64
Pattern: Condition . . . 64
Simple Conditional . . . 64
Pattern Matching . . . 64
Nested Conditions . . . 65
Condition with Monads . . . 65
Pattern: Iteration . . . 66
Mapping Collections . . . 66
Filtering and Transforming . . . 66
Async Iteration . . . 66
Common Mistakes . . . 67
Naming Conventions . . . 68
Factory Method Naming . . . 68
Validated Input Naming . . . 68
Zone-Based Naming Vocabulary . . . 68
Test Naming . . . 69
Key Takeaways . . . 69
Exercises . . . 69
What's Next . . . 69

Chapter 8: Advanced Patterns **70**
What You'll Learn . . . 70
Pattern: Sequencer . . . 70
The 2-5 Rule . . . 70
Sync Example . . . 70
Async Example . . . 71
When to Extract Sub-Sequencers . . . 72

Thread Safety . . . 72
Common Mistakes . . . 72
Pattern: Fork-Join . . . 74
Two Primary Flavors . . . 74
Special Fork-Join Cases . . . 75
Independence and Thread Safety . . . 75
Common Mistakes . . . 76
Pattern: Aspects (Decorators) . . . 77
Placement . . . 77
Example: Retry Aspect on a Step . . . 77
Example: Metrics Aspect on Use Case . . . 78
Composing Multiple Aspects . . . 78
Operational Semantics . . . 78
Testing Aspects . . . 79
Common Mistakes . . . 80
Key Takeaways . . . 80
Exercises . . . 80
What's Next . . . 80

Chapter 9: Thread Safety **81**
What You'll Learn . . . 81
Core Rules . . . 81
Pattern-by-Pattern Safety Guarantees . . . 81
Leaf (Sequential) . . . 81
Sequencer (Sequential) . . . 82
Fork-Join (Parallel) . . . 82
Condition (Sequential) . . . 82
Iteration (Sequential) . . . 83
Aspects (Inherits) . . . 83
Promise Resolution Thread Safety . . . 83
Common Mistakes . . . 83
Mistake 1: Shared Mutable State in Fork-Join . . . 83
Mistake 2: Assuming Sequential Execution in Promise.all . . . 84
Mistake 3: Mutating Input Parameters . . . 84
Independence Validation Checklist . . . 84
When to Use Mutable State . . . 85
Testing for Thread Safety . . . 85
Quick Reference Table . . . 85
Key Takeaways . . . 86
Exercises . . . 86
What's Next . . . 86

Chapter 10: Testing Philosophy **87**
What You'll Learn . . . 87
The Problem with Traditional Testing . . . 87
Traditional Approach: Component-Focused . . . 87
What We Actually Want to Test . . . 88
Philosophy: Integration-First Testing . . . 88
The Core Principle . . . 88
The Three Testing Layers . . . 88
The Evolutionary Testing Process . . . 90
Overview . . . 90
Phase 1: Stub Everything . . . 90
Phase 2: Implement Validation . . . 91

Phase 3-N: Continue Expanding . . . 92
Handling Complex Input Objects . . . 92
Test Data Builders . . . 92
Canonical Test Vectors . . . 92
Which Approach to Use? . . . 93
Key Takeaways . . . 93
Exercises . . . 93
What's Next . . . 93

Chapter 11: Testing in Practice 94
What You'll Learn . . . 94
Managing Large Test Counts . . . 94
The "Problem" . . . 94
Strategy 1: Nested Test Classes . . . 94
Strategy 2: Parameterized Tests . . . 95
Strategy 3: Test Organization in Files . . . 96
What to Test Where . . . 96
Coverage Criteria by Component Type . . . 96
The Testing Pyramid . . . 97
Complete Worked Example: RegisterUser . . . 98
Phase 1: Stub Everything . . . 98
Phase 2: Add Validation . . . 98
Phase 3-6: Add Steps Incrementally . . . 99
Migration Guide: From Traditional to Evolutionary . . . 100
Step 1: Add Integration Tests . . . 100
Step 2: Identify Redundancy . . . 100
Step 3: Remove Redundant Unit Tests . . . 100
Step 4: End State . . . 101
Test Speed Classification . . . 101
Fast Tests vs Slow Tests . . . 101
Testing Async Promise-Based Flows . . . 102
Key Takeaways . . . 103
Exercises . . . 103
What's Next . . . 103

Chapter 12: Complete Example - RegisterUser 104
What You'll Learn . . . 104
Requirements . . . 104
Step 1: Use Case Interface . . . 104
Step 2: Validated Request . . . 105
Step 3: Value Objects . . . 106
Step 4: Step Implementations . . . 107
Step 5: Error Types . . . 109
Step 6: Tests . . . 110
Pattern Summary . . . 111
Key Takeaways . . . 111
Exercises . . . 112
What's Next . . . 112

Chapter 13: Complete Example - PlaceOrder 113
Requirements . . . 113
Domain Analysis . . . 113
Value Objects . . . 113
Patterns Identified . . . 114

Package Structure . . . 114
Step 1: Value Objects . . . 114
ProductId . . . 114
Quantity . . . 115
Money . . . 115
Address . . . 116
Step 2: Error Types . . . 117
Step 3: Validated Request . . . 118
Raw Request (from API) . . . 118
Validated Order Line . . . 119
Validated Request . . . 119
Step 4: Step Interfaces . . . 120
Step 5: Supporting Types . . . 122
ReservedInventory . . . 122
PaymentConfirmation . . . 123
PriceCalculator . . . 123
Step 6: Step Implementations . . . 123
CheckInventory (Fork-Join Pattern) . . . 123
ProcessPayment (Leaf with Error Mapping) . . . 125
CreateOrder (Leaf - Database) . . . 126
Step 7: Testing . . . 127
Value Object Tests . . . 127
Use Case Integration Test . . . 128
Key Patterns Demonstrated . . . 130
Compensation Pattern . . . 131
Exercises . . . 131
Summary . . . 131

Chapter 14a: Focused Example - PublishArticle 133
Requirements . . . 133
Domain Model . . . 133
Error Types . . . 134
Validated Request . . . 134
Use Case with Condition Pattern . . . 135
Key Points: Condition Pattern . . . 137
What Condition Does . . . 137
What Condition Does NOT Do . . . 137
Benefits . . . 138
Testing . . . 138
Exercises . . . 140
Summary . . . 140

Chapter 14b: Focused Example - TransferFunds 141
Requirements . . . 141
Domain Model . . . 141
Error Types . . . 142
Validated Request . . . 143
Use Case with Aspects . . . 144
Supporting Types . . . 146
AuditEntry . . . 146
RetryPolicy . . . 146
Step Implementations . . . 147
CheckIdempotency . . . 147
CheckBalance . . . 147

ExecuteTransfer . . . 148
Key Points: Aspects Pattern . . . 149
What Aspects Are . . . 149
Aspect Composition Order . . . 149
Standard Composition Order . . . 149
Aspect Independence . . . 149
Testing . . . 150
Exercises . . . 153
Summary . . . 153

Chapter 15: Project Structure & Framework Integration 154
What You'll Learn . . . 154
Vertical Slicing Philosophy . . . 154
Package Structure . . . 154
Package Placement Rules . . . 155
Use Case Packages . . . 155
Domain Shared . . . 155
Adapter Packages . . . 155
Config Package . . . 156
Module Organization (Optional) . . . 156
When to Use Modules . . . 156
When Single Module is Sufficient . . . 156
Module Dependencies . . . 156
Framework Integration . . . 157
Complete Example: Spring REST -> Use Case -> JOOQ . . . 157
Key Principles . . . 159
1. Vertical Slicing . . . 159
2. Minimal Sharing . . . 159
3. Framework at Edges . . . 159
4. Clear Dependencies . . . 159
5. Adapter Isolation . . . 159
Where Things Go . . . 160
Key Takeaways . . . 160
Exercises . . . 160
What's Next . . . 160

Chapter 16: Systematic Application Guide 161
What You'll Learn . . . 161
The Checkpoint Approach . . . 161
CHECKPOINT 1: Before Writing Any Lambda . . . 161
Rules to Check . . . 161
Allowed Lambda Forms (Exhaustive) . . . 162
Extraction Pattern . . . 162
CHECKPOINT 2: Choosing Return Type . . . 162
Decision Tree . . . 162
Rules to Check . . . 163
CHECKPOINT 3: Writing Factory Methods . . . 163
Rules to Check . . . 163
Pattern . . . 163
CHECKPOINT 4: Designing a Class/Interface . . . 164
Rules to Check . . . 164
Zone Placement . . . 164
CHECKPOINT 5: Writing Monadic Chains . . . 164
Rules to Check . . . 164

Pattern Separation . . . 165
CHECKPOINT 6: Adding Logging . . . 165
Rules to Check . . . 165
Ownership Pattern . . . 165
CHECKPOINT 7: Review Completeness . . . 166
Verification Checklist . . . 166
CHECKPOINT 8: Implementation Patterns . . . 166
Nested Record vs Lambda Pattern . . . 166
Quick Reference: Violation -> Fix Patterns . . . 167
Application Order . . . 167
When Implementing New Code . . . 167
When Reviewing Existing Code . . . 167
Key Takeaways . . . 168
Exercises . . . 168
What's Next . . . 168

Chapter 17: Migration Strategies 169
Assessment: Is Your Codebase Ready? . . . 169
Readiness Checklist . . . 169
Code Smell Indicators . . . 169
Migration Phases . . . 171
Phase 1: Value Objects Only (Week 1-2) . . . 171
Phase 2: Result in New Code (Week 3-4) . . . 172
Phase 3: Extract Use Cases (Week 5-8) . . . 173
Phase 4: Adapter Isolation (Week 9-12) . . . 175
Interoperability with Java Standard Library . . . 176
`Optional<T>` → `Option<T>` . . . 176
`CompletableFuture<T>` → `Promise<T>` . . . 177
Exceptions → Cause Mapping . . . 177
Transitional Adapter Layer . . . 178
Team Adoption Strategies . . . 179
Strategy 1: Champion-Led . . . 179
Strategy 2: New Feature First . . . 179
Strategy 3: Vertical Slice . . . 179
Recommendation . . . 179
Common Resistance and Responses . . . 179
"It's too different from what we know" . . . 179
"We don't have time to rewrite everything" . . . 180
"Our team doesn't know functional programming" . . . 180
"What about debugging?" . . . 180
"Performance overhead?" . . . 180
Migration Checklist . . . 180
Phase 1 Complete When: . . . 180
Phase 2 Complete When: . . . 181
Phase 3 Complete When: . . . 181
Phase 4 Complete When: . . . 181
Exercises . . . 181
Summary . . . 181

Chapter 18: Comparison with Other Approaches 183
Traditional Layered Architecture . . . 183
Structure . . . 183
Example . . . 183
What JBCT Changes . . . 184

What JBCT Keeps . . . 184
Verdict . . . 184
Hexagonal Architecture (Ports and Adapters) . . . 184
Structure . . . 185
Example . . . 185
What JBCT Borrows . . . 186
What JBCT Adds . . . 186
Key Difference . . . 186
Verdict . . . 186
Clean Architecture . . . 186
Structure . . . 186
Example . . . 187
What JBCT Borrows . . . 187
What JBCT Changes . . . 187
Key Difference . . . 188
Verdict . . . 188
Railway-Oriented Programming . . . 188
Concept . . . 188
Example (F# style in Java) . . . 188
What JBCT Borrows . . . 188
What JBCT Adds . . . 189
Key Difference . . . 189
Verdict . . . 189
vavr (formerly Javaslang) . . . 189
What vavr Provides . . . 189
Comparison . . . 189
Key Differences . . . 190
When to Use vavr . . . 190
When to Use Pragmatica Lite . . . 190
Verdict . . . 190
Arrow-kt (Kotlin) . . . 190
What Arrow Provides . . . 191
Why Mention It? . . . 191
Verdict . . . 191
Summary: Where JBCT Fits . . . 191
Exercises . . . 192
Summary . . . 192

Chapter 19: Troubleshooting & FAQ 194
Debugging Monadic Chains . . . 194
Problem: “I can’t see what’s happening inside flatMap” . . . 194
Problem: “My Promise never resolves” . . . 195
Problem: “I’m getting CompositeCause but don’t know which validations failed” . . . 195
Common Mistakes and Fixes . . . 196
Mistake: Nested Result/Promise . . . 196
Mistake: Ignoring the Result . . . 196
Mistake: Throwing inside monadic chain . . . 197
Mistake: Multi-statement lambda . . . 197
Mistake: Using Optional instead of Option . . . 198
Mistake: Bypassing factory method . . . 198
IDE Setup . . . 199
IntelliJ IDEA . . . 199
VS Code . . . 199

Testing Troubleshooting . . . 200
Problem: “Test passes but shouldn’t” . . . 200
Problem: “Async test is flaky” . . . 200
Problem: “Can’t create stub for step interface” . . . 201
FAQ . . . 201
General . . . 201
Error Handling . . . 202
Performance . . . 202
Testing . . . 203
Architecture . . . 203
Error Message Reference . . . 203
“Cannot infer functional interface type” . . . 203
“Incompatible types: `Result<X>` cannot be converted to `Result<Y>`” . . . 204
“No instances of type variable(s) T exist” . . . 204
“Promise.await() blocks forever” . . . 204
Summary . . . 204

Appendix A: Pragmatica Lite Core API Reference 206
Library Information . . . 206
Type Conversions . . . 206
Option Conversions . . . 206
Result Conversions . . . 206
Promise Conversions . . . 207
Cause Conversions . . . 207
Creating Instances . . . 207
Option . . . 207
Result . . . 207
Promise . . . 207
Exception Handling (lift methods) . . . 208
Result.lift . . . 208
Promise.lift . . . 208
Aggregation (all/any/allOf) . . . 209
Result.all (accumulates failures) . . . 209
Promise.all (fail-fast) . . . 209
any Methods . . . 209
Common Methods . . . 209
map/flatMap . . . 209
filter (Result and Promise) . . . 210
Callback Methods . . . 210
fold . . . 210
Recovery . . . 210
Verify.Is Predicates . . . 210
Parse Subpackage . . . 211
Number Parsing . . . 211
DateTime Parsing . . . 211
Network Parsing . . . 212
Causes Utilities . . . 212
Promise-Specific Operations . . . 212
Example Usage Patterns . . . 213
Adapter with exception handling . . . 213
Lifting sync Result to async Promise . . . 213
Testing with functional assertions . . . 213

Appendix B: Exercises and Solutions 214

Part I: Foundations (Chapters 1-3) . . . 214
Exercise 1.1 [Beginner] - Return Type Selection . . . 214
Exercise 1.2 [Beginner] - Option vs Result . . . 214
Exercise 1.3 [Intermediate] - Type Lifting . . . 215
Exercise 1.4 [Beginner] - Cause Creation . . . 216
Exercise 1.5 [Intermediate] - Pragmatica Lite Operations . . . 217
Part II: Core Principles (Chapters 4-6) . . . 217
Exercise 2.1 [Beginner] - Value Object Creation . . . 217
Exercise 2.2 [Intermediate] - Aggregate Validation . . . 218
Exercise 2.3 [Intermediate] - Error Accumulation vs Short-Circuit . . . 219
Exercise 2.4 [Advanced] - Null Handling Strategy . . . 219
Exercise 2.5 [Beginner] - Recovery Patterns . . . 221
Part III: Patterns (Chapters 7-9) . . . 221
Exercise 3.1 [Beginner] - Pattern Identification . . . 221
Exercise 3.2 [Intermediate] - Implement Fork-Join . . . 222
Exercise 3.3 [Intermediate] - Implement Condition Pattern . . . 223
Exercise 3.4 [Advanced] - Implement Aspects . . . 224
Exercise 3.5 [Intermediate] - Thread Safety Analysis . . . 225
Part IV: Testing (Chapters 10-11) . . . 227
Exercise 4.1 [Beginner] - Test Structure . . . 227
Exercise 4.2 [Intermediate] - Stub Implementation . . . 228
Exercise 4.3 [Intermediate] - Testing Async Behavior . . . 230
Part V: Production Systems (Chapters 12-15) . . . 231
Exercise 5.1 [Intermediate] - Complete Use Case Design . . . 231
Exercise 5.2 [Advanced] - Compensation Pattern . . . 232
Exercise 5.3 [Intermediate] - Project Structure . . . 233
Part VI: Adoption (Chapters 16-19) . . . 234
Exercise 6.1 [Beginner] - Code Review Checklist . . . 234
Exercise 6.2 [Intermediate] - Migration Planning . . . 236
Exercise 6.3 [Beginner] - Debugging Practice . . . 237
Summary . . . 237

Appendix C: Glossary 239
A . . . 239
B . . . 239
C . . . 239
D . . . 240
E . . . 240
F . . . 240
G . . . 241
H . . . 241
I . . . 241
J . . . 241
L . . . 241
M . . . 241
O . . . 242
P . . . 242
R . . . 242
S . . . 243
T . . . 243
U . . . 243
V . . . 243
W . . . 244
Quick Reference . . . 244

Chapter 1: Introduction - Code Unification

What You'll Learn

- Why code unification matters in the AI era
- The problems JBCT solves
- The five evaluation criteria for design decisions
- Foundational concepts: side effects, composition, monads

Prerequisites: Mid-/senior Java developers comfortable with Java 21+ features (records, sealed interfaces, pattern matching), lambdas, generics, and basic functional patterns.

Code in a New Era

Software development is changing faster than ever. AI-powered code generation tools have moved from experimental novelty to daily workflow staple in just a few years. We now write code alongside - and increasingly with - intelligent assistants that can generate entire functions, refactor modules, and suggest architectural patterns. This shift creates new challenges that traditional coding practices weren't designed to handle.

Historically, code has carried a heavy burden of personal style. Every developer brings preferences about naming, structure, error handling, and abstraction. Teams spend countless hours in code review debating subjective choices. Style guides help, but they can't capture the deeper structural decisions that make code readable or maintainable. When AI generates code, it inherits these same inconsistencies - we just don't know whose preferences it's channeling or why it made particular choices.

This creates a context problem. When you read AI-generated code, you're reverse-engineering decisions made by a model trained on millions of examples with conflicting styles. When AI reads your code to suggest changes, it must infer your intentions from structure that may not clearly express them. The cognitive overhead compounds: developers burn mental cycles translating between their mental model, the code's structure, and what the AI "thinks" the code means.

Meanwhile, technical debt accumulates silently. Small deviations from good structure - a validation check here, an exception there, a bit of mixed abstraction levels - seem harmless in isolation. But they compound. Refactoring becomes risky. Testing

becomes difficult. The codebase becomes a collection of special cases rather than a coherent system.

> Traditional approaches don't provide clear, mechanical rules for when to refactor or how to structure new code, so these decisions remain subjective and inconsistent.

The JBCT Approach

Java Backend Coding Technology (JBCT) proposes a different approach: **reduce the space of valid choices until there's essentially one good way to do most things**. Not through rigid frameworks or heavy ceremony, but through a small set of rules that make structure predictable, refactoring mechanical, and business logic clearly separated from technical concerns.

The benefits compound:

Unified structure means humans can read AI-generated code without guessing about hidden assumptions, and AI can read human code without inferring structure from context. A use case looks the same whether you wrote it, your colleague wrote it, or an AI assistant generated it. The structure carries the intent.

Minimal technical debt emerges naturally because refactoring rules are built into the methodology. When a function grows beyond one clear responsibility, the rules tell you exactly how to split it. When a component gets reused, there's one obvious place to move it. Debt doesn't accumulate because prevention is cheaper than cleanup.

Close business modeling happens when you're not fighting technical noise. Value objects enforce domain invariants at construction time. Use cases read like business processes because each step does one thing. Errors are domain concepts, not stack traces. Product owners can read the code structure and recognize their requirements.

Requirement discovery becomes systematic. When you structure code as validation → steps → composition, gaps become obvious. Missing validation rules surface when you define value objects. Unclear business logic reveals itself when you can't name a step clearly. Edge cases emerge when you model errors as explicit types.

Business logic as a readable language happens when patterns become vocabulary. The four return types, parse-don't-validate, and the fixed pattern catalog form a consistent way to express domain concepts in code. Anyone who understands the domain can pick up a new codebase virtually instantly.

Deterministic code generation becomes possible when the mapping from requirements to code is mechanical. Given a use case specification - inputs, outputs, validation rules, steps - there's essentially one correct structure. Different developers (or AI assistants) should produce nearly identical implementations.

The Five Evaluation Criteria

Before diving into patterns, understand how we evaluate every decision in JBCT. Traditional "best practices" rely on subjective "readability" - but what does that mean? JBCT uses five objective criteria:

1. **Mental Overhead** - "Don't forget to..." and "Keep in mind..." items you must track. Lower is better.
2. **Business/Technical Ratio** - Domain concepts vs framework/infrastructure noise. Higher domain visibility is better.
3. **Design Impact** - Does an approach enforce good patterns or allow bad ones? Improves consistency or breaks it?
4. **Reliability** - Does the compiler catch mistakes, or must you remember? Type safety eliminates bug classes.
5. **Complexity** - Number of elements, connections, and hidden coupling. Fewer moving parts are better.

These aren't preferences - they're measurable. When we say "don't use business exceptions," we prove it: - **Mental Overhead**: Checked exceptions pollute signatures; unchecked are invisible (+2 for Result) - **Reliability**: Exceptions bypass type checker; Result makes failures explicit (+1 for Result) - **Complexity**: Exception hierarchies create coupling (+1 for Result)

Example: Applying the Criteria

Question: Should we use `@Transactional` annotation or explicit transaction management in use cases?

Analysis using the five criteria:

1. **Mental Overhead:**
   - `@Transactional`: Invisible behavior - must remember that methods run in transactions, requires understanding proxy mechanics
   - Explicit: Transaction boundaries are visible in code
   - **Score: +2 for explicit**
2. **Business/Technical Ratio:**
   - Both approaches are technical infrastructure
   - **Score: 0** (neutral)
3. **Design Impact:**
   - `@Transactional`: Couples business logic to Spring framework
   - Explicit: Business logic stays framework-agnostic
   - **Score: +2 for explicit**
4. **Reliability:**
   - `@Transactional`: Fails silently in some cases (private methods, self-invocation)
   - Explicit: Compiler errors if you forget transaction handling
   - **Score: +1 for explicit**

5. **Complexity:**
   - `@Transactional`: Hidden control flow
   - Explicit: Control flow is visible
   - **Score: +1 for explicit**

Verdict: Use explicit transaction management (Aspect pattern) - Total: +6 points for explicit

This is how every decision in JBCT is made—not based on opinion, but on measurable impact across five dimensions.

Foundational Concepts

What Are Side Effects?

A **side effect** is anything a function does beyond computing and returning a value:

- Writing to a database
- Making an HTTP call
- Writing to a file
- Printing to console
- Modifying a global variable
- Throwing an exception

Pure function (no side effects):

```
public int add(int a, int b) {
    return a + b;  // Only computes and returns
}
```

Impure function (has side effects):

```
public void saveUser(User user) {
    database.save(user);  // Side effect: modifies external state
    logger.info("User saved");  // Side effect: writes to log
}
```

Why care? **Pure functions are predictable**: same inputs always produce same output. They're easy to test (no mocking needed) and safe to run anywhere, anytime.

Impure functions are necessary - your app must interact with the world - but they're **unpredictable**: network might fail, disk might be full, database might be down.

JBCT's approach: **push side effects to the edges**. Keep business logic pure. Isolate impure operations in adapter leaves. This makes your core logic easy to test and reason about.

What Is Composition?

Composition means building complex operations by combining simpler ones.

Traditional imperative style:

```
public String processUser(String email) {
    String trimmed = email.trim();
    String lowercase = trimmed.toLowerCase();
    String validated = validate(lowercase);
    String saved = save(validated);
    return saved;
}
```

Functional composition:

```
public Result<String> processUser(String email) {
    return Result.success(email)
                 .map(String::trim)
                 .map(String::toLowerCase)
                 .flatMap(this::validate)
                 .flatMap(this::save);
}
```

The second version **chains** operations. Each step takes the output of the previous step as input. The data flows through a pipeline.

Why this matters: composition lets you **build complex logic from simple pieces** without intermediate variables or explicit error checking at each step. The structure itself handles error propagation.

What Are Monads (Smart Wrappers)?

In JBCT, we use types that wrap values and control how operations are applied to them.

> **Note:** In functional programming, these are called *monads*. This book uses both terms - "Smart Wrapper" for intuition, "monad" for precision.

A monad controls when and if your operations run.

The Key Insight: Inversion of Control

Traditional code: **you** decide when to do something. Monad code: **the monad** decides when to do something.

Think: "Do this operation, **if/when the value is available**."

```
// Traditional: YOU check, YOU decide
String result;
if (email != null) {
    String trimmed = email.trim();
    if (isValid(trimmed)) {
        result = save(trimmed);
        if (result == null) {
            // Error: save failed
        }
```

```
    } else {
        // Error: invalid
    }
} else {
    // Error: null input
}

// Monad: WRAPPER checks, WRAPPER decides
Result<String> result = Result.success(email)
                             .map(String::trim)
                             .flatMap(this::validate)
                             .flatMap(this::save);
```

You're saying: "Here's what to do with the value... **if** you have one and **when** you're ready."

The monad decides: - **Option**: "I'll apply your operation **if** the value is present" - **Result**: "I'll apply your operation **if** there's no error so far" - **Promise**: "I'll apply your operation **when** the async result arrives"

Each monad has: - **map**: "Transform the value, if/when available" - **flatMap**: "Chain another operation, if/when the current one succeeds"

Pragmatic, Not Pure

JBCT uses *pragmatic* monads. Monad laws are not required. Purity is not a goal. **Predictability is.**

We borrow functional patterns because they make code more predictable and composable, not because we're pursuing theoretical purity. Side effects happen. I/O is necessary. The goal is to make side effects *explicit* and *isolated*, not to eliminate them.

If you're coming from Haskell or Scala, adjust your expectations: this is practical Java, not academic FP.

Immutability

All input data passed to operations must be treated as immutable and read-only. This isn't about dogmatic functional purity - it's about maintaining safety guarantees that make concurrent code predictable.

What MUST be immutable: - Data passed between parallel operations (Fork-Join pattern) - Input parameters to any operation - Response types returned from use cases - Value objects used as map keys or in collections

What CAN be mutable (thread-confined): - Local state within single operation (accumulators, builders) - Working objects within adapter boundaries - State confined to sequential patterns

Key principle: Mutability is safe when state is **thread-confined** (accessed by single thread). Parallel patterns require immutable inputs because no isolation exists.

Who Should Use JBCT?

You should use this if: - You're building backend services (REST APIs, microservices, batch processors) - You want code that's easy for new team members to understand - You're working with AI coding assistants and want generated code to match your structure - You value testability and want to minimize mocking - You're tired of architectural debates and want mechanical rules

This might not fit if: - You're building UI applications (different concerns, different patterns) - You need extreme performance optimization (some abstraction overhead) - Your team is heavily invested in a conflicting architecture - You prefer object-oriented design with mutable state

---

Key Takeaways

1. **JBCT unifies code** - Making code predictable across teams and AI collaboration
2. **Five criteria** guide all decisions - Mental overhead, business/technical ratio, design impact, reliability, complexity
3. **Side effects at edges** - Keep business logic pure, isolate I/O in adapters
4. **Composition over imperative** - Chain operations, let structure handle errors
5. **Monads invert control** - You describe transformations, the monad handles execution

---

Exercises

See Appendix B for exercises on: - Exercise 1.1: Return Type Selection - Exercise 1.2: Option vs Result

---

What's Next

Chapter 2 introduces the four return types that form the foundation of everything else: `T`, `Option<T>`, `Result<T>`, and `Promise<T>`. You'll learn when to use each one, how they compose, and why these four types are all you need.

Chapter 2: The Four Return Types

What You'll Learn

- Why exactly four return types are sufficient
- When to use each type: T, Option, Result, Promise
- How to convert between types
- Why Java's standard library isn't enough

Overview

Every function in JBCT returns exactly one of four types. Not "usually" or "preferably"—exactly one, always. This isn't an arbitrary restriction; it's intentional compression of complexity into type signatures.

Why by criteria: - **Mental Overhead**: Hidden error channels (exceptions), hidden optionality (null), hidden asynchrony (blocking I/O) force remembering behavior not in signatures. Explicit types eliminate this (+3). - **Reliability**: Compiler verifies error handling, null safety, and async boundaries when encoded in types (+3). - **Complexity**: Four types cover all scenarios - no guessing about combinations (+2).

T - Synchronous, Cannot Fail, Always Present

Use this when the operation is pure computation with no possibility of failure or missing data. Mathematical calculations, transformations of valid data, simple getters. If you can't think of a way this function could fail or return nothing, it returns `T`.

```
public record FullName(String value) {
    public String initials() {  // returns String (T)
        return value.chars()
                    .filter(Character::isUpperCase)
                    .collect(StringBuilder::new,
                             StringBuilder::appendCodePoint,
                             StringBuilder::append)
                    .toString();
```

```
    }
}
```

The signature `String initials()` tells you: this always succeeds, always returns a value, completes immediately.

Return `T` when: - Pure computation (e.g., `calculateTotal`, `formatCurrency`) - Transformation of already-valid data (e.g., `toUpperCase`, `extractId`) - Getters for required fields (e.g., `user.email()`, `order.total()`)

`Option<T>` - Synchronous, Cannot Fail, May Be Missing

Use this when absence is a valid outcome but failure isn't possible. Lookups that might not find anything, optional configuration, nullable database columns when null is semantically meaningful. The key: missing data is normal business behavior, not an error.

```
public interface PreferenceRepository {
    Option<Theme> findThemePreference(UserId id);  // might not be set
}
```

The signature `Option<Theme>` tells you: this always succeeds, but the value might be absent. The caller must handle both cases.

Return `Option<T>` when: - Lookup might not find anything (e.g., `findByUsername`) - Field is genuinely optional in the domain (e.g., `user.middleName()`) - "Not found" is a normal outcome, not an error

---

`Result<T>` - Synchronous, Can Fail, Business/Validation Errors

Use this when an operation might fail for business or validation reasons. Parsing input, enforcing invariants, business rules that can be violated. Failures are represented as typed `Cause` objects, not exceptions.

> **Note:** `Cause` represents domain failures or error reasons. Think of it as "FailureReason" or "DomainError"—the typed representation of why a business operation failed.

```
public record Email(String value) {
    private static final Pattern EMAIL_PATTERN =
        Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
    private static final Fn1<Cause, String> INVALID_EMAIL =
        Causes.forOneValue("Invalid email format: %s");
```

```
    public static Result<Email> email(String raw) {
        return Verify.ensure(raw, Verify.Is::notNull)
                     .map(String::trim)
                     .flatMap(Verify.ensureFn(INVALID_EMAIL,
                                              Verify.Is::matches,
                                              EMAIL_PATTERN))
                     .map(Email::new);
    }
}
```

The signature `Result<Email>` tells you: this might fail (invalid format), completes immediately, failure is typed.

Return `Result<T>` when: - Validating input (e.g., `Email.email(raw)`) - Enforcing business rules (e.g., `checkInvariant`) - Parsing or constructing domain objects (e.g., `OrderId.orderId(raw)`)

> **Why Result: Error Handling Philosophy**
>
> Error handling logic belongs where business context exists to make decisions. Sometimes that's close to where the error occurred; sometimes the error propagates unchanged because only the caller has enough context to decide.
>
> Different mechanisms have distinct trade-offs:

Mechanism	Transparency	Ergonomics	Reliability
Checked exceptions	□ Explicit	□ Verbose, coupling	□ Compiler-enforced
Unchecked exceptions	□ Hidden	□ Mental overhead	□ Silent failures
Errors as values (Go)	□ Visible	□ Manual propagation	□ Easy to ignore
Functional (Result)	□ In type	□ Monadic composition	□ Compiler-enforced

> `Result<T>` combines the best: explicit in signature, ergonomic via `map`/`flatMap`, compiler-verified handling.

`Promise<T>` - Asynchronous, Can Fail

Use this for any I/O operation, external service call, or computation that might block. `Promise<T>` is semantically equivalent to `Result<T>` but asynchronous - failures are carried in the Promise itself, not nested inside it.

```
public interface AccountRepository {
    Promise<Account> findById(AccountId id);  // async lookup, can fail
}
```

The signature `Promise<Account>` tells you: this completes later (async), might fail (network, database), failure is carried in the Promise.

> **Promise as Async Result**
>
> Think of `Promise<T>` as the asynchronous counterpart to `Result<T>`. Both represent operations that can succeed or fail with typed errors. The only difference is timing: `Result<T>` completes immediately, `Promise<T>` completes later. The same `map`/`flatMap` patterns work identically; converting is trivial (`result.async()` lifts to Promise, `promise.await()` blocks to Result). When you understand `Result<T>`, you understand `Promise<T>`.

Promise Resolution and Thread Safety:

Promise resolution is **thread-safe** and happens **exactly once**:

- Multiple threads can attempt resolution - only the first succeeds
- Resolution serves as synchronization point
- Transformations execute after resolution
- Side effects execute independently

```
var promise = Promise.<User>promise();

// Multiple threads racing to resolve - only first wins
executor.submit(() -> promise.succeed(user1));  // First to resolve
executor.submit(() -> promise.succeed(user2));  // Ignored

// All transformations see the same result (user1)
promise.map(this::processUser)
       .flatMap(this::saveToDatabase)
       .onSuccess(this::logSuccess);
```

Return `Promise<T>` when: - Any I/O operation (database, HTTP, file system) - External service calls - Operations that might block or take time

---

Why Exactly Four?

These four types form a complete basis for composition. You can lift "up" when needed (`Option` to `Result` to `Promise`), but you never nest the same concern twice.

Each type represents one orthogonal concern: - **Synchronous vs. asynchronous**: now vs. later - **Can fail vs cannot fail**: error channel present or absent - **Value vs optional value**: presence guaranteed or not

Decision table:

Sync?	Can Fail?	May Be Absent?	Return Type
Yes	No	No	`T`
Yes	No	Yes	`Option<T>`
Yes	Yes	No	`Result<T>`
Async	Yes	No	`Promise<T>`

Return Type Matrix

Allowed Return Types

Type	Use Case
`T`	Synchronous, cannot fail, always present
`Option<T>`	Synchronous, cannot fail, might be absent
`Result<T>`	Synchronous, can fail
`Promise<T>`	Asynchronous, can fail
`Result<Option<T>>`	Optional value that can fail validation
`Promise<Option<T>>`	Async lookup that might not find anything

Discouraged

Type	Why Discouraged
`Optional<T>`	Use `Option<T>` for consistency
`CompletableFuture<T>`	Use `Promise<T>` for consistent error handling
Framework-specific types (`Mono<T>`, `ResponseEntity<T>`)	Keep business logic framework-agnostic

Forbidden (Double-Monad Nesting)

Type	Why Forbidden
`Promise<Result<T>>`	`Promise` already carries failures - double error channel
`Result<Result<T>>`	Nested failures create unwrapping ceremony
`Option<Option<T>>`	Nested optionality is meaningless
`Promise<Option<Result<T>>>`	Triple nesting - architectural smell

Rule: Each monadic concern (optionality, failure, asynchrony) appears at most once in a return type.

Why Not Java's Built-in Types?

"Can't I just use `Optional`, `CompletableFuture`, and exceptions?"

You could, but you'd hit these problems:

Java Approach	Problem	JBCT Solution
`return null`	Hidden optionality → NPE at runtime	`Option<T>` explicit in type
`Optional<T>`	Can't represent failures	`Result<T>` for typed errors
`try-catch`	Invisible control flow	`Result<T>` - errors as values
`CompletableFuture<T>`	Complex error handling	`Promise<T>` consistent patterns

Key differences: - **Explicit in types**: Signature tells you failure modes, no hidden exceptions - **Consistent composition**: `map`/`flatMap` work the same across all types - **No nesting**: One concern per type level - **Better inference**: AI can generate correct error handling from types alone

---

Type Conversions

Lifting: Moving Between Types

You can lift a "lower" type into a "higher" one:

```
// T → Option/Result/Promise
Option.option(value)
Result.success(value)
Promise.success(value)

// Option → Result/Promise
option.toResult(cause)
option.async(cause)

// Result → Promise
result.async()
```

Example:

```
public Promise<Response> execute(Request request) {
    return ValidRequest.validRequest(request)
                       .async()  // Result → Promise
                       .flatMap(step1::apply)
```

```
                        .flatMap(step2::apply);
}
```

Forbidden Nesting: `Promise<Result<T>>`

`Promise<Result<T>>` is forbidden. `Promise<T>` already carries failures - nesting `Result` inside creates two error channels.

Wrong:

```
Promise<Result<User>> loadUser(UserId id) { /* ... */ }

// Caller must unwrap twice - absurd ceremony
loadUser(id)
    .flatMap(resultUser -> resultUser.fold(
        Cause::promise,
        user -> Promise.success(user)
    ));
```

Right:

```
Promise<User> loadUser(UserId id) { /* ... */ }

// Caller just chains
return loadUser(id).flatMap(nextStep);
```

Allowed Nesting: `Result<Option<T>>`

`Result<Option<T>>` is permitted sparingly for "optional value that can fail validation."

```
Result<Option<ReferralCode>> refCode = ReferralCode.referralCode(input);
// Success(None) = not provided, valid
// Success(Some(code)) = provided and valid
// Failure(cause) = provided but invalid
```

API Quick Reference

Type Conversions

```
// Option conversions
option.toResult(cause)      // Option → Result
option.async(cause)         // Option → Promise
option.toOptional()         // Option → Java Optional

// Result conversions
result.async()              // Result → Promise
```

```
result.option()              // Result → Option (loses error)

// Promise conversions
promise.await()              // Promise → Result (blocks)
promise.await(timeout)       // Promise → Result (with timeout)

// Cause conversions (preferred)
cause.result()               // Cause → Result failure
cause.promise()              // Cause → Promise failure
```

Creating Instances

```
// Option
Option.some(value)
Option.none()
Option.option(nullable)      // null → none

// Result
Result.success(value)
cause.result()               // Preferred for failure

// Promise
Promise.success(value)
cause.promise()              // Preferred for failure
Promise.promise()            // Unresolved
```

Key Takeaways

1. **Four types cover all cases** - T, Option, Result, Promise
2. **Signatures tell everything** - failure modes, optionality, synchrony
3. **Lift up, never nest** - Convert to higher types, never `Promise<Result<T>>`
4. **Consistent composition** - Same map/flatMap across all types
5. **Use Cause methods** - Prefer `cause.result()` over `Result.failure(cause)`

Exercises

See Appendix B for exercises on: - Exercise 1.3: Type Lifting - Exercise 1.4: Cause Creation - Exercise 1.5: Pragmatica Lite Operations

---

What's Next

Chapter 3 covers Pragmatica Lite Core - the library that provides these four types. You'll learn the API in depth and see common usage patterns.

Chapter 3: Pragmatica Lite Essentials

This chapter introduces Pragmatica Lite Core—the library that provides the foundational types for JBCT. You'll learn why the library exists, its design philosophy, and how to use its core features effectively.

Why a Custom Library?

Java provides `Optional` and `CompletableFuture`. Why not use them?

Problems with Java's Built-in Types

Type	Problem
`Optional`	No error channel - you know something is missing but not why
`Optional`	Not designed for validation - orElseThrow loses context
`CompletableFuture`	Exceptions as control flow - checked exceptions don't compose
`CompletableFuture`	Verbose API - thenApply, thenCompose, exceptionally chains
`null`	Billion-dollar mistake - no type safety, NPE at runtime

What JBCT Needs

1. **Four distinct return types** - Each with clear semantics
2. **Composable error handling** - Errors as values, not exceptions
3. **Consistent API** - Same method names across all types
4. **Type-safe transformations** - Lift between types without losing information

Pragmatica Lite Core provides exactly this.

Installation

Maven (preferred):

```
<dependency>
   <groupId>org.pragmatica-lite</groupId>
   <artifactId>core</artifactId>
   <version>0.8.4</version>
</dependency>
```

Gradle:

```
implementation 'org.pragmatica-lite:core:0.8.4'
```

The Four Core Types

`Option<T>` - Value May Be Absent

```
import org.pragmatica.lang.Option;

Option<User> user = Option.some(new User("Alice"));
Option<User> nobody = Option.none();
Option<User> fromNullable = Option.option(possiblyNullValue);
```

Use when: Value might not exist, but absence is not an error.

Examples: Cache lookup, optional configuration, search result.

`Result<T>` - Operation May Fail

```
import org.pragmatica.lang.Result;
import org.pragmatica.lang.Cause;

Result<Email> valid = Result.success(new Email("user@example.com"));
Result<Email> invalid = INVALID_EMAIL.result();  // Preferred: fluent style

// Also works (but verbose)
Result<Email> invalid2 = Causes.cause("Invalid format").result();
```

Use when: Synchronous operation that can fail with typed error.

Examples: Validation, parsing, business rule checks.

`Promise<T>` - Async Operation May Fail

```java
import org.pragmatica.lang.Promise;

Promise<User> fetching = Promise.promise();  // Unresolved
Promise<User> ready = Promise.success(user);  // Already resolved
Promise<User> failed = USER_NOT_FOUND.promise();  // Preferred: fluent style
```

Use when: Asynchronous operation (I/O, external service).

Examples: Database query, HTTP call, file read.

Cause - Typed Error

```java
import org.pragmatica.lang.Cause;
import org.pragmatica.lang.utils.Causes;

// Simple cause
Cause simple = Causes.cause("Something went wrong");

// Cause with context
Fn1<Cause, String> INVALID_EMAIL = Causes.forOneValue("Invalid email: %s");
Cause withContext = INVALID_EMAIL.apply("not-an-email");

// Cause from exception
Cause fromException = Causes.fromThrowable(exception);
```

Use when: Representing failure reason in Result or Promise.

Design Philosophy

Consistent Method Names

All types share the same vocabulary where applicable:

Method	Option	Result	Promise
Transform value	`map()`	`map()`	`map()`
Chain operations	`flatMap()`	`flatMap()`	`flatMap()`
Filter with condition	-	`filter()`	`filter()`
Provide fallback	`or()`	`or()`	`or()`
Handle success	`onPresent()`	`onSuccess()`	`onSuccess()`
Handle failure	`onEmpty()`	`onFailure()`	`onFailure()`

Prefer Cause Methods Over Factory Methods

```
// DO: Use Cause methods
Cause error = VALIDATION_FAILED;
return error.result();   // Result<T>
return error.promise();  // Promise<T>

// DON'T: Use factory methods with Cause
return Result.failure(error);   // Works but verbose
return Promise.failure(error);  // Works but verbose
```

Lift to Higher Types, Not Lower

```
// Lifting UP is safe
Option<T> option = ...;
Result<T> result = option.toResult(CAUSE_IF_EMPTY);
Promise<T> promise = result.async();

// Going DOWN loses information
Result<T> result = ...;
Option<T> option = result.option();  // Loses error info!
```

Type Transformations

Option Conversions

```
Option<User> option = findUser(id);

// Option -> Result (provide cause for empty case)
Result<User> result = option.toResult(USER_NOT_FOUND);

// Option -> Promise (provide cause for empty case)
Promise<User> promise = option.async(USER_NOT_FOUND);

// Option -> Java Optional (interop only)
Optional<User> javaOptional = option.toOptional();
```

Result Conversions

```
Result<User> result = validateUser(input);

// Result -> Promise (lift to async context)
Promise<User> promise = result.async();
```

```
// Result -> Option (loses error - use sparingly)
Option<User> option = result.option();
```

Promise Conversions

```
Promise<User> promise = fetchUser(id);

// Promise -> Result (blocks current thread)
Result<User> result = promise.await();

// Promise -> Result with timeout
Result<User> result = promise.await(TimeSpan.timeSpan(5).seconds());
```

Common Operations

map - Transform Success Value

```
Result<String> raw = Result.success("  USER@EXAMPLE.COM  ");

Result<String> normalized = raw
    .map(String::trim)
    .map(String::toLowerCase);
// Result.success("user@example.com")
```

flatMap - Chain Dependent Operations

```
Result<Email> email = Email.email(rawEmail);
Result<Password> password = Password.password(rawPassword);

// Wrong: Nested Result
Result<Result<ValidRequest>> nested = email.map(e ->
    password.map(p -> new ValidRequest(e, p))
);

// Correct: Flat chain
Result<ValidRequest> flat = email.flatMap(e ->
    password.map(p -> new ValidRequest(e, p))
);
```

filter - Validate with Predicate

```
Result<Integer> age = Result.success(25);

Result<Integer> adult = age.filter(
    Causes.cause("Must be 18 or older"),
    a -> a >= 18
);
```

recover - Handle Specific Failures

```
Promise<Config> config = loadFromDatabase()
    .recover(cause -> switch (cause) {
        case DatabaseError _ -> loadFromFile();
        case FileError _ -> Promise.success(Config.defaults());
        default -> cause.promise();
    });
```

or / orElse - Provide Fallback

```
// or: Fallback value
String name = option.or("Anonymous");

// orElse: Fallback wrapped value
Option<User> user = cache.find(id).orElse(database.find(id));
```

Aggregation

Result.all - Validate Multiple Fields

```
Result<ValidRequest> request = Result.all(
    Email.email(raw.email()),
    Password.password(raw.password()),
    Name.name(raw.name())
).map(ValidRequest::new);
```

Behavior: Accumulates ALL failures into CompositeCause. Does not short-circuit.

Promise.all - Parallel Execution

```
Promise<Dashboard> dashboard = Promise.all(
    fetchProfile(userId),
    fetchOrders(userId),
```

```
    fetchPreferences(userId)
).map(Dashboard::new);
```

Behavior: Executes in parallel, fails fast on first failure.

Result.allOf / Promise.allOf - Collection Aggregation

```
// Validate list of emails
Result<List<Email>> emails = Result.allOf(
    rawEmails.stream().map(Email::email).toList()
);

// Fetch multiple users in parallel
Promise<List<Result<User>>> users = Promise.allOf(
    userIds.stream().map(this::fetchUser).toList()
);
```

any - First Success Wins

```
Promise<Config> config = Promise.any(
    loadFromPrimarySource(),
    loadFromSecondarySource(),
    loadFromFallback()
);
```

Exception Handling with lift

Basic lift - Wrap Throwing Code

```
// Wrap supplier that may throw
Result<Integer> parsed = Result.lift(() -> Integer.parseInt(raw));

// With custom error mapping
Result<Integer> parsed = Result.lift(
    ParseError::new,
    () -> Integer.parseInt(raw)
);
```

lift with Parameters

```
// Single parameter
Result<Hash> hashed = Result.lift1(
    HashError::new,
```

```
    encoder::encode,
    password.value()
);

// Two parameters
Result<Token> token = Result.lift2(
    TokenError::new,
    tokenService::generate,
    userId,
    expiry
);
```

Promise.lift - Async Exception Wrapping

```
Promise<User> user = Promise.lift(
    DatabaseError::new,
    () -> jdbcTemplate.queryForObject(sql, mapper, id)
);
```

Validation Utilities

Verify.Is Predicates

```
import org.pragmatica.lang.utils.Verify;

// String checks
Verify.ensure(value, Verify.Is::notNull)
Verify.ensure(value, Verify.Is::notBlank)
Verify.ensure(value, Verify.Is::notEmpty)

// Length checks
Verify.ensure(password, Verify.Is::lenBetween, 8, 128)

// Pattern matching
Verify.ensure(email, Verify.Is::matches, EMAIL_PATTERN)

// Numeric checks
Verify.ensure(age, Verify.Is::positive)
Verify.ensure(age, Verify.Is::between, 0, 150)
Verify.ensure(quantity, Verify.Is::lessThanOrEqualTo, 100)
```

Combining Validations

```
public static Result<Password> password(String raw) {
    return Verify.ensure(raw, Verify.Is::notNull)
        .flatMap(Verify.ensureFn(TOO_SHORT, Verify.Is::lenBetween, 8, 128))
        .flatMap(Verify.ensureFn(NO_DIGIT, Verify.Is::matches, DIGIT_PATTERN))
        .flatMap(Verify.ensureFn(NO_UPPER, Verify.Is::matches, UPPER_PATTERN))
        .map(Password::new);
}
```

Parse Utilities - JDK Wrappers

```
import org.pragmatica.lang.parse.*;

// Instead of Result.lift(() -> Integer.parseInt(raw))
Result<Integer> number = Number.parseInt(raw);
Result<Long> bigNumber = Number.parseLong(raw);
Result<BigDecimal> decimal = Number.parseBigDecimal(raw);

// Date/time parsing
Result<LocalDate> date = DateTime.parseLocalDate(raw);
Result<Instant> instant = DateTime.parseInstant(raw);

// Network types
Result<UUID> uuid = Network.parseUUID(raw);
Result<URI> uri = Network.parseURI(raw);
```

Callback Methods

Success/Failure Handlers

```
result
    .onSuccess(user -> log.info("Found user: {}", user.id()))
    .onFailure(cause -> log.warn("User not found: {}", cause.message()));

promise
    .onSuccess(data -> cache.put(key, data))
    .onFailure(cause -> metrics.incrementFailure());
```

Result Handler (Both Cases)

```
promise.onResult(result -> {
    if (result.isSuccess()) {
```

```
        metrics.recordSuccess();
    } else {
        metrics.recordFailure();
    }
});
```

fold - Transform Both Cases

```
// Result -> ResponseEntity
ResponseEntity<?> response = result.fold(
    cause -> toErrorResponse(cause),    // Failure case first
    data -> ResponseEntity.ok(data)     // Success case second
);
```

Unit Type

For operations that succeed but produce no value:

```
// Use Unit, never Void
Result<Unit> validated = checkBusinessRule(input);
Promise<Unit> sent = sendNotification(user);

// Create success with no value
Result<Unit> ok = Result.unitResult();

// Convert any result to Unit
Promise<Unit> ignored = fetchData().mapToUnit();
```

Thread Safety

Promise Resolution

Promises have exactly-once resolution semantics:

```
Promise<User> promise = Promise.promise();

// Only first resolution wins
promise.succeed(user1);  // This resolves the promise
promise.succeed(user2);  // Ignored
promise.fail(cause);     // Ignored
```

Transformation Thread Safety

Transformations are thread-safe but execution order depends on resolution:

```
Promise<User> promise = fetchUser(id);

// These may execute on different threads
promise
    .map(this::enrichUser)      // Executes after resolution
    .onSuccess(this::cacheUser) // Executes after map
    .onFailure(this::logError); // Executes if failed
```

Quick Reference

Creating Instances

```
// Option
Option.some(value)          // Present
Option.none()               // Empty
Option.option(nullable)     // null -> none, value -> some

// Result
Result.success(value)       // Success
cause.result()              // Failure (preferred)
Result.unitResult()         // Success with Unit

// Promise
Promise.success(value)      // Resolved success
cause.promise()             // Resolved failure (preferred)
Promise.promise()           // Unresolved
```

Type Lifting

```
option.toResult(cause)      // Option -> Result
option.async(cause)         // Option -> Promise
result.async()              // Result -> Promise
promise.await()             // Promise -> Result (blocks)
```

Common Transformations

```
.map(fn)                    // Transform value
.flatMap(fn)                // Chain monadic operation
.filter(cause, predicate)   // Validate with predicate
.recover(fn)                // Handle failure
```

```
.or(fallback)                    // Provide default value
.mapToUnit()                     // Discard value, keep success/failure
```

Exercises

1. **Parse a date range:** Create a `DateRange` value object with `from` and `to` fields. Use `DateTime.parseLocalDate()` and validate that `from` is before `to`.
2. **Combine validations:** Write a `Username` value object that must be 3-20 characters, alphanumeric only, and not on a blocklist. Chain the validations.
3. **Handle nullable external API:** You receive a `Map<String, Object>` from a JSON parser. Write a method that safely extracts an optional `email` field and validates it.
4. **Lift JDBC code:** Wrap a `jdbcTemplate.query()` call in `Promise.lift()` with appropriate error mapping.

Summary

Pragmatica Lite Core provides:

- **Four types with clear semantics** - Option, Result, Promise, Cause
- **Consistent API** - Same method names across types
- **Composable error handling** - Errors as values, not exceptions
- **Type-safe transformations** - Lift between types without losing information
- **Validation utilities** - Verify.Is predicates, parse wrappers
- **Thread-safe promises** - Exactly-once resolution, safe transformations

The library is intentionally minimal. It provides the building blocks for JBCT without imposing architectural decisions. Your business logic remains framework-independent.

What's Next

With the foundation in place, we move to the core principles: - **Chapter 4:** Parse, Don't Validate - Making invalid states unrepresentable - **Chapter 5:** Error Handling & Composition - Typed errors and recovery patterns

Chapter 4: Parse, Don't Validate

What You'll Learn

- Why validation should be inseparable from construction
- How to use factory methods with `Result<T>` returns
- Cross-field validation techniques
- Real-world validation scenarios
- How to adopt this incrementally in existing codebases

Prerequisites: Chapter 2: The Four Return Types

The Principle

Most Java code validates data after construction. You create an object with raw values, then call a `validate()` method that might throw exceptions or return error lists. This approach is backwards.

The principle: Make invalid states unrepresentable. If construction succeeds, the object is valid by definition. Validation is parsing - converting untyped or weakly-typed input into strongly typed domain objects that enforce invariants at the type level.

Why by criteria: - **Mental Overhead**: No "remember to validate" - type system guarantees validity (+2) - **Reliability**: Compiler enforces that invalid objects cannot be constructed (+3) - **Design Impact**: Business invariants concentrated in factories, not scattered (+2) - **Complexity**: Single validation point per type eliminates redundant checks (+1)

The Traditional (Wrong) Approach

```
// DON'T: Validation separated from construction
public class Email {
    private final String value;

    public Email(String value) {
```

```
        this.value = value;  // accepts anything
    }

    public boolean isValid() {  // Caller must remember to check
        return value != null && value.matches("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
    }
}

// Client code must validate manually:
Email email = new Email(input);
if (!email.isValid()) {
    throw new ValidationException("Invalid email");
}
```

Problems: - You can construct invalid `Email` objects - Validation is a separate step that callers might forget - The `isValid()` method returns a boolean, discarding information about what's wrong - You can't distinguish "null" from "malformed" from "too long"

The Parse-Don't-Validate Approach

```
// DO: Validation IS construction
public record Email(String value) {
    private static final Pattern EMAIL_PATTERN =
        Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
    private static final Fn1<Cause, String> INVALID_EMAIL =
        Causes.forOneValue("Invalid email format: %s");

    public static Result<Email> email(String raw) {
        return Verify.ensure(raw, Verify.Is::notNull)
                     .map(String::trim)
                     .flatMap(Verify.ensureFn(INVALID_EMAIL,
                                              Verify.Is::matches,
                                              EMAIL_PATTERN))
                     .map(Email::new);
    }
}

// Client code gets the Result:
Result<Email> result = Email.email(input);
// If this is a Success, the Email is valid. Guaranteed.
```

The constructor is private (or package-private). The only way to get an `Email` is through the static factory `email()`, which returns `Result<Email>`. If you have an `Email` instance, it's valid - no separate check needed.

> **Note:** As of current Java versions, records do not support declaring the canonical constructor as private. Rely on team discipline and code review to ensure value objects are only constructed through their factory methods. Direct `new Email(...)` calls stand out immediately and are easy to catch.

Naming Conventions

Factory naming: Factories are always named after their type, lowercase-first (camelCase):

```
Email.email(raw)
Password.password(raw)
AccountId.accountId(raw)
```

This creates a natural, readable call site that's grep-friendly and allows static imports.

Validated input naming: Use the `Valid` prefix for types representing validated inputs:

```
// DO: Use Valid prefix
record ValidRequest(Email email, Password password) { ... }
record ValidUser(Email email, HashedPassword hashed) { ... }

// DON'T: Use Validated prefix (too verbose)
record ValidatedRequest(...)
```

Optional Fields with Validation

What if a field is optional but must be valid when present? Use `Result<Option<T>>`:

```
public record ReferralCode(String value) {
    private static final String PATTERN = "^[A-Z0-9]{6}$";

    public static Result<Option<ReferralCode>> referralCode(String raw) {
        return isAbsent(raw)
            ? Result.success(Option.none())
            : validatePresent(raw);
    }

    private static boolean isAbsent(String raw) {
        return raw == null || raw.isEmpty();
    }

    private static Result<Option<ReferralCode>> validatePresent(String raw) {
        return Verify.ensure(raw.trim(), Verify.Is::matches, PATTERN)
```

```
                    .map(ReferralCode::new)
                    .map(Option::some);
    }
}
```

Caller semantics: - `Failure(cause)`: Invalid input (provided but doesn't match pattern) - `Success(None)`: No value provided (valid state) - `Success(Some(code))`: Valid code provided

---

Normalization in Factories

Factories can normalize input as part of parsing:

```
public static Result<Email> email(String raw) {
    return Verify.ensure(raw, Verify.Is::notNull)
                 .map(String::trim)           // Remove whitespace
                 .map(String::toLowerCase)    // Lowercase for comparison
                 .flatMap(Verify.ensureFn(INVALID_EMAIL,
                                          Verify.Is::matches,
                                          EMAIL_PATTERN))
                 .map(Email::new);
}
```

Now all `Email` instances are trimmed and lowercased. Domain logic never worries about case or whitespace.

Where Validation Belongs

Clear rule for validation placement:

Validation Type	Where	Example
Single-field invariants	Value object factory	Email format, password strength, ID format
Cross-field invariants	ValidRequest factory	Password doesn't contain email, date range valid
External state invariants	Use case step	Email uniqueness (DB), credit check (external service)

Rationale: - Value objects enforce invariants that depend only on the field's own value

- ValidRequest enforces invariants that require multiple fields but no external state - Use cases handle invariants that require external lookups (database, services)

```
// Value object: single-field invariant
public record Email(String value) {
    public static Result<Email> email(String raw) { /* format check only */ }
}

// ValidRequest: cross-field invariant
public record ValidRegistration(Email email, Password password) {
    public static Result<ValidRegistration> validRegistration(Email email, Password pwd)
        // Check password doesn't contain email local part
    }
}

// Use case step: external state invariant
interface CheckEmailUnique {
    Promise<Email> apply(Email email);  // DB lookup
}
```

Cross-Field Validation with Result.all()

Use `Result.all()` to validate independent fields, then add cross-field rules:

Example: Password must not contain email local part

```
record ValidRegistration(Email email, Password password) {
    private static final Cause PASSWORD_CONTAINS_EMAIL =
        Causes.cause("Password cannot contain email local part");

    static Result<ValidRegistration> validRegistration(String emailRaw,
                                                       String passwordRaw) {
        return Result.all(Email.email(emailRaw),
                          Password.password(passwordRaw))
                     .flatMap(ValidRegistration::checkPasswordNotContainsEmail);
    }

    private static Result<ValidRegistration> checkPasswordNotContainsEmail(
            Email email, Password pwd) {
        String localPart = email.value().split("@")[0];
        return pwd.value().contains(localPart)
            ? PASSWORD_CONTAINS_EMAIL.result()
            : Result.success(new ValidRegistration(email, pwd));
    }
}
```

Pattern: 1. Validate individual fields with `Result.all()` → accumulates per-field er-

rors 2. Use `.flatMap()` to add cross-field validation → fail-fast on cross-field rules 3. Extract cross-field logic to named methods 4. Only construct if all validation passes

---

Real-World Validation Scenarios

Date Range Validation

```
public record DateRange(LocalDate start, LocalDate end) {
    private static final Fn1<Cause, LocalDate> END_BEFORE_START =
        date -> Causes.cause("End date must be after start date: " + date);

    public static Result<DateRange> dateRange(LocalDate start, LocalDate end) {
        return Verify.ensure(start, Verify.Is::notNull)
                     .flatMap(_ -> Verify.ensure(end, Verify.Is::notNull))
                     .flatMap(_ -> Verify.ensure(end, isAfter(start), END_BEFORE_START))
                     .map(_ -> new DateRange(start, end));
    }

    private static Predicate<LocalDate> isAfter(LocalDate start) {
        return end -> end.isAfter(start);
    }
}
```

Business Rule Validation

```
public record ValidOrder(OrderId id, Money total, List<LineItem> items) {
    private static final Fn1<Cause, Money> TOTAL_MISMATCH =
        Causes.forOneValue("Order total does not match line items. Expected: %s");

    public static Result<ValidOrder> validOrder(OrderId id,
                                                Money total,
                                                List<LineItem> items) {
        return total.equals(calculateTotal(items))
            ? Result.success(new ValidOrder(id, total, items))
            : TOTAL_MISMATCH.apply(calculateTotal(items)).result();
    }

    private static Money calculateTotal(List<LineItem> items) {
        return items.stream()
                    .map(LineItem::subtotal)
                    .reduce(Money.ZERO, Money::add);
    }
}
```

Collecting Multiple Errors

```
public record ValidRegistration(Email email, Password password, Age age) {
    public static Result<ValidRegistration> validate(String emailRaw,
                                                     String passwordRaw,
                                                     String ageRaw) {
        return Result.all(Email.email(emailRaw),
                          Password.password(passwordRaw),
                          Age.age(ageRaw))
                     .map(ValidRegistration::new);
        // If any field fails, Result.all() accumulates ALL errors
        // User sees "Invalid email AND password too short AND age out of range"
    }
}
```

Pragmatica Lite Validation Utilities

Verify.Is Predicates:

```
// Instead of custom lambdas:
.flatMap(s -> s.length() >= 8 ? Result.success(s) : Result.failure(...))

// Use standard predicates:
.flatMap(Verify.ensureFn(TOO_SHORT, Verify.Is::lenBetween, 8, 128))
```

Common predicates: `notNull`, `notBlank`, `lenBetween`, `matches`, `positive`, `nonNegative`, `between`, `greaterThan`, `lessThan`, `contains`.

Parse Subpackage:

```
import org.pragmatica.lang.parse.Number;
import org.pragmatica.lang.parse.DateTime;
import org.pragmatica.lang.parse.Network;

Number.parseInt(raw)              // Result<Integer>
DateTime.parseLocalDate(raw)      // Result<LocalDate>
Network.parseUUID(raw)            // Result<UUID>
```

Example:

```
public record Age(int value) {
    public static Result<Age> age(String raw) {
        return Number.parseInt(raw)
                     .flatMap(Verify.ensureFn(Causes.cause("Age 0-150"),
                                              Verify.Is::between, 0, 150))
                     .map(Age::new);
    }
```

```
}
```

Adopting Incrementally

Don't refactor everything at once. Adopt incrementally at boundaries.

1. **New features first** - Use parse-don't-validate for all new code
2. **Keep existing validation at controller boundaries:**

```
// BEFORE: Spring controller with @Valid
@PostMapping("/register")
public ResponseEntity<?> register(@Valid @RequestBody RegistrationRequest dto) {
    var request = new RegisterUser.Request(dto.email(), dto.password());
    return registerUser.execute(request)
                       .fold(this::errorResponse, this::successResponse);
}

// AFTER: Fully migrated
@PostMapping("/register")
public ResponseEntity<?> register(@RequestBody RegisterUser.Request raw) {
    return registerUser.execute(raw)
        .fold(this::errorResponse, this::successResponse);
}
```

3. **Gradually move validation from services to value objects:** - Find a service method with manual validation - Extract that validation into a value object factory - Update callers to use the value object - Repeat

---

Key Takeaways

1. **Validation IS construction** - If an instance exists, it's valid
2. **Factory methods return `Result<T>`** - Success means valid object
3. **`Result.all()` accumulates errors** - Show all problems at once
4. **Normalization in factories** - Trim, lowercase, etc. happen once
5. **`Result<Option<T>>`** - For optional values that must validate when present
6. **Adopt incrementally** - Start at boundaries, work inward

---

Exercises

See Appendix B for exercises on: - Exercise 2.1: PhoneNumber value object - Exercise 2.2: DateRange aggregate validation - Exercise 2.3: Error accumulation vs short-

circuit

What's Next

Chapter 5 covers error handling - how to define typed errors, compose them, and handle failures cleanly without exceptions.

Chapter 5: Error Handling & Composition

What You'll Learn

- Why business logic never throws exceptions
- How to define typed error hierarchies with Cause
- Error accumulation vs fail-fast semantics
- Monadic composition rules

Prerequisites: Chapter 4: Parse, Don't Validate

No Business Exceptions

Business failures are not exceptional—they're expected outcomes of business rules. An invalid email isn't an exception; it's a normal case of bad input. An account being locked isn't an exception; it's a business state.

The rule: Business logic never throws exceptions for business failures. All failures flow through `Result` or `Promise` as typed `Cause` objects.

Why Result: Error Handling Philosophy

Error handling logic belongs where business context exists to make decisions. Sometimes that's close to where the error occurred; sometimes the error propagates unchanged because only the caller has enough context to decide. This fundamental truth doesn't depend on the error mechanism—it's about where knowledge lives.

Different languages use different mechanisms for error propagation, each with distinct trade-offs in **transparency** (is failure visible?), **ergonomics** (is it pleasant to use?), and **reliability** (does the compiler help?):

Mechanism	Transparency	Ergonomics	Reliability
Checked exceptions	✓ Explicit in signature	✗ Verbose, tight coupling	✓ Compiler-enforced

Mechanism	Transparency	Ergonomics	Reliability
Unchecked exceptions	❌ Hidden in implementation	❌ Acceptable, but mental overhead	❌ Silent failures
Errors as values (Go)	✅ Return value visible	❌ Manual `if err != nil` everywhere	❌ Easy to ignore
Functional (Result/Either)	✅ Type signature	✅ Monadic composition	✅ Compiler-enforced

Checked exceptions couple caller and callee tightly—changes in lower-level methods cascade upward, forcing signature changes throughout the call stack.

Unchecked exceptions eliminate coupling but hide failure modes. Every method call requires reading implementation to discover what might throw. The mental overhead is constant; the bugs are intermittent.

Errors as values (Go-style) make failure visible but require manual propagation at every step. Complex scenarios with multiple error sources or interleaved resource management become error-prone boilerplate.

Functional style (`Result<T>`) combines the best properties: failure is explicit in the type signature (transparent), monadic composition eliminates manual propagation (ergonomic), and the compiler ensures every failure is either handled or propagated (reliable). The "do this if value is available" semantics of `map`/`flatMap` means error handling code only appears where decisions are made—not at every intermediate step.

Being absolutely clear about failure possibility isn't pedantry—it's the foundation of maintainable code.

The Traditional (Wrong) Approach

```
// DON'T: Exceptions for business logic
public User loginUser(String email, String password) throws
        InvalidEmailException,
        InvalidPasswordException,
        AccountLockedException,
        CredentialMismatchException {

    if (!isValidEmail(email)) {
        throw new InvalidEmailException(email);
    }
    // ... more checks throwing exceptions
}
```

Problems: - Checked exceptions pollute signatures - Unchecked exceptions are invisible - Exception hierarchies create coupling - Stack traces are expensive for business

failures - Testing requires catching exceptions

The Result-Based Approach

```
// DO: Failures as typed values
public Result<User> loginUser(String emailRaw, String passwordRaw) {
    return Result.all(Email.email(emailRaw),
                      Password.password(passwordRaw))
                 .flatMap(this::validateAndCheckStatus);
}

private Result<User> validateAndCheckStatus(Email email, Password password) {
    return checkCredentials(email, password)
            .flatMap(this::checkAccountStatus);
}

private Result<User> checkAccountStatus(User user) {
    return user.isLocked()
            ? new LoginError.AccountLocked(user.id()).result()
            : Result.success(user);
}
```

Defining Typed Errors

Every failure is a `Cause`. Define error types as sealed interfaces:

```
public sealed interface LoginError extends Cause {
    record AccountLocked(UserId userId) implements LoginError {
        @Override
        public String message() {
            return "Account is locked: " + userId;
        }
    }

    enum InvalidCredentials implements LoginError {
        INSTANCE;

        @Override
        public String message() {
            return "Invalid email or password";
        }
    }
}
```

Benefits: - Exhaustive switch expressions - Compiler checks all cases handled - Clear documentation of failure modes

---

Error Accumulation with Result.all()

`Result.all()` collects validation failures into a `CompositeCause`:

```
// If both fail:
Result.all(Email.email("not-an-email"),
           Password.password("weak"))
      .flatMap(this::processLogin);

// Returns: CompositeCause([
//   "Invalid email format: not-an-email",
//   "Password must be at least 8 characters"
// ])
```

Users see all errors at once, not fix-and-retry repeatedly.

Programming Errors vs Business Errors

Business Errors → Result/Promise with Typed Cause

Business errors are expected outcomes of business rules:

Error Type	Example	Representation
Validation failure	Invalid email format	`Result<T>` with `ValidationError`
Business rule violation	Insufficient funds	`Result<T>` with `InsufficientFunds`
Not found	User doesn't exist	`Result<T>` with `NotFound` or `Option.none()`
External service failure	Payment declined	`Promise<T>` with `PaymentError`

Rule: If a user action can trigger it, it's a business error—represent with `Result` or `Promise`.

Programming Errors → Exceptions (Fail Fast)

Programming errors indicate bugs that should never reach production:

Error Type	Example	Handling
Null invariant violation	Null passed to non-null parameter	`IllegalArgumentException`
Impossible state	Switch case that should never happen	`IllegalStateException`
Configuration error	Missing required config at startup	Fail fast, don't start

Rule: Exceptions for programming errors only when there's no meaningful recovery and the application should terminate or restart.

Adapter Boundaries → Map to Domain Types

Adapter code catches external exceptions and maps to domain types:

```
// Adapter: map null → Option, exception → Cause
public Promise<Option<User>> findById(UserId id) {
    return Promise.lift(
        RepositoryError.DatabaseFailure::new,  // Exception → Cause
        () -> Option.option(jdbcTemplate.queryForObject(...))  // null → Option
    );
}
```

Rule: No external exception or null crosses adapter boundaries—map immediately.

Adapter Exceptions: The Boundary

Foreign code throws exceptions. **Adapters** catch and convert them:

```
class JpaUserRepository implements UserRepository {
    public Promise<Option<User>> findByEmail(Email email) {
        return Promise.lift(
            RepositoryError.DatabaseFailure::new,  // Convert exception → Cause
            () -> {
                // JDBC/JPA code that might throw
                return Option.option(result);
            });
    }
}
```

Business logic never sees foreign exceptions - only domain errors.

Monadic Composition Rules

map vs flatMap

```
// map: Transform success value (T → U)
result.map(String::toUpperCase)

// flatMap: Chain operations (T → Result<U>)
result.flatMap(this::validate)
```

Use `flatMap` when the transformation itself can fail.

Lifting Between Types

```
// Option → Result
option.toResult(cause)

// Result → Promise
result.async()

// Full chain
return ValidRequest.validRequest(request)
            .async()              // Result → Promise
            .flatMap(step1)       // Promise chains
            .flatMap(step2);
```

Forbidden: `Promise<Result<T>>`

`Promise<T>` already carries failures. Never nest:

```
// WRONG
Promise<Result<User>> loadUser(UserId id)

// RIGHT
Promise<User> loadUser(UserId id)
```

Allowed: `Result<Option<T>>`

For optional values that must validate when present:

```
Result<Option<ReferralCode>> refCode = ReferralCode.referralCode(input);
// Success(None) = not provided
// Success(Some(code)) = valid code
// Failure(cause) = invalid code
```

Testing Errors

```
// Test failure
@Test
void email_rejectsInvalidFormat() {
    Email.email("not-an-email")
         .onSuccess(Assertions::fail);  // Fail if unexpectedly succeeds
}

// Test success
@Test
void email_acceptsValidFormat() {
    Email.email("user@example.com")
        .onFailure(Assertions::fail)
        .onSuccess(email -> assertEquals("user@example.com", email.value()));
}

// Test async
@Test
void execute_succeeds() {
    useCase.execute(request)
           .await()
           .onFailure(Assertions::fail)
           .onSuccess(response -> assertEquals("expected", response.value()));
}
```

Key Takeaways

1. **No business exceptions** - Errors are values, not thrown
2. **Sealed interfaces** - Define exhaustive error types
3. **Result.all()** - Accumulates all failures, not just first
4. **Adapters convert** - Exceptions → Cause at boundaries
5. **Never nest** - `Promise<Result<T>>` is forbidden
6. **Test functionally** - Use `onSuccess`/`onFailure` assertions

Exercises

See Appendix B for exercises on: - Exercise 2.3: Error accumulation vs short-circuit - Exercise 2.5: Recovery patterns

---

What's Next

Chapter 6 covers null policy - when null is acceptable and how to recover from errors with fallback values.

Chapter 6: Null Policy & Error Recovery

What You'll Learn

- When null is acceptable (and when it isn't)
- How to handle null at adapter boundaries
- Error recovery patterns with fallback values
- Graceful degradation strategies

Prerequisites: Chapter 5: Error Handling & Composition

Null Policy

> **The One Rule:** No null crosses domain boundaries. Incoming null is mapped to `Option.none()`; outbound domain null is forbidden.

In traditional Java, `null` has two meanings: "value not found" and "error occurred." This ambiguity forces defensive null checks throughout your codebase. JBCT eliminates this confusion with a simple rule: **business logic never returns null**.

Never Return Null

Core Rule: JBCT code NEVER returns null. Use `Option<T>` for optional values.

Traditional Java uses null for semantically different cases - "value not found" and "error occurred." This creates hidden failure modes:

```
// DON'T: Traditional Java with null
public User findUser(UserId id) {
    return repository.findById(id.value());  // May return null - but why?
}

// Caller must defend:
User user = findUser(id);
if (user == null) {  // Not found? Error? Database down? Unknown!
    // What happened? We don't know.
}
```

The caller has no idea whether null means "user doesn't exist" or "database connection failed." This ambiguity spreads defensive null checks everywhere.

JBCT eliminates this:

```
// DO: Using Option
public Option<User> findUser(UserId id) {
    return Option.option(repository.findById(id.value()));
}

// Caller gets explicit semantics:
findUser(id)
    .onPresent(user -> process(user))
    .onEmpty(() -> handleNotFound());
```

Now the type signature tells us exactly what's happening: the user might not be present, but the operation itself cannot fail. If the operation can fail (database error), use `Result<Option<User>>` or `Promise<Option<User>>`.

When Null IS Acceptable

Null appears only at **adapter boundaries** when interfacing with external code that uses null:

1. Wrapping External APIs

When calling external libraries that may return null, wrap immediately at the adapter boundary:

```
// Adapter layer - wrap nullable external API
public Option<User> findUser(UserId id) {
    User user = repository.findById(id.value());  // External API may return null
    return Option.option(user);  // Wrap immediately: null -> none(), value -> some(value
}
```

Spring Data JPA example:

```
public Option<User> findByEmail(Email email) {
    return Option.option(userRepository.findByEmail(email.value()));  // JPA returns null
}
```

JDBC ResultSet example:

```
public Promise<Option<User>> loadUser(UserId id) {
    return Promise.lift(RepositoryError.DatabaseFailure::new,
                        () -> {
                            ResultSet rs = executeQuery(id);
                            User user = rs.next() ? mapUser(rs) : null;  // null if not f
                            return Option.option(user);  // Wrap before returning
```

```
                        });
}
```

Pattern: `Option.option(nullable)` immediately converts external null to `Option.none()`.

2. Writing to Nullable Database Columns

When persisting to databases with nullable columns, convert `Option<T>` to null for the column:

```
// Adapter layer - JOOQ insert with optional field
public Promise<Unit> saveUser(User user) {
    return Promise.lift(
            RepositoryError.DatabaseFailure::new,
            () -> {
                dsl.insertInto(USERS)
                   .set(USERS.ID, user.id().value())
                   .set(USERS.EMAIL, user.email().value())
                   .set(USERS.REFERRAL_CODE,
                        user.refCode().map(ReferralCode::value).orElse(null))  // Option
                   .execute();
                return Unit.unit();
            });
}
```

JDBC PreparedStatement example:

```
PreparedStatement stmt = connection.prepareStatement(
    "INSERT INTO users (id, email, referral_code) VALUES (?, ?, ?)");
stmt.setString(1, user.id().value());
stmt.setString(2, user.email().value());
stmt.setString(3, user.refCode().map(ReferralCode::value).orElse(null));  // Option -> nu
```

Pattern: `.orElse(null)` ONLY when mapping `Option<T>` to nullable database column.

3. Testing Validation

Use null in test inputs to verify that validation correctly rejects null:

```
@Test
void email_fails_forNull() {
    Email.email(null)  // Test null input
         .onSuccess(Assertions::fail);
}

@Test
void validRequest_fails_whenFieldNull() {
    var request = new Request("valid@example.com", null);  // Test null password
```

```
    ValidRequest.validRequest(request)
                .onSuccess(Assertions::fail);
}
```

Pattern: Use null in test inputs to verify validation behavior.

When Null is NOT Acceptable

Never Pass Null Between JBCT Components

Business logic components communicate using domain types, never null:

```
// DON'T: Defensive null checking in business logic
public Result<Order> processOrder(User user, Cart cart) {
    if (user == null || cart == null) {  // Don't do this
        return OrderError.InvalidInput.INSTANCE.result();
    }
    // ...
}

// DO: Parameters guaranteed non-null by convention
public Result<Order> processOrder(User user, Cart cart) {
    // If cart might be absent, parameter should be Option<Cart>
    // If user might be absent, operation shouldn't be called
    // ...
}

// DO: Explicit optionality when needed
public Result<Order> processOrder(User user, Option<Cart> cart) {
    return cart.toResult(OrderError.EmptyCart.INSTANCE)
               .flatMap(c -> validateAndProcess(user, c));
}
```

Rule: If a value might be absent, use `Option<T>` parameter, never null.

Never Use Null for "Unknown" vs "Absent"

Null conflates two meanings: "value not set" and "value unknown/error". Use types to distinguish:

```
// DON'T: Null means "unknown"
public String getUserTheme(UserId id) {
    Theme theme = findTheme(id);
    return theme != null ? theme.name() : null;  // What does null mean?
}
```

```java
// DO: Option distinguishes "not set" from "error"
public Option<Theme> getUserTheme(UserId id) {
    return findTheme(id);  // none() = not set, some(theme) = set
}

// DO: Result distinguishes "not found" from "error"
public Result<Theme> getRequiredTheme(UserId id) {
    return findTheme(id)
        .toResult(ThemeError.NotFound.INSTANCE);
}
```

Never Return Null from Business Logic

Business logic always uses typed returns:

```java
// DON'T: Returning null from business logic
public User enrichUser(User user) {
    Profile profile = loadProfile(user.id());
    if (profile == null) {
        return null;  // Don't return null!
    }
    return user.withProfile(profile);
}

// DO: Using Option
public Option<User> enrichUser(User user) {
    return loadProfile(user.id())  // Returns Option<Profile>
        .map(profile -> user.withProfile(profile));
}

// DO: Using Result if enrichment can fail
public Result<User> enrichUser(User user) {
    return loadProfile(user.id())
        .toResult(ProfileError.NotFound.INSTANCE)
        .map(profile -> user.withProfile(profile));
}
```

Null Policy Summary

Context	Null Usage	Correct Approach
Return values from JBCT code	Never	Use `Option<T>`

Context	Null Usage	Correct Approach
Parameters between JBCT components	Never	Use `Option<T>` or required types
Wrapping external API returns	Allowed	`Option.option(nullable)` immediately
Writing to nullable DB columns	Allowed	`.orElse(null)` at write boundary
Test inputs for validation	Allowed	Test null rejection
"Unknown" or "absent" semantics	Never	Use `Option<T>` or `Result<T>`

Core Principle: Null exists only at system boundaries (adapters). Inside JBCT code, absence is represented by `Option.none()`, never null.

Why This Matters: - **Mental Overhead**: No defensive null checks in business logic - **Reliability**: Compiler enforces null handling at boundaries - **Complexity**: Clear semantics - `Option.none()` vs null confusion eliminated

---

Error Recovery Patterns

So far we've focused on error **handling** - how to represent and propagate failures through `Result` and `Promise`. But real systems need **recovery** - providing fallback values, retrying operations, or gracefully degrading when errors occur.

Providing Fallback Values

When an operation fails, you can substitute a default value using `.or()`:

```
// Provide a literal fallback value
public Theme getUserTheme(UserId userId) {
    return loadTheme(userId)  // Returns Option<Theme>
        .or(Theme.DEFAULT);   // Use DEFAULT if none
}

// Lazy evaluation with supplier
public Config getConfig(String key) {
    return loadFromCache(key)  // Returns Option<Config>
        .or(() -> loadFromDefaults(key))  // Only evaluated if cache miss
        .or(Config.EMPTY);  // Final fallback
}
```

The `.or()` method works with `Option`, `Result`, and `Promise`:

```
// Option<T> - provide value if empty
option.or(defaultValue)
option.or(() -> computeDefault())

// Result<T> - provide value if failure
result.or(defaultValue)
result.or(() -> computeDefault())

// Promise<T> - provide value if failure (async)
promise.or(defaultValue)
promise.or(() -> computeDefault())
```

Fallback to Alternative Operations

Use `.orElse()` when you want to try an alternative operation that returns the same monadic type:

```
// Try cache, then database, then in-memory
public Promise<User> findUser(UserId id) {
    return cacheRepository.find(id)      // Promise<User>
                          .orElse(databaseRepository.find(id))  // Try DB if cache fails
                          .orElse(Promise.success(User.GUEST)); // Final fallback
}

// Result version
public Result<Config> loadConfig(String key) {
    return loadFromFile(key)      // Result<Config>
        .orElse(loadFromEnv(key))  // Try environment if file fails
        .orElse(Result.success(Config.DEFAULT));
}
```

The difference: - `.or(value)` - substitute a plain value - `.orElse(M<T>)` - substitute with another `Result`/`Promise`/`Option`

Recovering from Specific Failures

Use `.recover()` when you want to transform failures into successes based on the error:

```
public Result<User> findUserOrGuest(UserId id) {
    return userRepository.find(id)
        .recover(this::recoverUserErrors);
}

private User recoverUserErrors(Cause cause) {
    return switch (cause) {
        case UserError.NotFound ignored -> User.GUEST;
        case UserError.DatabaseError ignored -> User.OFFLINE;
        default -> throw new IllegalStateException("Unexpected error: " + cause);
```

```
    };
}
```

Promise version with async recovery:

```
public Promise<Order> processOrder(OrderRequest request) {
    return paymentService.charge(request)
        .recover(this::recoverPaymentErrors);
}

private Promise<Order> recoverPaymentErrors(Cause cause) {
    return switch (cause) {
        case PaymentError.InsufficientFunds ignored ->
            splitPaymentService.process(request);  // Async alternative
        case PaymentError.TemporaryFailure ignored ->
            Promise.success(request).flatMap(paymentService::charge);  // Retry once
        default -> cause.promise();  // Can't recover
    };
}
```

Graceful Degradation Pattern

Combine recovery with feature detection:

```
public Promise<Dashboard> loadDashboard(UserId userId) {
    return Promise.all(loadUserProfile(userId),
                       loadRecentOrders(userId).or(List.of()),
                       loadRecommendations(userId).or(List.of()))
                  .map(Dashboard::new);
}
```

Here, profile is **required** (must succeed), but orders and recommendations gracefully degrade to empty lists if unavailable. The dashboard still loads with reduced functionality.

Real-World Example: Configuration with Fallbacks

```
public interface LoadConfig {
    record Config(String apiUrl, int timeout, boolean retryEnabled) {}

    Promise<Config> load();

    static LoadConfig loadConfig(LoadFromFile loadFile,
                                 LoadFromEnv loadEnv) {
        return () -> loadFile.apply()
```

```
                                .orElse(loadEnv.apply())
                                .or(() -> new Config("https://api.default.com", 30, true));
    }
}
```

This tries file configuration, falls back to environment variables, and finally uses hard-coded defaults. Each layer adds resiliency.

Recovery Patterns Summary

Pattern	Method	Use When
Default value	`.or(value)`	Fixed fallback available
Lazy default	`.or(() -> compute())`	Fallback is expensive to create
Alternative operation	`.orElse(M<T>)`	Try another source/service
Conditional recovery	`.recover(cause -> ...)`	Transform specific errors to success
Graceful degradation	`.or(emptyValue)`	Feature optional, show partial data

When NOT to Recover

Don't use recovery to hide genuine errors:

```
// DON'T: Swallowing errors silently
loadCriticalData(id)
    .or(Data.EMPTY)  // User never knows load failed!

// DO: Let critical errors propagate
loadCriticalData(id)  // Caller handles the error
```

Recovery is for **expected** failures where degradation makes sense (cache miss, optional feature unavailable). Critical errors should propagate so callers can handle them appropriately.

Why This Matters: - **Reliability**: Systems stay operational despite partial failures - **User Experience**: Graceful degradation better than total failure - **Complexity**: Clear recovery strategy, no hidden try-catch blocks

---

Key Takeaways

1. **Null only at boundaries** - Adapters wrap external nulls in `Option`

2. **Option.option(nullable)** - Immediate conversion at entry point
3. **Never return null** - Use `Option<T>` for optional values
4. **.or() for defaults** - Provide fallback values
5. **.orElse() for alternatives** - Try another operation
6. **.recover() for conditional** - Transform specific failures to success

---

Exercises

See Appendix B for exercises on: - Exercise 2.5: Recovery patterns - Exercise 5.2: Graceful degradation scenarios

---

What's Next

Chapter 7 introduces the basic structural patterns - Leaf, Condition, and Iteration - that handle 80% of your daily coding needs.

Chapter 7: Basic Patterns & Structure

What You'll Learn

- When to extract functions (mechanical rules, not judgment calls)
- How to keep code at a single level of abstraction
- The three basic patterns: Leaf, Condition, Iteration
- Zone-based naming conventions

Prerequisites: Chapter 6: Null Policy & Error Recovery

Single Pattern Per Function

Every function implements exactly one pattern from a fixed catalog: Leaf, Sequencer, Fork-Join, Condition, or Iteration. (Aspects are the exception—they decorate other patterns.)

Why? Cognitive load. When reading a function, you should recognize its shape immediately. If it's a Sequencer, you know it chains dependent steps linearly. If it's Fork-Join, you know it runs independent operations and combines results. Mixing patterns within a function creates mixed abstraction levels and forces readers to hold multiple mental models simultaneously.

This rule has a mechanical benefit: it makes refactoring deterministic. When a function grows beyond one pattern, you extract the second pattern into its own function. There's no subjective judgment about "is this too complex?" - if you're doing two patterns, split it.

Why by criteria: - **Mental Overhead**: One pattern per function means immediate recognition - no mental model switching (+2) - **Complexity**: Mechanical refactoring rule eliminates subjective debates about "too complex" (+2) - **Design Impact**: Forces proper abstraction layers - no mixing orchestration with computation (+2)

The Pain of Mixed Patterns

Before showing violations, understand the **concrete problems** that mixing patterns causes:

Testing becomes brittle:

```
// Mixed pattern function
public Result<Report> generateReport(ReportRequest request) {
    // Validates, fetches in parallel, computes, formats - all in one
}

// To test, you need:
// 1. Valid request setup
// 2. Mock for fetchUserData
// 3. Mock for fetchSalesData
// 4. Verify computeMetrics logic
// 5. Verify formatReport logic
// Can't test parts independently - it's all or nothing
```

Debugging is unclear:

```
// Stack trace points to generateReport() line 45
// But which step failed? Validation? Fork-Join fetch? Compute? Format?
// You can't tell without stepping through the whole function
```

Reuse is impossible:

```
// Another use case needs the same Fork-Join fetch logic
// But it's buried inside generateReport()
// Can't reuse it - have to copy-paste or refactor
```

With single pattern per function: - Each function is independently testable - Patterns are reusable across use cases - Failures are localized (stack trace says "fetchReportData failed") - Structure is predictable and scannable

Example Violation

```
// DON'T: Mixing Sequencer and Fork-Join
public Result<Report> generateReport(ReportRequest request) {
    return ValidRequest.validRequest(request)
                       .flatMap(valid -> {
                           // Sequencer starts here
                           var userData = fetchUserData(valid.userId());
                           var salesData = fetchSalesData(valid.dateRange());

                           // Wait, now we're doing Fork-Join?
                           return Result.all(userData, salesData)
                                        .flatMap((user, sales) -> computeMetrics(user, sa
                                        .flatMap(this::formatReport);  // Back to Sequenc
                       });
}
```

This function starts as a Sequencer (validate -> fetch -> compute -> format), but `fetchUserData` and `fetchSalesData` are independent, so we suddenly do a Fork-Join in the middle. Mixed abstraction levels. Hard to test. Unclear at a glance what the

function does.

Corrected

```
// DO: One pattern per function
public Result<Report> generateReport(ReportRequest request) {
    return ValidRequest.validRequest(request)
                       .flatMap(this::fetchReportData)
                       .flatMap(this::computeMetrics)
                       .flatMap(this::formatReport);
}

private Result<ReportData> fetchReportData(ValidRequest request) {
    // This function is a Fork-Join
    return Result.all(fetchUserData(request.userId()),
                      fetchSalesData(request.dateRange()))
                 .map(ReportData::new);
}
```

Now `generateReport` is a pure Sequencer (validate -> fetch -> compute -> format), and `fetchReportData` is a pure Fork-Join. Each function has one clear job.

Mechanical Refactoring

If you're writing a Sequencer and realize step 3 needs to do a Fork-Join internally, extract step 3 into its own function that implements Fork-Join. The original Sequencer stays clean.

Rule of thumb: One pattern per function. If you see two patterns, extract one.

Single Level of Abstraction

The rule: No complex logic inside lambdas. Lambdas passed to `map`, `flatMap`, and similar combinators may contain only: - Method references (e.g., `Email::new`, `this::processUser`) - Single method calls with parameter forwarding (e.g., `param -> someMethod(outerParam, param)`)

Why? Lambdas are composition points, not implementation locations. When you bury logic inside a lambda, you hide abstraction levels and make the code harder to read, test, and reuse. Extract complex logic to named functions - the name documents intent, the function becomes testable in isolation, and the composition chain stays flat and readable.

Why by criteria: - **Mental Overhead**: Flat composition chains scan linearly - no descending into nested logic (+2) - **Business/Technical Ratio**: Named functions document intent; anonymous lambdas hide it (+2) - **Complexity**: Each function testable in isolation; buried lambda logic requires testing through container (+2)

Anti-Pattern

```
// DON'T: Complex logic inside lambda
return fetchUser(userId)
    .flatMap(user -> {
        if (user.isActive() && user.hasPermission("admin")) {
            return loadAdminDashboard(user)
                .map(dashboard -> {
                    var summary = new Summary(
                        dashboard.metrics(),
                        dashboard.alerts().stream()
                            .filter(Alert::isUrgent)
                            .toList()
                    );
                    return new Response(user, summary);
                });
        } else {
            return AccessError.InsufficientPermissions.INSTANCE.promise();
        }
    });
```

This lambda contains: conditional logic, nested map, stream processing, object construction. Mixed abstraction levels. Hard to test. Hard to read.

Correct Approach

```
// DO: Extract to named functions
return fetchUser(userId)
    .flatMap(this::checkAdminAccess)
    .flatMap(this::loadAdminDashboard)
    .map(this::buildResponse);

private Promise<User> checkAdminAccess(User user) {
    return isActiveAdministrator(user)
            ? Promise.success(user)
            : AccessError.InsufficientPermissions.INSTANCE.promise();
}

private boolean isActiveAdministrator(User user) {
    return user.isActive() && user.hasPermission("admin");
}

private Response buildResponse(Dashboard dashboard) {
    var urgentAlerts = filterUrgentAlerts(dashboard.alerts());
    var summary = new Summary(dashboard.metrics(), urgentAlerts);
    return new Response(dashboard.user(), summary);
}
```

```
private List<Alert> filterUrgentAlerts(List<Alert> alerts) {
    return alerts.stream()
                 .filter(Alert::isUrgent)
                 .toList();
}
```

Now the top-level chain reads linearly: fetch -> check access -> load dashboard -> build response. Each step is named, testable, and at a single abstraction level.

Allowed in Lambdas

Method references:

```
.map(Email::new)
.flatMap(this::saveUser)
.map(User::id)
```

Single method call with parameter forwarding:

```
.flatMap(user -> checkPermissions(requiredRole, user))
.map(order -> calculateTotal(taxRate, order))
```

Forbidden in Lambdas

- Conditionals (`if`, ternary, `switch`)
- Try-catch blocks
- Multi-statement blocks
- Object construction beyond simple factory calls

No ternaries (they are the Condition pattern):

```
// DON'T: Ternary in lambda
.flatMap(user -> user.isPremium()
    ? applyPremiumDiscount(user)
    : applyStandardDiscount(user))

// DO: Extract to named function
.flatMap(this::applyApplicableDiscount)

private Result<Discount> applyApplicableDiscount(User user) {
    return user.isPremium()
        ? applyPremiumDiscount(user)
        : applyStandardDiscount(user);
}
```

The Three-Zone Framework

Maintaining "single level of abstraction" becomes mechanical when you think of your codebase in three distinct zones, each with its own vocabulary:

Zone 1 (Use Case Level) - High-level business goals: - `RegisterUser.execute()`, `ProcessOrder.execute()`, `LoadDashboard.execute()` - One Zone 1 function per use case - the entry point

Zone 2 (Orchestration Level) - Coordinating steps that break down the goal: - Step interfaces in Sequencer/Fork-Join patterns - Verbs: `validate`, `process`, `handle`, `transform`, `apply`, `check`, `load`, `save`, `manage`, `configure`, `initialize` - Examples: `ValidateInput.apply()`, `ProcessPayment.apply()`, `HandleNotification.apply()`

Zone 3 (Implementation Level) - Concrete technical operations: - Business and adapter leaves - Verbs: `get`, `set`, `fetch`, `parse`, `calculate`, `convert`, `hash`, `format`, `encode`, `decode`, `extract`, `split`, `join`, `log`, `send`, `receive`, `read`, `write`, `add`, `remove` - Examples: `hashPassword()`, `parseJson()`, `fetchFromDatabase()`, `calculateTax()`

The key insight: Functions at each zone should only call functions from the same zone or one level down. Zone 2 functions call other Zone 2 steps or Zone 3 leaves. Zone 3 leaves perform atomic operations. This creates natural layering.

Example - Maintaining zone consistency:

```
// GOOD - All steps at Zone 2 (orchestration level)
public Promise<Response> execute(Request request) {
    return ValidRequest.validRequest(request)              // Zone 2: validate
                       .async()
                       .flatMap(this::processCredentials)  // Zone 2: process
                       .flatMap(this::saveUser)            // Zone 2: save
                       .flatMap(this::sendConfirmation);   // Zone 2: send
}

// BAD - Mixing Zone 2 and Zone 3 in same chain
public Promise<Response> execute(Request request) {
    return ValidRequest.validRequest(request)              // Zone 2
                       .async()
                       .flatMap(this::hashPassword)        // Zone 3 - too specific!
                       .flatMap(this::saveUser)            // Zone 2
                       .flatMap(this::fetchConfirmToken);  // Zone 3 - too specific!
}
```

In the bad example, `hashPassword` and `fetchConfirmToken` are Zone 3 operations (concrete technical details). They should be wrapped in Zone 2 steps like `processCredentials` and `sendConfirmation`.

The Stepdown Rule Test

A simple way to verify your abstraction levels: read your code aloud by adding "to" before each function. It should sound like a natural narrative:

```
// Reading this aloud:
// "To execute, we validate the request,
//  then we process payment,
//  then we send confirmation."
return ValidRequest.validRequest(request)
                   .async()
                   .flatMap(this::processPayment)
                   .flatMap(this::sendConfirmation);
```

If adding "to" makes it sound awkward or overly detailed ("to hash password, then to save to database, then to fetch from cache"), you're mixing abstraction levels.

Pattern: Leaf

Definition: A **Leaf** (an atomic operation with no substeps) is the smallest unit of processing - a function that does one thing and has no internal steps. It's either a business leaf (pure computation) or an adapter leaf (I/O or side effects).

Rationale (by criteria): - **Mental Overhead**: Atomic operations have no internal steps to track - immediate comprehension (+2) - **Business/Technical Ratio**: Business leaves are pure domain logic; adapter leaves isolate technical concerns (+2) - **Complexity**: Single responsibility per leaf - no hidden interactions (+2) - **Reliability**: Pure business leaves are deterministic and easily testable (+1)

Business Leaves

Business leaves are pure functions that transform data or enforce business rules:

```
// Simple calculation leaf
public static Price calculateDiscount(Price original, Percentage rate) {
    return original.multiply(rate);
}

// Domain rule enforcement leaf
public static Result<Unit> checkInventory(Product product, Quantity requested) {
    return product.availableQuantity().isGreaterThanOrEqual(requested)
        ? Result.unitResult()
        : InsufficientInventory.cause(product.id(), requested);
}

// Data transformation leaf
public static OrderSummary toSummary(Order order) {
    return new OrderSummary(order.id(),
                            order.totalAmount(),
                            order.items().size());
}
```

If there's no I/O and no side effects, it's a business leaf. Keep each leaf focused on one transformation or one business rule.

Adapter Leaves

Adapter leaves integrate with external systems: databases, HTTP clients, message queues, file systems. They map foreign errors to domain Causes.

```
public interface UserRepository {
    Promise<Option<User>> findByEmail(Email email);
}

// Adapter leaf implementation
class PostgresUserRepository implements UserRepository {
    private final DataSource dataSource;

    public Promise<Option<User>> findByEmail(Email email) {
        return Promise.lift(RepositoryError.DatabaseFailure::new,
                            () -> {
                                try (var conn = dataSource.getConnection();
                                     var stmt = conn.prepareStatement(
                                         "SELECT * FROM users WHERE email = ?")) {

                                    stmt.setString(1, email.value());
                                    var rs = stmt.executeQuery();

                                    return rs.next() ? mapUser(rs) : null;
                                }
                            }).map(Option::option);
    }

    private User mapUser(ResultSet rs) throws SQLException {
        return new User(/* ... */);
    }
}
```

The adapter catches `SQLException` and wraps it in `RepositoryError.DatabaseFailure`, a domain `Cause`. Callers never see `SQLException`.

Thread Safety

Leaf operations are **thread-safe through confinement** - each invocation operates independently with its own local state. Mutable local variables (accumulators, builders, working objects) are safe within a leaf because they never escape the function scope. Input parameters must be treated as read-only.

Placement Rules

If a leaf is only used by one caller, keep it nearby (same file, same package).

If it's reused, move it immediately to the nearest `shared` package. Don't defer - tech debt accumulates when shared code stays in wrong locations.

Pattern: Condition

Definition: Condition represents branching logic based on data. The key: express conditions as values, not control-flow side effects. Keep branches at the same abstraction level.

Rationale (by criteria): - **Mental Overhead**: Conditions as expressions - evaluates to single value, not control flow scatter (+2) - **Business/Technical Ratio**: Branch logic mirrors domain rules, not imperative jumps (+2) - **Complexity**: Same abstraction level per branch - prevents tangled logic (+2) - **Reliability**: Type-checked branches ensure all cases return compatible types (+2)

Simple Conditional

```
// DO: Condition as expression returning the monad
Result<Discount> calculateDiscount(Order order) {
    return order.isPremiumUser()
        ? premiumDiscount(order)      // returns Result<Discount>
        : standardDiscount(order);    // returns Result<Discount>
}
```

Both branches return the same type (`Result<Discount>`), so the ternary is just choosing which function to call. No mixed abstractions.

Pattern Matching

Using Java's switch expressions:

```
Result<ShippingCost> calculateShipping(Order order, ShippingMethod method) {
    return switch (method) {
        case STANDARD -> standardShipping(order);
        case EXPRESS -> expressShipping(order);
        case OVERNIGHT -> overnightShipping(order);
    };
}
```

Each case returns `Result<ShippingCost>`. The switch expression evaluates to a single result.

Nested Conditions

Avoid deep nesting by extracting subdecisions into named functions:

```
// DON'T: Nested ternaries
return user.isPremium()
    ? (order.total().greaterThan(THRESHOLD)
        ? largeOrderPremiumDiscount(order)
        : smallOrderPremiumDiscount(order))
    : (order.total().greaterThan(THRESHOLD)
        ? largeOrderStandardDiscount(order)
        : smallOrderStandardDiscount(order));
```

Extract:

```
// DO: Extract nested logic
Result<Discount> calculateDiscount(User user, Order order) {
    return user.isPremium()
        ? premiumDiscount(order)
        : standardDiscount(order);
}

private Result<Discount> premiumDiscount(Order order) {
    return order.total().greaterThan(THRESHOLD)
        ? largeOrderPremiumDiscount(order)
        : smallOrderPremiumDiscount(order);
}

private Result<Discount> standardDiscount(Order order) {
    return order.total().greaterThan(THRESHOLD)
        ? largeOrderStandardDiscount(order)
        : smallOrderStandardDiscount(order);
}
```

Now each function has one level of branching. Much clearer.

Condition with Monads

Use `filter` for cleaner composition when applicable:

```
// DON'T: Ternary in lambda
return fetchUser(userId)
    .flatMap(user -> user.isActive()
        ? processActiveUser(user)
        : UserError.InactiveAccount.INSTANCE.result());

// DO: Using filter (preferred)
return fetchUser(userId)
    .filter(User::isActive, UserError.InactiveAccount.INSTANCE)
    .flatMap(this::processActiveUser);
```

Pattern: Iteration

Definition: Iteration processes collections, streams, or recursive structures. Prefer functional combinators over explicit loops. Keep transformations pure.

Rationale (by criteria): - **Mental Overhead**: Declarative combinators state intent; imperative loops require tracing (+2) - **Business/Technical Ratio**: map/filter express business logic; loops are iteration mechanics (+2) - **Complexity**: Functional composition eliminates index management and loop state (+2) - **Reliability**: Pure transformations have no side effects - deterministic and testable (+2)

Mapping Collections

```
// Transforming a list of raw inputs to domain objects
Result<List<Email>> parseEmails(List<String> rawEmails) {
    return Result.allOf(rawEmails.stream()
                                 .map(Email::email)
                                 .toList());
}
```

`Result.allOf` aggregates a `List<Result<Email>>` into `Result<List<Email>>`. If any email is invalid, you get a `CompositeCause` with all failures.

Filtering and Transforming

```
List<ActiveUser> activeUsers(List<User> users) {
    return users.stream()
                .filter(User::isActive)
                .map(this::toActiveUser)
                .toList();
}

private ActiveUser toActiveUser(User user) {
    return new ActiveUser(user.id(), user.email());
}
```

Pure transformation, no side effects, returns `List<ActiveUser>` (type `T`, not `Result`, because this can't fail).

Async Iteration

When processing collections with async operations, decide between sequential and parallel:

Sequential:

```
// Process orders one at a time
Promise<List<Receipt>> processOrders(List<Order> orders) {
    Promise<List<Receipt>> result = Promise.success(List.of());

    for (Order order : orders) {
        result = result.flatMap2(this::appendReceipt, order);
    }

    return result;
}

private Promise<List<Receipt>> appendReceipt(List<Receipt> receipts, Order order) {
    return processOrder(order)
                .map(receipt -> appendToList(receipts, receipt));
}
```

Parallel (when orders are independent):

```
// Process orders in parallel
Promise<List<Receipt>> processOrders(List<Order> orders) {
    return Promise.allOf(orders.stream()
                               .map(this::processOrder)
                               .toList());
}
```

Use parallel when operations are independent.

Common Mistakes

DON'T mix side effects into stream operations:

```
// DON'T: Side effect in the map
users.stream()
    .map(user -> {
        logger.info("Processing user: {}", user.id());  // Side effect!
        return processUser(user);
    })
    .toList();
```

DON'T use imperative loops when combinators exist:

```
// DON'T: Imperative accumulation
List<Result<Email>> results = new ArrayList<>();
for (String raw : rawEmails) {
    results.add(Email.email(raw));
}

// DO: Declarative collection
Result<List<Email>> emails = Result.allOf(
```

```
    rawEmails.stream().map(Email::email).toList()
);
```

Naming Conventions

Consistent naming reduces cognitive load and makes code self-documenting.

Factory Method Naming

Factories are always named after their type, lowercase-first (camelCase):

```
Email.email("user@example.com")
Password.password("Secret123")
AccountId.accountId("ACC-001")
UserId.userId(raw)
```

This pattern is grep-friendly: searching for `Email.email` finds all email construction sites.

Validated Input Naming

Use the `Valid` prefix (not `Validated`) for types representing validated inputs:

```
// DO: Use Valid prefix
record ValidRequest(Email email, Password password) {
    static Result<ValidRequest> validRequest(Request raw) { ... }
}

// DON'T: Use Validated prefix (too verbose)
record ValidatedRequest(...)
```

Zone-Based Naming Vocabulary

Zone 2 Verbs (Step Interfaces - Orchestration):

Verb	When to Use	Example
`validate`	Checking rules/constraints	`ValidateInput`
`process`	Transforming or interpreting data	`ProcessPayment`
`handle`	Coordinating reactions to events	`HandleRefund`
`load`	Retrieving data for use	`LoadUserProfile`
`save`	Persisting changes	`SaveOrder`
`check`	Verifying conditions	`CheckInventory`

Zone 3 Verbs (Leaves - Implementation):

Verb	Typical Use	Example
`get`	Retrieve a value	`getTimestamp()`
`fetch`	Pull from external source	`fetchWeatherData()`
`parse`	Break down structured input	`parseJson()`
`calculate`	Perform computation	`calculateTax()`
`hash`	Cryptographic transformation	`hashPassword()`
`format`	Build structured output	`formatDate()`
`send`	Transmit over network	`sendEmail()`

Test Naming

Follow the pattern: `methodName_outcome_condition`

```
void validRequest_succeeds_forValidInput()
void validRequest_fails_forInvalidEmail()
void execute_succeeds_forValidInput()
void execute_fails_whenEmailAlreadyExists()
```

Key Takeaways

1. **Single Pattern Per Function** - One pattern per function, extract if you see two
2. **Single Level of Abstraction** - No complex logic in lambdas, only method references or simple forwarding
3. **Leaf** - Atomic unit: pure computation (business) or I/O (adapter)
4. **Condition** - Branching as values, same type from all branches
5. **Iteration** - Functional combinators over collections, pure transformations
6. **Zone-based naming** - Use appropriate verbs for each abstraction level

---

Exercises

See Appendix B for exercises on: - Exercise 3.1: Pattern identification - Exercise 3.2: Leaf extraction - Exercise 3.4: Zone-based naming

---

What's Next

Chapter 8 covers advanced patterns - Sequencer, Fork-Join, and Aspects - that compose these basic patterns into sophisticated workflows.

Chapter 8: Advanced Patterns

What You'll Learn

- **Sequencer**: The workhorse pattern for chaining dependent steps
- **Fork-Join**: Parallel composition for independent operations
- **Aspects**: Adding cross-cutting concerns without mixing responsibilities

Prerequisites: Chapter 7: Basic Patterns & Structure

Pattern: Sequencer

Definition: A Sequencer chains dependent steps linearly using `map` and `flatMap`. Each step's output feeds the next step's input. This is the primary pattern for use case implementation.

Rationale (by criteria): - **Mental Overhead**: Linear flow, 2-5 steps fits short-term memory capacity - predictable structure (+3) - **Business/Technical Ratio**: Steps mirror business process language - reads like requirements (+3) - **Complexity**: Fail-fast semantics, each step isolated and testable (+2) - **Design Impact**: Forces proper step decomposition, prevents monolithic functions (+2)

The 2-5 Rule

A Sequencer should have 2 to 5 steps. Fewer than 2, and it's probably just a Leaf. More than 5, and it needs decomposition—extract sub-sequencers or group steps.

> The rule is intended to limit local complexity. It is derived from the average size of short-term memory - 7 +/- 2 elements.

Domain requirements take precedence: Some functions inherently require more steps because the domain demands it. Value object factories may need multiple validation and normalization steps to ensure invariants. Fork-Join patterns may need to aggregate 6+ independent results because that's what the domain requires. Don't artificially fit domain logic into numeric rules. The 2-5 guideline helps you recognize when to consider refactoring, but domain semantics always win.

Sync Example

```
public interface ProcessOrder {
    record Request(String orderId, String paymentToken) {}
```

```
    record Response(OrderConfirmation confirmation) {}

    Result<Response> execute(Request request);

    interface ValidateInput {
        Result<ValidRequest> apply(Request raw);
    }
    interface ReserveInventory {
        Result<Reservation> apply(ValidRequest req);
    }
    interface ProcessPayment {
        Result<Payment> apply(Reservation reservation);
    }
    interface ConfirmOrder {
        Result<Response> apply(Payment payment);
    }

    static ProcessOrder processOrder(ValidateInput validate,
                                     ReserveInventory reserve,
                                     ProcessPayment processPayment,
                                     ConfirmOrder confirm) {
        return request -> validate.apply(request)                // Step 1
                                  .flatMap(reserve::apply)       // Step 2
                                  .flatMap(processPayment::apply) // Step 3
                                  .flatMap(confirm::apply);      // Step 4
    }
}
```

Four steps, each a single-method interface. The `execute()` body reads top-to-bottom: validate -> reserve -> process payment -> confirm. Each step returns `Result<T>`, so we chain with `flatMap`. If any step fails, the chain short-circuits and returns the failure.

Async Example

Same structure, different types:

```
public Promise<Response> execute(Request request) {
    return ValidateInput.validate(request)              // returns Result<ValidInput>
                        .async()                        // lift to Promise<ValidInput>
                        .flatMap(reserve::apply)        // returns Promise<Reservation>
                        .flatMap(processPayment::apply) // returns Promise<Payment>
                        .flatMap(confirm::apply);       // returns Promise<Response>
}
```

Validation is synchronous (returns `Result`), so we lift it to `Promise` using `.async()`. The rest of the chain is async.

When to Extract Sub-Sequencers

If a step grows complex internally, extract it to its own interface with a nested structure. Suppose `processPayment` actually needs to: authorize card -> capture funds -> record transaction. That's three dependent steps - a Sequencer. Extract:

```
// Original step interface
interface ProcessPayment {
    Promise<Payment> apply(Reservation reservation);
}

// Implementation delegates to a sub-sequencer
interface CreditCardPaymentProcessor extends ProcessPayment {
    interface AuthorizeCard {
        Promise<Reservation> apply(Reservation reservation);
    }
    interface CaptureFunds {
        Promise<Reservation> apply(Reservation reservation);
    }
    interface RecordTransaction {
        Promise<Payment> apply(Reservation reservation);
    }

    static CreditCardPaymentProcessor creditCardPaymentProcessor(
            AuthorizeCard authorizeCard,
            CaptureFunds captureFunds,
            RecordTransaction recordTransaction) {
        return (reservation) -> authorizeCard.apply(reservation)
                                             .flatMap(captureFunds::apply)
                                             .flatMap(recordTransaction::apply);
    }
}
```

Now `CreditCardPaymentProcessor` is itself a Sequencer with three steps. The top-level use case remains a clean 4-step chain.

Thread Safety

Sequencer pattern is **thread-safe through sequential execution** - steps execute one after another, never in parallel. Each step is isolated and thread-confined. Mutable local state within individual steps is safe because steps don't overlap. Data passed between steps must be immutable.

Common Mistakes

DON'T nest logic inside flatMap:

```
// DON'T: Business logic buried in lambda
return validate.apply(request)
```

```
    .flatMap(valid -> {
        if (valid.isPremiumUser()) {
            return applyDiscount(valid)
                .flatMap(reserve::apply);
        } else {
            return reserve.apply(valid);
        }
    })
    .flatMap(processPayment::apply);
```

Extract:

```
// DO: Extract to named function
return validate.apply(request)
               .flatMap(this::applyDiscountIfEligible)
               .flatMap(reserve::apply)
               .flatMap(processPayment::apply);

private Result<ValidRequest> applyDiscountIfEligible(ValidRequest request) {
    return request.isPremiumUser()
        ? applyDiscount(request)
        : Result.success(request);
}
```

DON'T mix Fork-Join inside a Sequencer without extraction:

```
// DON'T: Suddenly doing Fork-Join mid-sequence
return validate.apply(request)
    .flatMap(valid -> {
        var userPromise = fetchUser(valid.userId());
        var productPromise = fetchProduct(valid.productId());
        return Promise.all(userPromise, productPromise)
            .flatMap((user, product) -> reserve.apply(user, product));
    })
    .flatMap(processPayment::apply);
```

Extract:

```
// DO: Extract Fork-Join to its own step
return validate.apply(request)
               .flatMap(this::fetchUserAndProduct)  // Fork-Join inside
               .flatMap(reserve::apply)
               .flatMap(processPayment::apply);

private Promise<ReservationInput> fetchUserAndProduct(ValidRequest request) {
    return Promise.all(fetchUser(request.userId()),
                       fetchProduct(request.productId()))
                  .map(ReservationInput::new);
}
```

Pattern: Fork-Join

Definition: Fork-Join (also known as Fan-Out-Fan-In) executes independent operations concurrently and combines their results. Use it when you have parallel work with no dependencies between branches.

Rationale (by criteria): - **Mental Overhead**: Parallel execution explicit in structure - no hidden concurrency (+2) - **Complexity**: Independence constraint acts as design validator - forces proper data organization (+3) - **Reliability**: Type system prevents dependent operations from being parallelized (+2) - **Design Impact**: Reveals coupling issues - dependencies surface as compile errors (+3)

Two Primary Flavors

1. Result.all(...) - Synchronous aggregation:

Not concurrent, just collects multiple Results:

```
// Validating multiple independent fields
Result<ValidRequest> validated = Result.all(Email.email(raw.email()),
                                            Password.password(raw.password()),
                                            AccountId.accountId(raw.accountId()))
                                       .flatMap(ValidRequest::new);
```

If all succeed, you get a tuple of values to pass to the combiner. If any fail, you get a `CompositeCause` containing all failures (not just the first).

2. Promise.all(...) - Parallel async execution:

```
// Running independent I/O operations in parallel
Promise<Dashboard> buildDashboard(UserId userId) {
    return Promise.all(userService.fetchProfile(userId),
                       orderService.fetchRecentOrders(userId),
                       notificationService.fetchUnread(userId))
                  .map(this::createDashboard);
}

private Dashboard createDashboard(Profile profile,
                                  List<Order> orders,
                                  List<Notification> notifications) {
    return new Dashboard(profile, orders, notifications);
}
```

All three fetches run concurrently. The Promise completes when all inputs complete successfully or fails immediately if any input fails.

Special Fork-Join Cases

1. `Promise.allOf(Collection<Promise<T>>)` - Resilient collection:

```
// Fetching data from dynamic number of sources
Promise<Report> generateSystemReport(List<ServiceId> services) {
    var healthChecks = services.stream()
                               .map(healthCheckService::check)
                               .toList();

    return Promise.allOf(healthChecks)
                  .map(this::createReport);
}

private Report createReport(List<Result<HealthStatus>> results) {
    var successes = results.stream()
                           .filter(Result::isSuccess)
                           .map(Result::value)
                           .toList();
    var failures = results.stream()
                          .filter(Result::isFailure)
                          .map(Result::cause)
                          .toList();
    return new Report(successes, failures);
}
```

Returns `Promise<List<Result<T>>>` - unlike `Promise.all()` which fails fast, `allOf()` waits for all promises to complete and collects both successes and failures.

2. `Promise.any(Promise<T>...)` - First-success wins:

```
// Racing multiple data sources
Promise<ExchangeRate> fetchRate(Currency from, Currency to) {
    return Promise.any(primaryRateProvider.getRate(from, to),
                       secondaryRateProvider.getRate(from, to),
                       fallbackRateProvider.getRate(from, to));
}
```

Returns the first successfully completed Promise, canceling remaining operations.

Independence and Thread Safety

Fork-Join has a crucial constraint: **all branches must be truly independent with immutable inputs**.

> **Caution:** Type-level independence is not the same as infrastructure-level independence. Operations that appear independent at the code level may conflict at the infrastructure level: - **Database locks:** Two queries may deadlock on shared rows - **Rate limits:** Parallel API calls may exhaust quotas - **Connection pools:** Parallel operations may compete for connections

- **Transactions:** Operations in the same transaction may have ordering requirements

Validate infrastructure constraints, not just type signatures.

Design Independence - When you try to write a Fork-Join and discover hidden dependencies, it reveals design issues: - Data redundancy: If branch A needs data from branch B, maybe that data should be provided upfront - Incorrect data organization: Dependencies signal that data is split across sources when it should be colocated - Missing abstraction: Hidden dependencies may indicate a missing concept

Thread Safety - Parallel execution requires immutable inputs: - All input data MUST be immutable - no shared mutable state between parallel branches - Local mutable state is safe - thread-confined accumulators, builders within each branch are fine - Results must be immutable - data returned from branches will be combined

Example design issue:

```
// DON'T: Logical dependency
Promise.all(fetchUserProfile(userId),      // Returns User
            fetchUserPreferences(userId))  // Needs User.timezone from profile!
```

Example thread safety violation:

```
// DON'T: Shared mutable state
private final DiscountContext context = new DiscountContext();  // Mutable

Promise<Result> calculate() {
    return Promise.all(applyBogo(cart, context),       // DATA RACE
                       applyPercentOff(cart, context))  // Both mutate context
                  .map(this::merge);
}

// DO: Immutable inputs
Promise<Result> calculate(Cart cart) {
    return Promise.all(applyBogo(cart),        // Immutable cart
                       applyPercentOff(cart))  // Immutable cart
                  .map(this::mergeDiscounts);
}
```

Common Mistakes

DON'T use Fork-Join when there are hidden dependencies:

```
// DON'T: These aren't actually independent
Promise.all(allocateInventory(orderId),   // Might lock inventory
            chargePayment(paymentToken))  // Should only charge if inventory succeeds
```

If inventory allocation fails, we've already charged the customer. Use a Sequencer.

DON'T ignore errors in Fork-Join branches:

```
// DON'T: Silently swallowing failures
Promise.all(fetchPrimary(id).recover(err -> Option.none()),  // Hides failure
            fetchSecondary(id).recover(err -> Option.none()))
```

Model the "best-effort" case explicitly with proper types.

Pattern: Aspects (Decorators)

Definition: Aspects are higher-order functions that wrap steps or use cases to add cross-cutting concerns - retry, timeout, logging, metrics - without changing business semantics.

Rationale (by criteria): - **Mental Overhead**: Cross-cutting concerns separated from business logic - clear responsibilities (+3) - **Business/Technical Ratio**: Business logic stays pure; technical concerns isolated in decorators (+3) - **Complexity**: Composable aspects via higher-order functions - no framework magic (+2) - **Design Impact**: Business logic independent of retry/metrics/logging - testable separately (+3)

Placement

Local concerns: Wrap individual steps when the aspect applies to just that step. Example: retry only on external API calls.

Cross-cutting concerns: Wrap the entire `execute()` method. Example: metrics for the whole use case.

Example: Retry Aspect on a Step

```
public interface FetchUserProfile {
    Promise<Profile> apply(UserId userId);
}

// Step implementation
class UserServiceClient implements FetchUserProfile {
    public Promise<Profile> apply(UserId userId) {
        return httpClient.get("/users/" + userId.value())
                         .map(this::parseProfile);
    }
}

// Applying a retry aspect at construction:
static ProcessUserData processUserData(..., UserServiceClient userServiceClient, ...) {
    var retryPolicy = RetryPolicy.builder()
                                 .maxAttempts(3)
                                 .backoff(exponential(100, 2.0))
```

```
                                    .build();

    return new processUserData(validateInput,
                                withRetry(retryPolicy, userServiceClient),  // Decorated
                                processData);
}
```

Example: Metrics Aspect on Use Case

```
public interface LoginUser {
    Promise<LoginResponse> execute(LoginRequest request);

    static LoginUser loginUser(...) {
        var rawUseCase = new loginUser(...);
        var metricsPolicy = MetricsPolicy.metricsPolicy("user_login");
        return withMetrics(metricsPolicy, rawUseCase);
    }
}
```

Composing Multiple Aspects

Order matters. Typical ordering (outermost to innermost): 1. Metrics/Logging (outermost - observe everything) 2. Timeout (global deadline) 3. CircuitBreaker (fail-fast if system is degraded) 4. Retry (per-attempt) 5. RateLimit (throttle requests) 6. Business logic (innermost)

```
var decoratedStep = withMetrics(metricsPolicy,
    withTimeout(timeoutPolicy,
        withCircuitBreaker(breakerPolicy,
            withRetry(retryPolicy, rawStep))));
```

Operational Semantics

Timeout Behavior: - **Logical timeout:** Promise resolves with `TimeoutError` after deadline - **Actual cancellation:** The underlying operation may continue running - **Rule:** Timeout doesn't guarantee resource cleanup—idempotent operations are safer

```
// Timeout resolves promise but underlying HTTP call may complete
promise.timeout(TimeSpan.seconds(5))  // Returns TimeoutError after 5s
       .onFailure(this::logTimeout);   // Operation may still finish
```

Retry Semantics: - **Idempotency required:** Retried operations must be safe to repeat - **State changes:** Non-idempotent operations (money transfers, order creation) need idempotency keys - **Backoff:** Exponential backoff prevents thundering herd

```
// Safe: idempotent read
withRetry(policy, () -> fetchUser(id))
```

```
// Unsafe without idempotency key: creates duplicate
withRetry(policy, () -> createOrder(request))  // DON'T

// Safe: idempotent write with key
withRetry(policy, () -> createOrder(request, idempotencyKey))  // DO
```

Composition Order Semantics:

Order	Meaning
Metrics → Timeout → Retry → Operation	Metrics count total time; timeout applies to all retries
Timeout → Metrics → Retry → Operation	Timeout per attempt; metrics count each retry
Retry → Timeout → Operation	Each attempt has own timeout

Rule: Define composition order based on what you want to observe and control. Document your choice.

Testing Aspects

Test aspects in isolation with synthetic steps. Use case tests remain aspect-agnostic:

```
// Aspect test (isolated)
@Test
void retryAspect_retriesOnFailure() {
    var failingStep = new FlakyStep(2);  // Fail times
    var retryPolicy = RetryPolicy.maxAttempts(3);
    var decorated = withRetry(retryPolicy, failingStep);

    var result = decorated.apply(input).await();

    assertTrue(result.isSuccess());
    assertEquals(3, failingStep.invocationCount());
}

// Use case test (aspect-agnostic)
@Test
void loginUser_success() {
    var useCase = LoginUser.loginUser(mockValidate, mockCheckCreds, mockGenerateToken);

    var result = useCase.execute(validRequest).await();

    assertTrue(result.isSuccess());
    // No assertions about retries, timeouts, etc.
}
```

Common Mistakes

DON'T mix aspect logic into business logic:

```
// DON'T: Retry logic inside the step
Promise<Profile> fetchProfile(UserId id) {
    return retryWithBackoff(() -> httpClient.get("/users/" + id.value()))
            .map(this::parseProfile);
}
```

Extract to an aspect decorator.

DO keep aspects composable and reusable:

```
static <I, O> Fn1<I, Promise<O>> withTimeout(TimeSpan timeout, Fn1<I, Promise<O>> step) {
    return input -> step.apply(input).timeout(timeout);
}

static <I, O> Fn1<I, Promise<O>> withRetry(RetryPolicy policy, Fn1<I, Promise<O>> step) {
    return input -> retryLogic(policy, () -> step.apply(input));
}
```

Key Takeaways

1. **Sequencer** - Chain dependent steps (2-5 rule), fail-fast semantics
2. **Fork-Join** - Parallel independent operations, strict immutability requirement
3. **Aspects** - Cross-cutting concerns as decorators, tested separately
4. **Independence validation** - Prevents Fork-Join mistakes; use Sequencer if unsure
5. **Pattern composition** - Sequencers call Fork-Joins, Aspects wrap both

---

Exercises

See Appendix B for exercises on: - Exercise 3.3: Pattern composition - Exercise 3.5: Aspect implementation

---

What's Next

Chapter 9 provides a comprehensive thread safety quick reference, consolidating all the thread safety rules across JBCT patterns.

Chapter 9: Thread Safety

What You'll Learn

- Core thread safety principles in JBCT
- Pattern-by-pattern safety guarantees
- Promise resolution semantics
- Common mistakes and how to avoid them

Prerequisites: Chapter 8: Advanced Patterns

Core Rules

Two principles ensure thread safety:

1. **Immutable at boundaries**: All data passed between functions (parameters, return values) must be immutable
2. **Thread confinement**: Mutable state is allowed within a single-threaded execution path (sequential patterns)

These rules mean you can use mutable local variables in sequential code, but any data shared across threads must be immutable.

Pattern-by-Pattern Safety Guarantees

Leaf (Sequential)

Thread-safe through sequential execution:

```
// SAFE: Mutable local state OK
public Result<Order> calculateTotal(Cart cart) {
    var total = 0.0;  // Mutable local variable
    for (Item item : cart.items()) {
        total += item.price();  // Sequential mutation
    }
    return Result.success(new Order(cart, total));
}
```

- Mutable local state confined to single thread

- Input (`cart`) is read-only
- Output (`Order`) is immutable

Sequencer (Sequential)

Thread-safe, mutable local state OK:

```
// SAFE: Each step runs sequentially
return ValidRequest.validRequest(request)
                   .async()
                   .flatMap(checkEmail::apply)    // Runs first
                   .flatMap(hashPassword::apply)  // Runs second
                   .flatMap(saveUser::apply);     // Runs third
```

- Steps execute in order, never concurrently
- Each step can use mutable local state
- Promise resolution is a synchronization point

Fork-Join (Parallel)

Requires strict immutability:

```
// SAFE: Immutable cart passed to both operations
Promise.all(applyBogo(cart),          // cart is immutable
            applyPercentOff(cart))    // cart is immutable
       .map(this::mergeDiscounts);

// UNSAFE: Shared mutable context creates data race
private final DiscountContext context = new DiscountContext();  // Mutable!
Promise.all(applyBogo(cart, context),     // Both access context
            applyPercentOff(cart, context))  // DATA RACE
       .map(this::merge);
```

- All parallel operations receive immutable inputs
- No shared mutable state between parallel operations
- Results merged after all complete

Condition (Sequential)

Thread-safe, no shared mutable state:

```
// SAFE: Only one branch executes
return user.isPremium()
    ? processPremium(user)
    : processBasic(user);
```

- Branches never execute concurrently
- Each branch can use mutable local state

Iteration (Sequential)

Thread-safe through sequential processing:

```
// SAFE: Stream processes sequentially
var results = items.stream()
                   .map(Item::validate)  // Sequential
                   .toList();
```

- Stream operations sequential by default
- Parallel streams require immutable inputs (use with caution)

Aspects (Inherits)

Safety depends on wrapped operation:

```
// SAFE: Aspect preserves underlying safety
var retried = withRetry(retryPolicy, sequentialStep);  // Still sequential
var retried = withRetry(retryPolicy, forkJoinStep);    // Still parallel
```

- Decorators don't change concurrency model
- Retry/timeout/metrics don't introduce shared state

Promise Resolution Thread Safety

Promise resolution has exactly-once semantics with built-in synchronization:

```
Promise<User> promise = Promise.promise();

// Thread 1
promise.resolve(Result.success(user));

// Thread 2
promise.resolve(Result.success(anotherUser));  // Ignored - already resolved
```

- First `resolve()` wins, subsequent calls ignored
- `flatMap`/`map` chains wait for resolution before executing
- Resolution acts as synchronization point for sequential chains

---

Common Mistakes

Mistake 1: Shared Mutable State in Fork-Join

```
// WRONG: Mutable list shared across parallel operations
private final List<String> errors = new ArrayList<>();
```

```
Promise.all(validateEmail(request).onFailure(e -> errors.add(e.message())),     // DATA R
            validatePassword(request).onFailure(e -> errors.add(e.message())))  // DATA R
       .map(this::process);
```

Fix: Use immutable results and combine after:

```
// CORRECT: Each operation returns its own result
Promise.all(validateEmail(request),
            validatePassword(request))
       .map((emailResult, passwordResult) -> combineResults(emailResult, passwordResult))
```

Mistake 2: Assuming Sequential Execution in Promise.all

```
// WRONG: Assuming order
Promise.all(incrementCounter(), incrementCounter())  // May execute concurrently!
```

Promise.all runs operations in parallel. If order matters, use Sequencer.

Mistake 3: Mutating Input Parameters

```
// WRONG: Mutating shared input
private Promise<Discount> applyDiscount(Cart cart) {
    cart.setSubtotal(cart.subtotal().subtract(discount));  // DATA RACE
    return Promise.success(new Discount(discount));
}

// CORRECT: Create new instances
private Promise<Discount> applyDiscount(Cart cart) {
    var discountAmount = calculateDiscountFor(cart);
    return Promise.success(new Discount(cart, discountAmount));  // Returns new data
}
```

Independence Validation Checklist

Use this checklist to verify parallel operations are truly independent:

- ☐ **No shared mutable state** - Operations don't modify shared objects
- ☐ **No execution order dependency** - Results same regardless of execution order
- ☐ **Immutable inputs** - All parameters are immutable or read-only
- ☐ **Side effects independent** - I/O operations don't conflict (different database rows, files, etc.)
- ☐ **No hidden coupling** - No shared caches, connection pools with state, etc.

If any checkbox fails, use Sequencer instead of Fork-Join.

When to Use Mutable State

Allowed: - Local variables in sequential patterns (Leaf, Sequencer, Condition, Iteration) - Builders constructing immutable objects - Accumulators in sequential loops

Forbidden: - Shared across parallel operations (Fork-Join) - Passed between use case steps - Returned from functions (return immutable copy instead)

Testing for Thread Safety

Mutable test fixtures are acceptable - tests execute sequentially:

```
// SAFE: Test-scoped mutable state
@Test
void execute_succeeds_forValidInput() {
    List<String> callLog = new ArrayList<>();  // Mutable, but test-scoped

    CheckEmail checkEmail = req -> {
        callLog.add("email");  // Sequential execution
        return Promise.success(req);
    };

    useCase.execute(request)
           .await()
           .onSuccess(response -> assertEquals(List.of("email"), callLog));
}
```

Tests are sequential, so mutable fixtures don't create races.

Quick Reference Table

Pattern	Execution	Local Mutable State	Input Requirements
Leaf	Sequential	Safe	Read-only
Sequencer	Sequential	Safe	Immutable between steps
Fork-Join	Parallel	Safe (confined)	Strictly immutable
Condition	Sequential	Safe	Read-only
Iteration	Sequential	Safe	Read-only
Aspects	Inherits	Inherits	Inherits

Key Takeaways

1. **Immutable at boundaries** - Parameters and return values always immutable
2. **Thread confinement** - Mutable local state safe in sequential patterns
3. **Fork-Join strictness** - All inputs must be immutable, no shared state
4. **Promise resolution** - Exactly-once, acts as synchronization point
5. **Independence validation** - Use checklist before parallelizing
6. **When in doubt** - Use Sequencer; premature parallelization causes subtle bugs

---

Exercises

See Appendix B for exercises on: - Exercise 3.6: Identifying thread safety issues - Exercise 3.7: Converting unsafe code to safe patterns

---

What's Next

Chapter 10 introduces the testing philosophy - evolutionary testing and integration-first approaches that work naturally with JBCT patterns.

Chapter 10: Testing Philosophy

What You'll Learn

- Why integration-first testing aligns with functional composition
- The evolutionary process: stub everything -> implement incrementally -> production-ready
- How to handle complex test inputs with builders and factories
- When you still need unit tests

Prerequisites: Chapter 9: Thread Safety

The Problem with Traditional Testing

Traditional Approach: Component-Focused

Most Java testing follows this pattern:

```
// Separate tests for each component
class ValidateInputTest {
    @Test void emailValidation() { /* ... */ }
    @Test void passwordValidation() { /* ... */ }
    // 10 tests
}

class CheckCredentialsTest {
    @Test void validCredentials() { /* ... */ }
    @Test void invalidCredentials() { /* ... */ }
    // 5 tests
}

class CheckAccountStatusTest {
    @Test void activeAccount() { /* ... */ }
    @Test void inactiveAccount() { /* ... */ }
    // 3 tests
}

class GenerateTokenTest {
    @Test void tokenGeneration() { /* ... */ }
    // 4 tests
```

```
}

// Total: 22 tests, never testing them TOGETHER
```

Problems: 1. **Doesn't test composition** - Steps work individually but fail when chained 2. **Doesn't test error propagation** - How do failures bubble through the chain? 3. **Doesn't test actual behavior** - Tests verify components, not use cases 4. **Brittle** - Interface changes break all tests, even when behavior unchanged 5. **False confidence** - All tests pass, production fails because integration untested

What We Actually Want to Test

When a user calls `UserLogin.execute(request)`, we care about:

- **Does the request get validated correctly?**
- **Do all steps execute in order?**
- **Does each step failure propagate correctly?**
- **Do branch conditions work as expected?**
- **Does the complete behavior match requirements?**

These are **integration questions**, not unit questions.

Philosophy: Integration-First Testing

The Core Principle

Test assembled use cases, not isolated components.

Your use case is a composition of steps. Test the composition. Stub only at adapter boundaries (database, HTTP, external services). Test all business logic together.

Why by criteria: - **Mental Overhead**: One test suite per use case, not per component (+2). Test names directly map to scenarios. - **Business/Technical Ratio**: Tests read like behavior specifications, not technical assertions (+3). - **Reliability**: Tests verify actual end-to-end behavior, not isolated fragments (+3). - **Complexity**: Fewer test contexts, clearer boundaries (business vs adapters) (+2).

The Three Testing Layers

1. Value Objects: Unit Tests (100% coverage)

Value objects are pure, isolated, and enforce invariants. Test them comprehensively:

```
class EmailTest {
    @ParameterizedTest
    @ValueSource(strings = {"bad", "no@domain", "@missing", "CAPS@TEST.COM"})
    void email_rejectsInvalidFormat(String raw) {
        Email.email(raw).onSuccess(Assertions::fail);
```

```
    }

    @Test
    void email_normalizesToLowercase() {
        Email.email("USER@EXAMPLE.COM")
             .onSuccess(email -> assertEquals("user@example.com", email.value()));
    }
}
```

Why unit test here? Value objects have zero dependencies. They're pure functions. Unit testing is natural.

2. Business Leaves: Unit Tests if Complex

Simple business leaves (single calculation, simple transformation) don't need isolated tests - they're covered by use case integration tests.

Complex business leaves (rich algorithms, many branches) deserve unit tests:

```
class PricingEngineTest {
    @Test void volumeDiscount_appliesAtThreshold() { /* ... */ }
    @Test void combinedDiscounts_stackCorrectly() { /* ... */ }
    @Test void edgeCases_handleGracefully() { /* ... */ }
}
```

Guideline: If a leaf has 3+ conditional branches or complex logic, write unit tests.

3. Use Cases: Integration Tests (Test Vectors)

The heart of your testing: test complete use case behavior with all steps assembled, only adapters stubbed.

```
class UserLoginTest {
    CheckCredentials mockCredentials;
    CheckAccountStatus mockStatus;
    GenerateToken mockToken;
    UserLogin useCase;

    @BeforeEach
    void setup() {
        mockCredentials = vr -> Result.success(new Credentials("user-1"));
        mockStatus = c -> Result.success(new Account(c.userId(), true));
        mockToken = acc -> Result.success(new Response("token-" + acc.userId()));
        useCase = UserLogin.userLogin(mockCredentials, mockStatus, mockToken);
    }

    @Test
    void execute_succeeds_forValidInput() {
        var request = new Request("john@example.com", "Valid123", null);

        useCase.execute(request)
```

```
                .onFailure(Assertions::fail)
                .onSuccess(response -> assertEquals("token-user-1", response.token()));
    }
}
```

This tests **real behavior**: validation -> credentials -> status -> token, with error propagation.

The Evolutionary Testing Process

Overview

Instead of writing tests after implementation, evolve them **alongside** implementation:

```
Phase 1: Stub Everything
    |
Phase 2: Implement & Test Validation
    |
Phase 3-N: Implement Steps Incrementally
    |
Final: Production-Ready
```

At each phase, **all tests remain green**. You're not breaking and fixing - you're growing.

Phase 1: Stub Everything

Goal: Establish test structure before implementing anything.

Step 1: Create use case interface with factory returning stub implementation:

```
public interface UserLogin {
    record Request(String email, String password, String referral) {}
    record Response(String token) {}

    Result<Response> execute(Request request);

    static UserLogin userLogin() {
        return request -> Result.success(new Response("stub-token"));
    }
}
```

Step 2: Write initial tests:

```
class UserLoginTest {
    @Test
    void execute_succeeds_forValidInput() {
        var useCase = UserLogin.userLogin();
```

```
        var request = new Request("john@example.com", "Valid123", null);

        useCase.execute(request)
               .onSuccess(response -> assertEquals("stub-token", response.token()));
    }
}
```

Phase 2: Implement Validation

Step 1: Add validated request with validation logic:

```
record ValidRequest(Email email, Password password, Option<ReferralCode> referral) {
    static Result<ValidRequest> validRequest(Request raw) {
        return Result.all(Email.email(raw.email()),
                          Password.password(raw.password()),
                          ReferralCode.referralCode(raw.referral()))
                     .map(ValidRequest::new);
    }
}
```

Step 2: Update factory to use validation:

```
static UserLogin userLogin() {
    return request -> ValidRequest.validRequest(request)
                                  .map(_ -> new Response("stub-token"));
}
```

Step 3: Add validation test vectors:

```
@Test
void execute_fails_forInvalidEmail() {
    var useCase = UserLogin.userLogin();
    var request = new Request("bad-email", "Valid123", null);

    useCase.execute(request)
           .onSuccess(Assertions::fail);
}

@Test
void execute_aggregatesMultipleErrors() {
    var useCase = UserLogin.userLogin();
    var request = new Request("bad", "weak", "invalid-ref");

    useCase.execute(request)
           .onSuccess(Assertions::fail)
           .onFailure(cause -> assertInstanceOf(Causes.CompositeCause.class, cause));
}
```

Phase 3-N: Continue Expanding

Repeat for each remaining step: - Add step interface - Update factory to accept dependency - Update existing test stubs - Add step failure scenarios

---

Handling Complex Input Objects

Test Data Builders

Fluent API for constructing test data:

```
public class TestData {
    public static RequestBuilder request() {
        return new RequestBuilder();
    }

    public static class RequestBuilder {
        private String email = "default@example.com";
        private String password = "DefaultValid123";
        private String referral = null;

        public RequestBuilder withEmail(String email) {
            this.email = email;
            return this;
        }

        public RequestBuilder withPassword(String password) {
            this.password = password;
            return this;
        }

        public Request build() {
            return new Request(email, password, referral);
        }
    }
}
```

Usage:

```
var request = TestData.request().build();
var invalidEmail = TestData.request().withEmail("bad").build();
```

Canonical Test Vectors

Pre-defined test data constants:

```
public interface TestVectors {
    Request VALID = new Request("user@example.com", "Valid123", null);
    Request INVALID_EMAIL = new Request("bad", "Valid123", null);
    Request WEAK_PASSWORD = new Request("user@example.com", "weak", null);
    Request MULTIPLE_ERRORS = new Request("bad", "weak", "invalid");
}
```

Which Approach to Use?

- **Canonical Vectors:** Simple use cases, few fields, limited variations
- **Factory Methods:** Medium complexity, systematic field variations
- **Builders:** Complex objects, many optional fields, many combinations

---

Key Takeaways

1. **Test composition, not components** - Use case is what matters
2. **Stub only adapters** - Database, HTTP, external services
3. **Evolve tests with implementation** - Always green, never break-and-fix
4. **Three layers** - Value objects (unit), complex leaves (unit), use cases (integration)
5. **Use test data utilities** - Builders, vectors, factories reduce boilerplate

---

Exercises

See Appendix B for exercises on: - Exercise 4.1: Building test data builders - Exercise 4.2: Evolutionary testing walkthrough

---

What's Next

Chapter 11 covers testing in practice - organizing large test suites, the complete RegisterUser example, and migrating from traditional unit testing.

Chapter 11: Testing in Practice

What You'll Learn

- How to organize large test counts without drowning in complexity
- What to test where (value objects, leaves, use cases, adapters)
- Complete worked example: RegisterUser from stub to production
- How to migrate from traditional unit testing

Prerequisites: Chapter 10: Testing Philosophy

Managing Large Test Counts

The "Problem"

Comprehensive testing generates many tests:

```
UserLoginTest: 35 tests
RegisterUserTest: 42 tests
UpdateProfileTest: 28 tests
```

This is not a problem - it's honest complexity. 35 tests = 35 real scenarios.

But we need organization to stay sane.

Strategy 1: Nested Test Classes

Group tests by scenario type:

```
class UserLoginTest {
    private UserLogin useCase;
    private CheckCredentials stubCreds;
    private CheckAccountStatus stubStatus;
    private GenerateToken stubToken;

    @BeforeEach
    void setup() {
        stubCreds = vr -> Result.success(new Credentials("user-1"));
        stubStatus = c -> Result.success(new Account(c.userId(), true));
        stubToken = acc -> Result.success(new Response("token-" + acc.userId()));
        useCase = UserLogin.userLogin(stubCreds, stubStatus, stubToken);
    }
```

```
    @Nested class HappyPath {
        @Test void execute_succeeds_forValidInput() { /* ... */ }
        @Test void execute_succeeds_withOptionalReferral() { /* ... */ }
    }

    @Nested class ValidationFailures {
        @Test void execute_fails_forInvalidEmail() { /* ... */ }
        @Test void execute_fails_forWeakPassword() { /* ... */ }
        @Test void execute_aggregatesMultipleErrors() { /* ... */ }
    }

    @Nested class StepFailures {
        @Test void execute_fails_whenCredentialsInvalid() { /* ... */ }
        @Test void execute_fails_whenAccountInactive() { /* ... */ }
        @Test void execute_fails_whenTokenGenerationFails() { /* ... */ }
    }

    @Nested class EdgeCases {
        @Test void execute_handlesNullReferral() { /* ... */ }
        @Test void execute_handlesEmptyStrings() { /* ... */ }
    }
}
```

Benefits: - IDE collapses nested classes - scan at high level - Clear categorization - find tests by scenario type - Shared setup per category - Test report groups meaningfully

Strategy 2: Parameterized Tests

Collapse similar tests into data-driven variants:

```
@ParameterizedTest
@ValueSource(strings = {"bad", "no@domain", "@missing", "user@", "CAPS@TEST.COM"})
void execute_fails_forInvalidEmail(String invalidEmail) {
    var request = TestData.request().withEmail(invalidEmail).build();

    useCase.execute(request)
           .onSuccess(Assertions::fail);
}

@ParameterizedTest
@CsvSource({
    "weak, TooShort",
    "alllowercase, NoUppercase",
    "ALLUPPERCASE, NoLowercase"
})
void execute_fails_forWeakPassword(String password, String expectedReason) {
    var request = TestData.request().withPassword(password).build();
```

```
    useCase.execute(request)
           .onSuccess(Assertions::fail)
           .onFailure(cause -> assertTrue(cause.message().contains(expectedReason)));
}
```

Reduces: 5 individual tests -> 1 parameterized test with 5 values.

Strategy 3: Test Organization in Files

Large use cases -> multiple test files:

```
usecase/
|-- userlogin/
    |-- UserLogin.java (implementation)
    |-- UserLoginValidationTest.java (validation scenarios)
    |-- UserLoginFlowTest.java (happy path + step failures)
    |-- UserLoginEdgeCasesTest.java (edge cases)
```

Guideline: Keep individual test files under 500 lines.

What to Test Where

Coverage Criteria by Component Type

1. Value Objects: 100% Coverage (Unit Tests)

```
class EmailTest {
    @Test void email_accepts_validFormat() { }
    @Test void email_rejects_missingAt() { }
    @Test void email_rejects_missingDomain() { }
    @Test void email_normalizesToLowercase() { }
    @Test void email_trimsWhitespace() { }
}
```

Why 100%? Value objects are pure, isolated, easy to test. No excuse for gaps.

2. Business Leaves: 100% if Complex, Skip if Trivial (Unit Tests)

Complex leaf (write unit tests):

```
class PricingEngine {
    Result<Price> calculatePrice(Order order) {
        // 50 lines, 8 branches, complex discounting logic
    }
}
// Deserves dedicated PricingEngineTest with 20+ tests
```

Trivial leaf (covered by integration):

```
static Price applyTax(Price base) {
    return new Price(base.amount().multiply(TAX_RATE));
}
// No dedicated test - covered when use case tested
```

Guideline: If leaf has 3+ branches or 20+ lines, write unit tests.

3. Use Cases: 90%+ Coverage (Integration Test Vectors)

```
class RegisterUserTest {
    // Happy path
    @Test void execute_succeeds_forValidInput() { }

    // Validation failures
    @Test void execute_fails_forInvalidEmail() { }
    @Test void execute_fails_forWeakPassword() { }

    // Step failures
    @Test void execute_fails_whenEmailAlreadyExists() { }
    @Test void execute_fails_whenPasswordHashingFails() { }

    // Branch conditions
    @Test void execute_sendsWelcomeEmail_forNewUser() { }
}
```

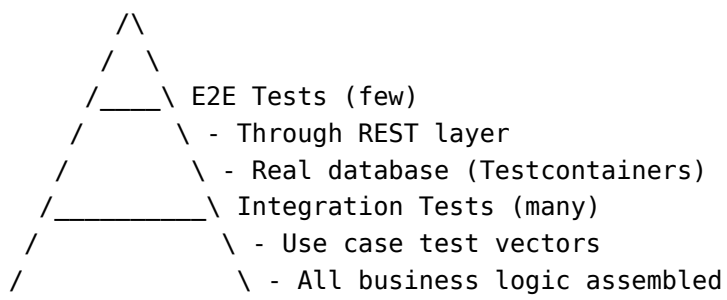
Why 90%+? Use cases are the behavior. Incomplete coverage = incomplete understanding.

4. Adapters: Success + Error Modes (Contract Tests)

```
class JooqUserRepositoryTest {
    @Test void findById_succeeds_whenUserExists() { }
    @Test void findById_returnsEmpty_whenUserNotFound() { }
    @Test void findById_wrapsException_whenDatabaseFails() { }
}
```

In use case tests: Stub adapters. Don't test database interaction 30 times.

The Testing Pyramid



```
     /               \ - Only adapters stubbed
    /_________________\ Unit Tests (some)
   /                   \ - Value objects (all)
  /                     \ - Complex business leaves
 /                       \ - Adapter contracts
/-------------------------\
```

This is inverted from traditional pyramid - we have MORE integration tests than unit tests.

Why? Because our business logic is composition. Testing fragments misses the point.

Complete Worked Example: RegisterUser

Phase 1: Stub Everything

```
public interface RegisterUser {
    record Request(String email, String password) {}
    record Response(String userId) {}

    Promise<Response> execute(Request request);

    static RegisterUser registerUser() {
        return request -> Promise.success(new Response("stub-user-id"));
    }
}
```

Test:

```
@Test
void execute_succeeds_forValidInput() {
    var useCase = RegisterUser.registerUser();
    var request = new Request("john@example.com", "Valid123");

    useCase.execute(request)
           .await()
           .onFailure(Assertions::fail)
           .onSuccess(response -> assertEquals("stub-user-id", response.userId()));
}
```

Phase 2: Add Validation

```
record ValidRequest(Email email, Password password) {
    static Result<ValidRequest> validRequest(Request raw) {
        return Result.all(Email.email(raw.email()),
                          Password.password(raw.password()))
```

```
                    .map(ValidRequest::new);
    }
}

static RegisterUser registerUser() {
    return request -> ValidRequest.validRequest(request)
                                  .async()
                                  .map(_ -> new Response("stub-user-id"));
}
```

Add validation tests:

```
@Test
void execute_fails_forInvalidEmail() {
    var request = new Request("bad", "Valid123");
    useCase.execute(request).await().onSuccess(Assertions::fail);
}
```

Phase 3-6: Add Steps Incrementally

After fully implementing:

```
static RegisterUser registerUser(CheckEmailUniqueness checkUniqueness,
                                 HashPassword hashPassword,
                                 SaveUser saveUser,
                                 SendWelcomeEmail sendEmail) {
    return request -> ValidRequest.validRequest(request)
                                  .async()
                                  .flatMap(checkUniqueness::apply)
                                  .flatMap(hashPassword::apply)
                                  .flatMap(saveUser::apply)
                                  .flatMap(sendEmail::apply)
                                  .map(user -> new Response(user.id()));
}
```

Final test suite:

```
class RegisterUserTest {
    @BeforeEach
    void setup() {
        CheckEmailUniqueness stubUniqueness = vr -> Promise.success(vr);
        HashPassword stubHash = vr -> Promise.success(new HashedPassword("hash"));
        SaveUser stubSave = hp -> Promise.success(new User("user-1"));
        SendWelcomeEmail stubEmail = u -> Promise.success(u);

        useCase = RegisterUser.registerUser(stubUniqueness, stubHash, stubSave, stubEmail
    }

    @Nested class HappyPath {
```

```
        @Test void execute_succeeds_forValidInput() { /* ... */ }
    }

    @Nested class ValidationFailures {
        @Test void execute_fails_forInvalidEmail() { /* ... */ }
        @Test void execute_fails_forWeakPassword() { /* ... */ }
    }

    @Nested class StepFailures {
        @Test void execute_fails_whenEmailExists() { /* ... */ }
        @Test void execute_fails_whenHashingFails() { /* ... */ }
        @Test void execute_fails_whenSaveFails() { /* ... */ }
        @Test void execute_fails_whenEmailSendingFails() { /* ... */ }
    }
}
```

Total: 8 integration tests, complete behavior coverage, only adapters stubbed.

Migration Guide: From Traditional to Evolutionary

Step 1: Add Integration Tests

Start by adding integration test vectors for key scenarios:

```
// Keep existing unit tests
class CheckCredentialsTest {
    @Test void validCredentials_succeeds() { /* ... */ }
}

// Add new integration tests
class UserLoginTest {
    @Test void execute_succeeds_forValidInput() { /* ... */ }
    @Test void execute_fails_whenCredentialsInvalid() { /* ... */ }
}
```

Step 2: Identify Redundancy

As you add integration tests, notice which unit tests become redundant.

Question: Is the unit test adding value beyond the integration test? - If **NO** -> Delete unit test - If **YES** (complex logic, many branches) -> Keep unit test

Step 3: Remove Redundant Unit Tests

Keep: - Complex business leaf tests - Value object tests (always) - Edge case tests not covered by integration

Delete: - Simple step tests (covered by integration) - Mock-heavy tests (testing mocking framework more than logic) - Tests that break on refactoring

Step 4: End State

```
Before Migration:
|-- 60 unit tests (component-focused)
|-- 5 integration tests
|-- Many mocks, brittle tests

After Migration:
|-- 30 integration tests (behavior-focused)
|-- 10 value object unit tests
|-- 5 complex leaf unit tests
|-- No mocks of business logic
```

Result: Fewer tests, better coverage, more confidence, less brittleness.

Test Speed Classification

Fast Tests vs Slow Tests

Test Type	Speed	What's Stubbed	When to Run
Unit tests	< 10ms each	Nothing (pure functions)	Every build
Fast integration	< 100ms each	Real business logic, stubbed adapters	Every build
Slow integration	100ms - 1s	Real adapters (DB, HTTP)	Pre-commit, CI
E2E tests	> 1s	Nothing (full stack)	CI only

Organize test suites:

```
// Fast tests - run always
@Tag("fast")
class UserLoginTest { /* stubbed adapters */ }

// Slow tests - run before commit
@Tag("slow")
class UserLoginIntegrationTest { /* real database */ }
```

Maven/Gradle configuration:

```
<!-- Run fast tests by default -->
<excludedGroups>slow</excludedGroups>
```

Testing Async Promise-Based Flows

Basic pattern - use `.await()`:

```
@Test
void async_succeeds() {
    useCase.execute(request)
           .await()  // Blocks until resolved
           .onFailure(Assertions::fail)
           .onSuccess(response -> assertEquals("expected", response.value()));
}
```

With timeout for slow operations:

```
@Test
void async_completesWithinTimeout() {
    useCase.execute(request)
           .await(TimeSpan.seconds(5))  // Fail if takes > 5s
           .onFailure(cause -> {
               if (cause instanceof CoreError.Timeout) {
                   fail("Operation timed out");
               }
               fail("Unexpected error: " + cause.message());
           })
           .onSuccess(response -> assertNotNull(response));
}
```

Testing timeout behavior:

```
@Test
void timeout_failsSlowOperation() {
    // Simulate slow operation
    var slowStep = input -> Promise.<Output>promise(promise -> {
        // Never resolves
    });

    var useCase = UseCase.create(slowStep);

    useCase.execute(request)
           .timeout(TimeSpan.millis(100))
           .await()
           .onSuccess(Assertions::fail)  // Should not succeed
           .onFailure(cause -> assertInstanceOf(CoreError.Timeout.class, cause));
}
```

Testing parallel operations (Fork-Join):

```
@Test
void forkJoin_completesAllBranches() {
    var results = Promise.all(
        Promise.success("a"),
```

```
        Promise.success("b"),
        Promise.success("c")
    ).await();

    results.onFailure(Assertions::fail)
           .onSuccess((a, b, c) -> {
               assertEquals("a", a);
               assertEquals("b", b);
               assertEquals("c", c);
           });
}
```

Key Takeaways

1. **Organize by scenario** - Nested classes, parameterized tests
2. **Value objects: 100%** - Unit tests, always
3. **Complex leaves: 100%** - Unit tests if 3+ branches
4. **Use cases: 90%+** - Integration tests with stubbed adapters
5. **Adapters: Contract tests** - Success + error modes only
6. **Migrate incrementally** - Add integration first, then remove redundant unit tests

---

Exercises

See Appendix B for exercises on: - Exercise 4.3: Test suite organization - Exercise 4.4: Migration from traditional testing

---

What's Next

Chapter 12 presents the complete RegisterUser use case - from requirements to production code, demonstrating all patterns working together.

Chapter 12: Complete Example - RegisterUser

What You'll Learn

- Complete use case from requirements to implementation
- How all patterns work together in practice
- Step-by-step evolutionary development

Prerequisites: Chapter 11: Testing in Practice

Requirements

Use case: Register a new user account.

Inputs (raw): - Email (string) - Password (string) - Referral code (optional string)

Outputs: - User ID - Confirmation token

Validation rules: - Email: not null, valid format, lowercase normalized - Password: not null, min 8 chars, at least one uppercase, one digit - Referral code: optional; if present, must be exactly 6 uppercase alphanumeric characters

Cross-field rules: - Email must not be registered yet

Steps: 1. Validate input 2. Check email uniqueness (async, database) 3. Hash password (sync, expensive computation) 4. Save the user to the database (async) 5. Generate confirmation token (async, calls external service)

Step 1: Use Case Interface

```
package com.example.app.usecase.registeruser;

import org.pragmatica.lang.*;

public interface RegisterUser {
    record Request(String email, String password, String referralCode) {}
    record Response(UserId userId, ConfirmationToken token) {}
```

```
    Promise<Response> execute(Request request);

    interface CheckEmailUniqueness {
        Promise<ValidRequest> apply(ValidRequest valid);
    }

    interface CreateValidUser {
        Promise<ValidUser> apply(ValidRequest valid);
    }

    interface SaveUser {
        Promise<User> apply(ValidUser validUser);
    }

    interface GenerateToken {
        Promise<Response> apply(User user);
    }

    static RegisterUser registerUser(CheckEmailUniqueness checkEmail,
                                     CreateValidUser createValidUser,
                                     SaveUser saveUser,
                                     GenerateToken generateToken) {
        return request -> ValidRequest.validRequest(request)
                                      .async()
                                      .flatMap(checkEmail::apply)
                                      .flatMap(createValidUser::apply)
                                      .flatMap(saveUser::apply)
                                      .flatMap(generateToken::apply);
    }
}
```

This is a **Sequencer pattern**: validate -> check uniqueness -> create user -> save -> generate token.

Step 2: Validated Request

```
record ValidRequest(Email email, Password password, Option<ReferralCode> referralCode) {

    public static Result<ValidRequest> validRequest(Request raw) {
        return Result.all(Email.email(raw.email()),
                          Password.password(raw.password()),
                          ReferralCode.referralCode(raw.referralCode()))
                     .flatMap(ValidRequest::new);
    }
```

```
}
```

This is **Fork-Join pattern**: validate three fields independently, collect results.

Step 3: Value Objects

Email:

```
package com.example.app.domain.shared;

public record Email(String value) {
    private static final Pattern EMAIL_PATTERN =
        Pattern.compile("^[a-z0-9+_.-]+@[a-z0-9.-]+$");
    private static final Fn1<Cause, String> INVALID_EMAIL =
        Causes.forOneValue("Invalid email format: %s");

    public static Result<Email> email(String raw) {
        return Verify.ensure(raw, Verify.Is::notNull)
                     .map(String::trim)
                     .map(String::toLowerCase)
                     .flatMap(Verify.ensureFn(INVALID_EMAIL,
                                              Verify.Is::matches,
                                              EMAIL_PATTERN))
                     .map(Email::new);
    }
}
```

Password:

```
public record Password(String value) {
    private static final Cause TOO_SHORT =
        Causes.cause("Password must be at least 8 characters");
    private static final Cause MISSING_UPPERCASE =
        Causes.cause("Password must contain uppercase letter");
    private static final Cause MISSING_DIGIT =
        Causes.cause("Password must contain digit");

    public static Result<Password> password(String raw) {
        return Verify.ensure(raw, Verify.Is::notNull)
                     .flatMap(Verify.ensureFn(TOO_SHORT,
                                              Verify.Is::lenBetween, 8, 128))
                     .flatMap(Password::ensureUppercase)
                     .flatMap(Password::ensureDigit)
                     .map(Password::new);
    }
```

```
    private static Result<String> ensureUppercase(String raw) {
        return contains(raw, Character::isUpperCase)
                ? Result.success(raw)
                : MISSING_UPPERCASE.result();
    }

    private static Result<String> ensureDigit(String raw) {
        return contains(raw, Character::isDigit)
                ? Result.success(raw)
                : MISSING_DIGIT.result();
    }

    private static boolean contains(CharSequence sequence, IntPredicate predicate) {
        return sequence.chars().anyMatch(predicate);
    }
}
```

ReferralCode (optional-with-validation):

```
public record ReferralCode(String value) {
    private static final String REFERRAL_PATTERN = "^[A-Z0-9]{6}$";

    public static Result<Option<ReferralCode>> referralCode(String raw) {
        return switch (raw) {
            case null, "" -> Result.success(Option.none());
            default -> Verify.ensure(raw.trim(),
                                     Verify.Is::matches,
                                     REFERRAL_PATTERN)
                             .map(ReferralCode::new)
                             .map(Option::some);
        };
    }

    public boolean isPremium() {
        return value.startsWith("VIP");
    }
}
```

Returns `Result<Option<ReferralCode>>`: validation can fail (Result), and if successful, value may be absent (Option).

Step 4: Step Implementations

CheckEmailUniqueness (adapter leaf):

```
interface CheckEmailUniqueness {
    Promise<ValidRequest> apply(ValidRequest request);

    static CheckEmailUniqueness checkEmailUniqueness(UserRepository repository) {
        return request -> repository.findByEmail(request.email())
                                    .flatMap(user -> checkPresence(user, request));
    }

    static Promise<ValidRequest> checkPresence(Option<User> user, ValidRequest request) {
        return user.isPresent()
                ? RegistrationError.General.EMAIL_ALREADY_REGISTERED.promise()
                : Promise.success(request);
    }
}
```

HashPassword (business leaf):

```
interface HashPassword {
    Result<HashedPassword> apply(Password password);

    static HashPassword hashPassword(BCryptPasswordEncoder encoder) {
        return password -> Result.lift1(RegistrationError.PasswordHashingFailed::new,
                                        encoder::encode,
                                        password.value())
                                 .map(HashedPassword::new);
    }
}
```

SaveUser (adapter leaf):

```
class JooqUserRepository implements SaveUser {
    private final DSLContext dsl;

    public Promise<User> apply(ValidUser user) {
        return Promise.lift(RepositoryError.DatabaseFailure::cause,
                            () -> saveUser(user));
    }

    private User saveUser(ValidUser user) {
        String id = dsl.insertInto(USERS)
                       .set(USERS.EMAIL, user.email().value())
                       .set(USERS.PASSWORD_HASH, user.hashed().value())
                       .set(USERS.REFERRAL_CODE,
                            user.refCode().map(ReferralCode::value).orElse(null))
                       .returningResult(USERS.ID)
                       .fetchSingle()
                       .value1();
```

```
        return new User(new UserId(id), user.email());
    }
}
```

GenerateToken (adapter leaf):

```
class TokenServiceClient implements GenerateToken {
    private final HttpClient httpClient;

    public Promise<Response> apply(User user) {
        return httpClient.post("/tokens/confirm",
                               Map.of("userId", user.id().value()))
                         .map(resp -> buildResponse(user.id(), resp))
                         .recover(this::mapTokenError);
    }

    private Response buildResponse(UserId userId, Map<String, String> resp) {
        return new Response(userId, new ConfirmationToken(resp.get("token")));
    }

    private Promise<Response> mapTokenError(Cause cause) {
        return RegistrationError.General.TOKEN_GENERATION_FAILED.promise();
    }
}
```

Step 5: Error Types

```
public sealed interface RegistrationError extends Cause {
    enum General implements RegistrationError {
        EMAIL_ALREADY_REGISTERED("Email already registered"),
        WEAK_PASSWORD_FOR_PREMIUM("Premium codes require 10+ char passwords"),
        TOKEN_GENERATION_FAILED("Token generation failed");

        private final String message;

        General(String message) {
            this.message = message;
        }

        @Override
        public String message() {
            return message;
        }
    }
```

```
    record PasswordHashingFailed(Throwable cause) implements RegistrationError {
        @Override
        public String message() {
            return "Password hashing failed: " + Causes.fromThrowable(cause);
        }
    }
}
```

Sealed interface ensures exhaustive pattern matching.

Step 6: Tests

Validation tests:

```
@Test
void validRequest_fails_forInvalidEmail() {
    var request = new Request("not-an-email", "Valid1234", null);

    ValidRequest.validRequest(request)
                .onSuccess(Assertions::fail);
}

@Test
void validRequest_succeeds_forValidInput() {
    var request = new Request("user@example.com", "Valid1234", "ABC123");

    ValidRequest.validRequest(request)
                .onFailure(Assertions::fail)
                .onSuccess(valid -> {
                    assertEquals("user@example.com", valid.email().value());
                    assertTrue(valid.referralCode().isPresent());
                });
}
```

Happy path test:

```
@Test
void execute_succeeds_forValidInput() {
    CheckEmailUniqueness checkEmail = req -> Promise.success(req);
    CreateValidUser createUser = req -> Promise.success(
        new ValidUser(req.email(), new HashedPassword("hashed"), req.referralCode()));
    SaveUser saveUser = user -> Promise.success(new User(new UserId("user-123"), user.ema
    GenerateToken generateToken = user -> Promise.success(
        new Response(user.id(), new ConfirmationToken("token-456")));

    var useCase = RegisterUser.registerUser(checkEmail, createUser, saveUser, generateTok
```

```
    var request = new Request("user@example.com", "Valid1234", null);

    useCase.execute(request)
           .await()
           .onFailure(Assertions::fail)
           .onSuccess(response -> {
               assertEquals("user-123", response.userId().value());
               assertEquals("token-456", response.token().value());
           });
}
```

Failure scenario:

```
@Test
void execute_fails_whenEmailAlreadyExists() {
    CheckEmailUniqueness checkEmail = req ->
        RegistrationError.General.EMAIL_ALREADY_REGISTERED.promise();
    // ... other stubs

    useCase.execute(request)
           .await()
           .onSuccess(Assertions::fail);
}
```

Pattern Summary

Component	Pattern	Purpose
RegisterUser.execute	Sequencer	Chain dependent steps
ValidRequest.validRequest	Fork-Join	Validate independent fields
Email.email	Leaf	Atomic validation
Password.password	Leaf	Atomic validation with Sequencer internally
CheckEmailUniqueness	Leaf + Condition	Database check with branching
SaveUser	Adapter Leaf	Database write
RegistrationError	Sealed Interface	Typed error hierarchy

Key Takeaways

1. **Use case interface** - Contains factory, step interfaces, types

2. **Sequencer for main flow** - Chain steps with flatMap
3. **Fork-Join for validation** - Accumulate errors with Result.all()
4. **Value objects as Leaves** - Parse-don't-validate
5. **Adapters implement step interfaces** - JOOQ, HTTP clients
6. **Tests use stubs** - Only adapters stubbed

Exercises

See Appendix B for exercises on: - Exercise 5.1: Extend RegisterUser with email verification - Exercise 5.3: Add premium referral validation

---

What's Next

Chapter 13 presents another complete example - PlaceOrder - demonstrating more complex patterns including Fork-Join for parallel data fetching.

Chapter 13: Complete Example - PlaceOrder

This chapter walks through building a complete e-commerce order placement use case from requirements to production-ready code.

Requirements

Business Goal: Allow customers to place orders for items in their cart.

Flow: 1. Validate the order request 2. Check inventory for all items (parallel) 3. Reserve inventory 4. Process payment 5. Create order record 6. Send confirmation notification

Business Rules: - All items must be in stock - Payment must succeed before inventory is committed - If payment fails, release reserved inventory - Order confirmation is best-effort (don't fail order if notification fails)

Error Scenarios: - Invalid request (missing fields, invalid quantities) - Insufficient inventory for one or more items - Payment declined - Payment timeout - Database failure

---

Domain Analysis

Value Objects

Type	Validation	Notes
OrderId	UUID format	Generated, not user input
CustomerId	Non-empty, UUID format	From session/auth
ProductId	Non-empty, UUID format	
Quantity	Positive integer, max 100	Per-item limit
Money	Non-negative, max 2 decimal places	
Address	Street, city, postal code, country	Composite

Patterns Identified

Step	Pattern	Rationale
Validate request	Fork-Join	Multiple independent field validations
Check inventory	Fork-Join	Check all items in parallel
Reserve + Pay + Create	Sequencer	Strict ordering required
Send notification	Leaf with Aspect	Best-effort, don't fail order

---

Package Structure

```
com.example.shop.usecase.placeorder/
  PlaceOrder.java           # Use case interface + factory
  OrderError.java           # Sealed error interface
  ValidOrderRequest.java    # Validated request record
  OrderLine.java            # Validated line item
  ReservedInventory.java    # Intermediate state

com.example.shop.domain.shared/
  CustomerId.java
  ProductId.java
  Quantity.java
  Money.java
  Address.java
  OrderId.java
```

Step 1: Value Objects

ProductId

```java
package com.example.shop.domain.shared;

import org.pragmatica.lang.Result;
import org.pragmatica.lang.Cause;
import org.pragmatica.lang.utils.Causes;
import org.pragmatica.lang.parse.Network;

public record ProductId(String value) {
    private static final Cause INVALID_PRODUCT_ID = Causes.cause("Invalid product ID form
```

```
    public static Result<ProductId> productId(String raw) {
        return Network.parseUUID(raw)
            .mapError(_ -> INVALID_PRODUCT_ID)
            .map(uuid -> new ProductId(uuid.toString()));
    }
}
```

Quantity

```
package com.example.shop.domain.shared;

import org.pragmatica.lang.Result;
import org.pragmatica.lang.Cause;
import org.pragmatica.lang.utils.Causes;
import org.pragmatica.lang.utils.Verify;

public record Quantity(int value) {
    private static final Cause NOT_POSITIVE = Causes.cause("Quantity must be positive");
    private static final Cause EXCEEDS_LIMIT = Causes.cause("Quantity cannot exceed 100")

    public static Result<Quantity> quantity(int raw) {
        return Verify.ensure(raw, Verify.Is::positive)
            .mapError(_ -> NOT_POSITIVE)
            .flatMap(Verify.ensureFn(EXCEEDS_LIMIT, Verify.Is::lessThanOrEqualTo, 100))
            .map(Quantity::new);
    }

    public boolean isGreaterThan(Quantity other) {
        return this.value > other.value;
    }
}
```

Money

```
package com.example.shop.domain.shared;

import org.pragmatica.lang.Result;
import org.pragmatica.lang.Cause;
import org.pragmatica.lang.utils.Causes;

import java.math.BigDecimal;
import java.math.RoundingMode;

public record Money(BigDecimal value) {
    private static final Cause NEGATIVE = Causes.cause("Money cannot be negative");
```

```
    private static final Cause TOO_MANY_DECIMALS = Causes.cause("Money allows max 2 decim

    public static Result<Money> money(BigDecimal raw) {
        if (raw.compareTo(BigDecimal.ZERO) < 0) {
            return NEGATIVE.result();
        }
        if (raw.scale() > 2) {
            return TOO_MANY_DECIMALS.result();
        }
        return Result.success(new Money(raw.setScale(2, RoundingMode.HALF_UP)));
    }

    public static Result<Money> money(String raw) {
        return Result.lift(() -> new BigDecimal(raw))
            .mapError(_ -> Causes.cause("Invalid money format"))
            .flatMap(Money::money);
    }

    public Money add(Money other) {
        return new Money(this.value.add(other.value));
    }

    public Money multiply(int quantity) {
        return new Money(this.value.multiply(BigDecimal.valueOf(quantity)));
    }

    public static final Money ZERO = new Money(BigDecimal.ZERO.setScale(2, RoundingMode.H
}
```

Address

```
package com.example.shop.domain.shared;

import org.pragmatica.lang.Result;
import org.pragmatica.lang.Cause;
import org.pragmatica.lang.utils.Causes;
import org.pragmatica.lang.utils.Verify;

public record Address(
    String street,
    String city,
    String postalCode,
    String country
) {
    private static final Cause STREET_REQUIRED = Causes.cause("Street is required");
    private static final Cause CITY_REQUIRED = Causes.cause("City is required");
```

```
    private static final Cause POSTAL_CODE_REQUIRED = Causes.cause("Postal code is requi
    private static final Cause COUNTRY_REQUIRED = Causes.cause("Country is required");

    public static Result<Address> address(String street, String city, String postalCode,
        return Result.all(
            validateField(street, STREET_REQUIRED),
            validateField(city, CITY_REQUIRED),
            validateField(postalCode, POSTAL_CODE_REQUIRED),
            validateField(country, COUNTRY_REQUIRED)
        ).map(Address::new);
    }

    private static Result<String> validateField(String value, Cause error) {
        return Verify.ensure(value, Verify.Is::notBlank)
            .mapError(_ -> error)
            .map(String::trim);
    }
}
```

Step 2: Error Types

```
package com.example.shop.usecase.placeorder;

import org.pragmatica.lang.Cause;
import org.pragmatica.lang.utils.Causes;
import com.example.shop.domain.shared.ProductId;
import com.example.shop.domain.shared.Quantity;

import java.util.List;

public sealed interface OrderError extends Cause {

    // Fixed-message errors grouped in enum
    enum General implements OrderError {
        EMPTY_ORDER("Order must contain at least one item"),
        PAYMENT_DECLINED("Payment was declined"),
        PAYMENT_TIMEOUT("Payment service timed out"),
        DATABASE_ERROR("Failed to save order");

        private final String message;

        General(String message) {
            this.message = message;
        }
```

```
        @Override
        public String message() {
            return message;
        }
    }

    // Errors with data
    record InsufficientInventory(List<ProductId> products) implements OrderError {
        @Override
        public String message() {
            return "Insufficient inventory for products: " +
                products.stream().map(ProductId::value).toList();
        }
    }

    record InvalidQuantity(ProductId productId, Quantity requested, Quantity available)
        implements OrderError {
        @Override
        public String message() {
            return String.format("Product %s: requested %d, available %d",
                productId.value(), requested.value(), available.value());
        }
    }

    record PaymentFailed(Throwable cause) implements OrderError {
        @Override
        public String message() {
            return "Payment processing failed: " + Causes.fromThrowable(cause);
        }
    }
}
```

Step 3: Validated Request

Raw Request (from API)

```
package com.example.shop.usecase.placeorder;

import java.util.List;

public record OrderRequest(
    String customerId,
    List<LineItemRequest> items,
```

```
    AddressRequest shippingAddress
) {
    public record LineItemRequest(String productId, int quantity) {}
    public record AddressRequest(String street, String city, String postalCode, String co
}
```

Validated Order Line

```
package com.example.shop.usecase.placeorder;

import org.pragmatica.lang.Result;
import com.example.shop.domain.shared.ProductId;
import com.example.shop.domain.shared.Quantity;

public record OrderLine(ProductId productId, Quantity quantity) {

    public static Result<OrderLine> orderLine(OrderRequest.LineItemRequest raw) {
        return Result.all(
            ProductId.productId(raw.productId()),
            Quantity.quantity(raw.quantity())
        ).map(OrderLine::new);
    }
}
```

Validated Request

```
package com.example.shop.usecase.placeorder;

import org.pragmatica.lang.Result;
import org.pragmatica.lang.Cause;
import org.pragmatica.lang.utils.Causes;
import com.example.shop.domain.shared.CustomerId;
import com.example.shop.domain.shared.Address;

import java.util.List;

public record ValidOrderRequest(
    CustomerId customerId,
    List<OrderLine> items,
    Address shippingAddress
) {
    private static final Cause EMPTY_ORDER = OrderError.General.EMPTY_ORDER;

    public static Result<ValidOrderRequest> validOrderRequest(OrderRequest raw) {
        return Result.all(
```

```
            CustomerId.customerId(raw.customerId()),
            validateItems(raw.items()),
            Address.address(
                raw.shippingAddress().street(),
                raw.shippingAddress().city(),
                raw.shippingAddress().postalCode(),
                raw.shippingAddress().country()
            )
        ).map(ValidOrderRequest::new);
    }

    private static Result<List<OrderLine>> validateItems(List<OrderRequest.LineItemReques
        if (items == null || items.isEmpty()) {
            return EMPTY_ORDER.result();
        }

        return Result.allOf(
            items.stream()
                .map(OrderLine::orderLine)
                .toList()
        );
    }
}
```

Step 4: Step Interfaces

```
package com.example.shop.usecase.placeorder;

import org.pragmatica.lang.Promise;
import org.pragmatica.lang.Result;
import com.example.shop.domain.shared.OrderId;
import com.example.shop.domain.shared.Money;

public interface PlaceOrder {

    record Response(OrderId orderId, Money total) {}

    Promise<Response> execute(OrderRequest request);

    // Step 1: Check inventory for all items (Fork-Join)
    interface CheckInventory {
        Promise<ValidOrderRequest> apply(ValidOrderRequest request);
    }
```

```
    // Step 2: Reserve inventory
    interface ReserveInventory {
        Promise<ReservedInventory> apply(ValidOrderRequest request);
    }

    // Step 3: Process payment
    interface ProcessPayment {
        Promise<PaymentConfirmation> apply(ReservedInventory reservation, Money total);
    }

    // Step 4: Create order record
    interface CreateOrder {
        Promise<OrderId> apply(ReservedInventory reservation, PaymentConfirmation payment
    }

    // Step 5: Send confirmation (best-effort)
    interface SendConfirmation {
        Promise<Void> apply(OrderId orderId, ValidOrderRequest request);
    }

    // Step 6: Release inventory (compensation)
    interface ReleaseInventory {
        Promise<Void> apply(ReservedInventory reservation);
    }

    // Factory
    static PlaceOrder placeOrder(
        CheckInventory checkInventory,
        ReserveInventory reserveInventory,
        ProcessPayment processPayment,
        CreateOrder createOrder,
        SendConfirmation sendConfirmation,
        ReleaseInventory releaseInventory,
        PriceCalculator priceCalculator
    ) {
        return request -> ValidOrderRequest.validOrderRequest(request)
            .async()
            .flatMap(checkInventory::apply)
            .flatMap(valid -> executeWithCompensation(
                valid,
                reserveInventory,
                processPayment,
                createOrder,
                releaseInventory,
                priceCalculator
            ))
```

```
            .onSuccess(response -> sendConfirmation.apply(response.orderId(), null)
                .onFailure(e -> { /* log but don't fail */ }));
    }

    private static Promise<Response> executeWithCompensation(
        ValidOrderRequest request,
        ReserveInventory reserveInventory,
        ProcessPayment processPayment,
        CreateOrder createOrder,
        ReleaseInventory releaseInventory,
        PriceCalculator priceCalculator
    ) {
        var total = priceCalculator.calculate(request);

        return reserveInventory.apply(request)
            .flatMap(reservation -> processPayment.apply(reservation, total)
                .flatMap(payment -> createOrder.apply(reservation, payment))
                .map(orderId -> new Response(orderId, total))
                .recover(cause -> compensateAndFail(reservation, releaseInventory, cause)
    }

    private static Promise<Response> compensateAndFail(
        ReservedInventory reservation,
        ReleaseInventory releaseInventory,
        Cause cause
    ) {
        return releaseInventory.apply(reservation)
            .flatMap(_ -> cause.promise());
    }
}
```

Step 5: Supporting Types

ReservedInventory

```
package com.example.shop.usecase.placeorder;

import java.time.Instant;
import java.util.List;

public record ReservedInventory(
    String reservationId,
    List<ReservedItem> items,
    Instant expiresAt
```

```
) {
    public record ReservedItem(String productId, int quantity) {}
}
```

PaymentConfirmation

```
package com.example.shop.usecase.placeorder;

import java.time.Instant;

public record PaymentConfirmation(
    String transactionId,
    Instant processedAt
) {}
```

PriceCalculator

```
package com.example.shop.usecase.placeorder;

import com.example.shop.domain.shared.Money;

public interface PriceCalculator {
    Money calculate(ValidOrderRequest request);
}
```

Step 6: Step Implementations

CheckInventory (Fork-Join Pattern)

```
package com.example.shop.adapter.inventory;

import org.pragmatica.lang.Promise;
import com.example.shop.usecase.placeorder.PlaceOrder.CheckInventory;
import com.example.shop.usecase.placeorder.ValidOrderRequest;
import com.example.shop.usecase.placeorder.OrderLine;
import com.example.shop.usecase.placeorder.OrderError;
import com.example.shop.domain.shared.ProductId;

import java.util.List;

public class InventoryChecker implements CheckInventory {
    private final InventoryClient inventoryClient;
```

```
    public InventoryChecker(InventoryClient inventoryClient) {
        this.inventoryClient = inventoryClient;
    }

    @Override
    public Promise<ValidOrderRequest> apply(ValidOrderRequest request) {
        // Check all items in parallel (Fork-Join)
        var checks = request.items().stream()
            .map(this::checkItem)
            .toList();

        return Promise.allOf(checks)
            .flatMap(results -> aggregateResults(results, request));
    }

    private Promise<OrderLine> checkItem(OrderLine line) {
        return inventoryClient.getAvailable(line.productId())
            .flatMap(available -> available.value() >= line.quantity().value()
                ? Promise.success(line)
                : OrderError.InvalidQuantity(
                    line.productId(),
                    line.quantity(),
                    available
                  ).promise());
    }

    private Promise<ValidOrderRequest> aggregateResults(
        List<Result<OrderLine>> results,
        ValidOrderRequest request
    ) {
        var failures = results.stream()
            .filter(Result::isFailure)
            .map(r -> r.cause())
            .toList();

        if (failures.isEmpty()) {
            return Promise.success(request);
        }

        var failedProducts = failures.stream()
            .filter(c -> c instanceof OrderError.InvalidQuantity)
            .map(c -> ((OrderError.InvalidQuantity) c).productId())
            .toList();

        return new OrderError.InsufficientInventory(failedProducts).promise();
    }
```

```
}
```

ProcessPayment (Leaf with Error Mapping)

```
package com.example.shop.adapter.payment;

import org.pragmatica.lang.Promise;
import com.example.shop.usecase.placeorder.PlaceOrder.ProcessPayment;
import com.example.shop.usecase.placeorder.ReservedInventory;
import com.example.shop.usecase.placeorder.PaymentConfirmation;
import com.example.shop.usecase.placeorder.OrderError;
import com.example.shop.domain.shared.Money;

import java.time.Instant;
import java.util.concurrent.TimeoutException;

public class PaymentProcessor implements ProcessPayment {
    private final PaymentGateway gateway;

    public PaymentProcessor(PaymentGateway gateway) {
        this.gateway = gateway;
    }

    @Override
    public Promise<PaymentConfirmation> apply(ReservedInventory reservation, Money total)
        return Promise.lift(
            OrderError.PaymentFailed::new,
            () -> gateway.charge(total.value())
        ).map(txId -> new PaymentConfirmation(txId, Instant.now()))
         .recover(this::mapPaymentError);
    }

    private Promise<PaymentConfirmation> mapPaymentError(Cause cause) {
        return switch (cause) {
            case OrderError.PaymentFailed pf -> {
                if (pf.cause() instanceof TimeoutException) {
                    yield OrderError.General.PAYMENT_TIMEOUT.promise();
                }
                if (isDeclined(pf.cause())) {
                    yield OrderError.General.PAYMENT_DECLINED.promise();
                }
                yield cause.promise();
            }
            default -> cause.promise();
        };
    }
```

```
    private boolean isDeclined(Throwable t) {
        return t.getMessage() != null && t.getMessage().contains("declined");
    }
}
```

CreateOrder (Leaf - Database)

```
package com.example.shop.adapter.persistence;

import org.pragmatica.lang.Promise;
import org.jooq.DSLContext;
import com.example.shop.usecase.placeorder.PlaceOrder.CreateOrder;
import com.example.shop.usecase.placeorder.ReservedInventory;
import com.example.shop.usecase.placeorder.PaymentConfirmation;
import com.example.shop.usecase.placeorder.OrderError;
import com.example.shop.domain.shared.OrderId;

import java.util.UUID;

import static com.example.shop.adapter.persistence.Tables.*;

public class JooqOrderRepository implements CreateOrder {
    private final DSLContext dsl;

    public JooqOrderRepository(DSLContext dsl) {
        this.dsl = dsl;
    }

    @Override
    public Promise<OrderId> apply(ReservedInventory reservation, PaymentConfirmation paym
        return Promise.lift(
            _ -> OrderError.General.DATABASE_ERROR,
            () -> {
                var orderId = UUID.randomUUID().toString();

                dsl.insertInto(ORDERS)
                    .set(ORDERS.ID, orderId)
                    .set(ORDERS.RESERVATION_ID, reservation.reservationId())
                    .set(ORDERS.PAYMENT_TX_ID, payment.transactionId())
                    .set(ORDERS.STATUS, "CONFIRMED")
                    .execute();

                for (var item : reservation.items()) {
                    dsl.insertInto(ORDER_ITEMS)
                        .set(ORDER_ITEMS.ORDER_ID, orderId)
```

```
                        .set(ORDER_ITEMS.PRODUCT_ID, item.productId())
                        .set(ORDER_ITEMS.QUANTITY, item.quantity())
                        .execute();
                }

                return new OrderId(orderId);
            }
        );
    }
}
```

Step 7: Testing

Value Object Tests

```
class QuantityTest {

    @Nested
    class ValidationSucceeds {
        @Test
        void quantity_succeeds_forPositiveValue() {
            Quantity.quantity(5)
                .onFailure(Assertions::fail)
                .onSuccess(q -> assertEquals(5, q.value()));
        }

        @Test
        void quantity_succeeds_forMaxValue() {
            Quantity.quantity(100)
                .onFailure(Assertions::fail)
                .onSuccess(q -> assertEquals(100, q.value()));
        }
    }

    @Nested
    class ValidationFails {
        @Test
        void quantity_fails_forZero() {
            Quantity.quantity(0).onSuccess(Assertions::fail);
        }

        @Test
        void quantity_fails_forNegative() {
            Quantity.quantity(-1).onSuccess(Assertions::fail);
```

```
        }

        @Test
        void quantity_fails_forExceedingLimit() {
            Quantity.quantity(101).onSuccess(Assertions::fail);
        }
    }
}
```

Use Case Integration Test

```
class PlaceOrderTest {
    private PlaceOrder useCase;

    @BeforeEach
    void setup() {
        // All stubs - happy path
        PlaceOrder.CheckInventory checkInventory = req -> Promise.success(req);
        PlaceOrder.ReserveInventory reserveInventory = req -> Promise.success(
            new ReservedInventory("res-123", List.of(), Instant.now().plusMinutes(15))
        );
        PlaceOrder.ProcessPayment processPayment = (res, total) -> Promise.success(
            new PaymentConfirmation("tx-456", Instant.now())
        );
        PlaceOrder.CreateOrder createOrder = (res, pay) -> Promise.success(
            new OrderId("order-789")
        );
        PlaceOrder.SendConfirmation sendConfirmation = (id, req) -> Promise.success(null)
        PlaceOrder.ReleaseInventory releaseInventory = res -> Promise.success(null);
        PriceCalculator priceCalculator = req -> new Money(BigDecimal.valueOf(99.99));

        useCase = PlaceOrder.placeOrder(
            checkInventory, reserveInventory, processPayment,
            createOrder, sendConfirmation, releaseInventory, priceCalculator
        );
    }

    @Nested
    class HappyPath {
        @Test
        void execute_succeeds_forValidOrder() {
            var request = new OrderRequest(
                "550e8400-e29b-41d4-a716-446655440000",
                List.of(new OrderRequest.LineItemRequest(
                    "660e8400-e29b-41d4-a716-446655440001", 2
                )),
```

```
                new OrderRequest.AddressRequest("123 Main St", "City", "12345", "US")
            );

            useCase.execute(request)
                .await()
                .onFailure(Assertions::fail)
                .onSuccess(response -> {
                    assertEquals("order-789", response.orderId().value());
                });
        }
    }

    @Nested
    class ValidationFailures {
        @Test
        void execute_fails_forEmptyOrder() {
            var request = new OrderRequest(
                "550e8400-e29b-41d4-a716-446655440000",
                List.of(),
                new OrderRequest.AddressRequest("123 Main St", "City", "12345", "US")
            );

            useCase.execute(request)
                .await()
                .onSuccess(Assertions::fail);
        }
    }

    @Nested
    class StepFailures {
        @Test
        void execute_fails_whenInventoryInsufficient() {
            PlaceOrder.CheckInventory failingCheck = req ->
                new OrderError.InsufficientInventory(List.of()).promise();

            var failingUseCase = PlaceOrder.placeOrder(
                failingCheck, null, null, null, null, null, null
            );

            failingUseCase.execute(validRequest())
                .await()
                .onSuccess(Assertions::fail);
        }

        @Test
        void execute_releasesInventory_whenPaymentFails() {
```

```
            var released = new AtomicBoolean(false);

            PlaceOrder.ReserveInventory reserveInventory = req -> Promise.success(
                new ReservedInventory("res-123", List.of(), Instant.now().plusMinutes(15)
            );
            PlaceOrder.ProcessPayment failingPayment = (res, total) ->
                OrderError.General.PAYMENT_DECLINED.promise();
            PlaceOrder.ReleaseInventory releaseInventory = res -> {
                released.set(true);
                return Promise.success(null);
            };

            var failingUseCase = PlaceOrder.placeOrder(
                req -> Promise.success(req),
                reserveInventory,
                failingPayment,
                null,
                null,
                releaseInventory,
                req -> Money.ZERO
            );

            failingUseCase.execute(validRequest())
                .await()
                .onSuccess(Assertions::fail);

            assertTrue(released.get(), "Inventory should be released on payment failure")
        }
    }

    private OrderRequest validRequest() {
        return new OrderRequest(
            "550e8400-e29b-41d4-a716-446655440000",
            List.of(new OrderRequest.LineItemRequest(
                "660e8400-e29b-41d4-a716-446655440001", 2
            )),
            new OrderRequest.AddressRequest("123 Main St", "City", "12345", "US")
        );
    }
}
```

Key Patterns Demonstrated

Pattern	Where Used	Purpose
Fork-Join	ValidOrderRequest, CheckInventory	Parallel field validation, parallel inventory checks
Sequencer	Main flow	Reserve -> Pay -> Create
Leaf	ProcessPayment, CreateOrder	Single I/O operations
Condition	Error mapping in ProcessPayment	Route to specific error types
Aspects	SendConfirmation	Best-effort with failure tolerance

Compensation Pattern

This example demonstrates compensation (rollback) when payment fails after inventory is reserved:

```
reserve() -> pay() -> create()
              |
              v (failure)
         release() -> propagate error
```

The `recover()` method enables clean compensation logic without try-catch.

Exercises

1. **Add discount support:** Modify `PriceCalculator` to accept a discount code and validate it as a value object.
2. **Add idempotency:** Ensure the same order request doesn't create duplicate orders. What value object would you create?
3. **Add retry aspect:** Wrap `ProcessPayment` with a retry aspect that retries on timeout but not on declined.
4. **Test the compensation:** Write a test verifying that inventory is released when `CreateOrder` fails (not just `ProcessPayment`).

Summary

PlaceOrder demonstrates:

- **Fork-Join for validation** - All fields validated in parallel, errors accumulated
- **Fork-Join for I/O** - Inventory checked for all items in parallel

- **Sequencer with compensation** - Reserve -> Pay -> Create with rollback on failure
- **Error type hierarchy** - Grouped enum for fixed messages, records for data-carrying errors
- **Best-effort operations** - Notification doesn't fail the order
- **Adapter isolation** - All external calls wrapped in Promise.lift

The pattern composition remains simple despite the complex business requirements. Each step is independently testable, and the compensation logic is explicit rather than hidden in catch blocks.

Chapter 14a: Focused Example - PublishArticle

This focused example demonstrates the **Condition pattern** for routing logic in a content management domain.

Requirements

Business Goal: Allow authors to submit articles for publication through an approval workflow.

Flow: 1. Validate article submission 2. Route based on author tier: - **Premium authors:** Auto-approve, publish immediately - **Standard authors:** Queue for editorial review - **New authors:** Queue for senior review + plagiarism check

Business Rules: - Premium authors bypass review (auto-publish) - Standard authors need one editorial approval - New authors need senior approval AND plagiarism check passes - All articles go through content validation (length, format)

Domain Model

```
public enum AuthorTier { PREMIUM, STANDARD, NEW }

public record ArticleId(String value) { /* factory omitted for brevity */ }

public record AuthorId(String value) { /* factory omitted for brevity */ }

public record Article(
    ArticleId id,
    AuthorId authorId,
    String title,
    String content,
    AuthorTier authorTier
) {}

public enum PublishStatus {
```

```
    PUBLISHED,
    PENDING_EDITORIAL_REVIEW,
    PENDING_SENIOR_REVIEW
}

public record PublishResult(ArticleId articleId, PublishStatus status) {}
```

Error Types

```
public sealed interface PublishError extends Cause {

    enum General implements PublishError {
        TITLE_TOO_SHORT("Title must be at least 10 characters"),
        CONTENT_TOO_SHORT("Content must be at least 500 characters"),
        PLAGIARISM_DETECTED("Content failed plagiarism check"),
        SAVE_FAILED("Failed to save article");

        private final String message;

        General(String message) {
            this.message = message;
        }

        @Override
        public String message() {
            return message;
        }
    }
}
```

Validated Request

```
public record ValidArticle(
    AuthorId authorId,
    String title,
    String content,
    AuthorTier authorTier
) {
    private static final int MIN_TITLE_LENGTH = 10;
    private static final int MIN_CONTENT_LENGTH = 500;
```

```
    public static Result<ValidArticle> validArticle(
        String authorId,
        String title,
        String content,
        AuthorTier tier
    ) {
        return Result.all(
            AuthorId.authorId(authorId),
            validateTitle(title),
            validateContent(content),
            Result.success(tier)
        ).map(ValidArticle::new);
    }

    private static Result<String> validateTitle(String title) {
        return Verify.ensure(title, Verify.Is::notBlank)
            .flatMap(t -> t.trim().length() >= MIN_TITLE_LENGTH
                ? Result.success(t.trim())
                : PublishError.General.TITLE_TOO_SHORT.result());
    }

    private static Result<String> validateContent(String content) {
        return Verify.ensure(content, Verify.Is::notBlank)
            .flatMap(c -> c.trim().length() >= MIN_CONTENT_LENGTH
                ? Result.success(c.trim())
                : PublishError.General.CONTENT_TOO_SHORT.result());
    }
}
```

Use Case with Condition Pattern

```
public interface PublishArticle {

    Promise<PublishResult> execute(ArticleRequest request);

    // Step interfaces
    interface SaveArticle {
        Promise<ArticleId> apply(ValidArticle article);
    }

    interface CheckPlagiarism {
        Promise<ValidArticle> apply(ValidArticle article);
    }
```

```
    interface QueueForReview {
        Promise<PublishResult> apply(ArticleId id, PublishStatus status);
    }

    // Factory demonstrating Condition pattern
    static PublishArticle publishArticle(
        SaveArticle saveArticle,
        CheckPlagiarism checkPlagiarism,
        QueueForReview queueForReview
    ) {
        return request -> ValidArticle.validArticle(
                request.authorId(),
                request.title(),
                request.content(),
                request.authorTier()
            )
            .async()
            .flatMap(article -> routeByAuthorTier(
                article, saveArticle, checkPlagiarism, queueForReview
            ));
    }

    // CONDITION PATTERN: Routing only, no transformation
    private static Promise<PublishResult> routeByAuthorTier(
        ValidArticle article,
        SaveArticle saveArticle,
        CheckPlagiarism checkPlagiarism,
        QueueForReview queueForReview
    ) {
        return switch (article.authorTier()) {
            case PREMIUM -> publishImmediately(article, saveArticle);
            case STANDARD -> queueForEditorialReview(article, saveArticle, queueForReview
            case NEW -> queueWithPlagiarismCheck(article, saveArticle, checkPlagiarism, q
        };
    }

    // Route 1: Premium - auto-publish
    private static Promise<PublishResult> publishImmediately(
        ValidArticle article,
        SaveArticle saveArticle
    ) {
        return saveArticle.apply(article)
            .map(id -> new PublishResult(id, PublishStatus.PUBLISHED));
    }

    // Route 2: Standard - editorial review
```

```
    private static Promise<PublishResult> queueForEditorialReview(
        ValidArticle article,
        SaveArticle saveArticle,
        QueueForReview queueForReview
    ) {
        return saveArticle.apply(article)
            .flatMap(id -> queueForReview.apply(id, PublishStatus.PENDING_EDITORIAL_REVIE
    }

    // Route 3: New - plagiarism check + senior review
    private static Promise<PublishResult> queueWithPlagiarismCheck(
        ValidArticle article,
        SaveArticle saveArticle,
        CheckPlagiarism checkPlagiarism,
        QueueForReview queueForReview
    ) {
        return checkPlagiarism.apply(article)
            .flatMap(saveArticle::apply)
            .flatMap(id -> queueForReview.apply(id, PublishStatus.PENDING_SENIOR_REVIEW))
    }
}
```

Key Points: Condition Pattern

What Condition Does

The Condition pattern performs **routing only** - it selects which execution path to follow based on input data. The switch expression returns a `Promise<PublishResult>` from each branch.

```
return switch (article.authorTier()) {
    case PREMIUM -> publishImmediately(...);      // Route to path A
    case STANDARD -> queueForEditorialReview(...); // Route to path B
    case NEW -> queueWithPlagiarismCheck(...);    // Route to path C
};
```

What Condition Does NOT Do

Condition does not transform data inline. Each route delegates to a separate method:

```
// WRONG: Transformation in condition
return switch (article.authorTier()) {
    case PREMIUM -> saveArticle.apply(article)
        .map(id -> new PublishResult(id, PublishStatus.PUBLISHED));  // Too much here
    ...
```

```
};

// CORRECT: Route to method that handles the path
return switch (article.authorTier()) {
    case PREMIUM -> publishImmediately(article, saveArticle);  // Delegation
    ...
};
```

Benefits

1. **Clear routing logic** - One place shows all possible paths
2. **Testable branches** - Each route method can be tested independently
3. **Extensible** - Adding new author tiers requires adding one case
4. **Type-safe** - Compiler warns if switch is non-exhaustive

---

Testing

```
class PublishArticleTest {

    @Nested
    class Routing {
        @Test
        void execute_publishesImmediately_forPremiumAuthor() {
            var saved = new AtomicReference<ValidArticle>();

            PublishArticle.SaveArticle saveArticle = article -> {
                saved.set(article);
                return Promise.success(new ArticleId("art-123"));
            };

            var useCase = PublishArticle.publishArticle(
                saveArticle,
                article -> Promise.success(article),
                (id, status) -> Promise.success(new PublishResult(id, status))
            );

            useCase.execute(requestWithTier(AuthorTier.PREMIUM))
                .await()
                .onFailure(Assertions::fail)
                .onSuccess(result -> {
                    assertEquals(PublishStatus.PUBLISHED, result.status());
                    assertNotNull(saved.get());
                });
        }
```

```
    @Test
    void execute_queuesForEditorialReview_forStandardAuthor() {
        var useCase = PublishArticle.publishArticle(
            article -> Promise.success(new ArticleId("art-123")),
            article -> Promise.success(article),
            (id, status) -> Promise.success(new PublishResult(id, status))
        );

        useCase.execute(requestWithTier(AuthorTier.STANDARD))
            .await()
            .onFailure(Assertions::fail)
            .onSuccess(result ->
                assertEquals(PublishStatus.PENDING_EDITORIAL_REVIEW, result.status())
            );
    }

    @Test
    void execute_checksPlagiarism_forNewAuthor() {
        var plagiarismChecked = new AtomicBoolean(false);

        PublishArticle.CheckPlagiarism checkPlagiarism = article -> {
            plagiarismChecked.set(true);
            return Promise.success(article);
        };

        var useCase = PublishArticle.publishArticle(
            article -> Promise.success(new ArticleId("art-123")),
            checkPlagiarism,
            (id, status) -> Promise.success(new PublishResult(id, status))
        );

        useCase.execute(requestWithTier(AuthorTier.NEW))
            .await()
            .onFailure(Assertions::fail)
            .onSuccess(result -> {
                assertTrue(plagiarismChecked.get());
                assertEquals(PublishStatus.PENDING_SENIOR_REVIEW, result.status());
            });
    }

    @Test
    void execute_fails_whenPlagiarismDetected() {
        PublishArticle.CheckPlagiarism failingCheck = article ->
            PublishError.General.PLAGIARISM_DETECTED.promise();

        var useCase = PublishArticle.publishArticle(
```

```
                article -> Promise.success(new ArticleId("art-123")),
                failingCheck,
                (id, status) -> Promise.success(new PublishResult(id, status))
            );

            useCase.execute(requestWithTier(AuthorTier.NEW))
                .await()
                .onSuccess(Assertions::fail);
        }
    }

    private ArticleRequest requestWithTier(AuthorTier tier) {
        return new ArticleRequest(
            "author-123",
            "This is a valid title for testing",
            "x".repeat(600),  // Valid content length
            tier
        );
    }
}
```

Exercises

1. **Add tier upgrade:** If a NEW author has 10+ published articles, route them as STANDARD. Where does this logic belong?
2. **Add scheduled publishing:** PREMIUM authors can specify a publish date. Modify the Condition to route scheduled articles differently.
3. **Extract to filter:** Rewrite the plagiarism check using `.filter()` instead of a separate step interface.

Summary

PublishArticle demonstrates:

- **Condition pattern for routing** - Switch expression selects execution path
- **No transformation in conditions** - Each branch delegates to a method
- **Pattern matching with enums** - Type-safe, exhaustive routing
- **Testable branches** - Each route tested independently
- **Composable with other patterns** - Routes contain Sequencer chains

Chapter 14b: Focused Example - TransferFunds

This focused example demonstrates **Aspects** for cross-cutting concerns in a financial domain with audit requirements.

Requirements

Business Goal: Transfer funds between accounts with full audit trail.

Flow: 1. Validate transfer request 2. Verify source account has sufficient balance 3. Execute the transfer (debit source, credit destination) 4. Record audit log

Regulatory Requirements: - Every operation must be logged with timestamp, actor, and outcome - Failed operations must also be logged (not just successes) - Audit logging must not fail the transfer (best-effort) - All transfers above threshold require additional authorization

Technical Requirements: - Idempotency - same request ID must not execute twice - Timeout - external calls must timeout after 5 seconds - Retry - transient failures should retry up to 3 times

---

Domain Model

```
public record AccountId(String value) { /* factory omitted */ }

public record TransferId(String value) { /* factory omitted */ }

public record Money(BigDecimal value) {
    public boolean isGreaterThan(Money other) {
        return this.value.compareTo(other.value) > 0;
    }

    public Money subtract(Money other) {
        return new Money(this.value.subtract(other.value));
    }
```

```
    public Money add(Money other) {
        return new Money(this.value.add(other.value));
    }
}

public record TransferRequest(
    String requestId,
    String sourceAccountId,
    String destinationAccountId,
    String amount,
    String initiatedBy
) {}

public record TransferResult(
    TransferId transferId,
    Money amount,
    Instant completedAt
) {}
```

Error Types

```
public sealed interface TransferError extends Cause {

    enum General implements TransferError {
        INSUFFICIENT_FUNDS("Insufficient funds in source account"),
        SAME_ACCOUNT("Cannot transfer to same account"),
        DUPLICATE_REQUEST("Transfer request already processed"),
        ACCOUNT_NOT_FOUND("Account not found"),
        TRANSFER_FAILED("Transfer execution failed");

        private final String message;

        General(String message) {
            this.message = message;
        }

        @Override
        public String message() {
            return message;
        }
    }

    record AuthorizationRequired(Money amount, Money threshold) implements TransferError
        @Override
```

```
        public String message() {
            return String.format("Transfer of %s requires authorization (threshold: %s)",
                amount.value(), threshold.value());
        }
    }
}
```

Validated Request

```
public record ValidTransfer(
    String requestId,
    AccountId sourceAccount,
    AccountId destinationAccount,
    Money amount,
    String initiatedBy
) {
    public static Result<ValidTransfer> validTransfer(TransferRequest raw) {
        return Result.all(
            validateRequestId(raw.requestId()),
            AccountId.accountId(raw.sourceAccountId()),
            AccountId.accountId(raw.destinationAccountId()),
            Money.money(raw.amount()),
            validateInitiator(raw.initiatedBy())
        ).flatMap(ValidTransfer::validateNotSameAccount);
    }

    private static Result<ValidTransfer> validateNotSameAccount(
        String requestId,
        AccountId source,
        AccountId destination,
        Money amount,
        String initiatedBy
    ) {
        if (source.value().equals(destination.value())) {
            return TransferError.General.SAME_ACCOUNT.result();
        }
        return Result.success(new ValidTransfer(requestId, source, destination, amount, i
    }

    private static Result<String> validateRequestId(String requestId) {
        return Verify.ensure(requestId, Verify.Is::notBlank)
            .mapError(_ -> Causes.cause("Request ID is required"));
    }
```

```
    private static Result<String> validateInitiator(String initiatedBy) {
        return Verify.ensure(initiatedBy, Verify.Is::notBlank)
            .mapError(_ -> Causes.cause("Initiator is required"));
    }
}
```

Use Case with Aspects

```
public interface TransferFunds {

    Promise<TransferResult> execute(TransferRequest request);

    // Step interfaces
    interface CheckIdempotency {
        Promise<ValidTransfer> apply(ValidTransfer transfer);
    }

    interface CheckBalance {
        Promise<ValidTransfer> apply(ValidTransfer transfer);
    }

    interface ExecuteTransfer {
        Promise<TransferResult> apply(ValidTransfer transfer);
    }

    interface AuditLog {
        Promise<Unit> apply(AuditEntry entry);
    }

    // Factory with aspect composition
    static TransferFunds transferFunds(
        CheckIdempotency checkIdempotency,
        CheckBalance checkBalance,
        ExecuteTransfer executeTransfer,
        AuditLog auditLog,
        TimeSpan timeout,
        RetryPolicy retryPolicy
    ) {
        // Wrap executeTransfer with aspects
        var decoratedExecute = withRetry(retryPolicy,
            withTimeout(timeout,
                withAudit(auditLog, executeTransfer)
            )
        );
```

```
        return request -> ValidTransfer.validTransfer(request)
            .async()
            .flatMap(checkIdempotency::apply)
            .flatMap(checkBalance::apply)
            .flatMap(decoratedExecute);
    }

    // ASPECT: Timeout
    private static Fn1<ValidTransfer, Promise<TransferResult>> withTimeout(
        TimeSpan timeout,
        Fn1<ValidTransfer, Promise<TransferResult>> step
    ) {
        return transfer -> step.apply(transfer).timeout(timeout);
    }

    // ASPECT: Retry
    private static Fn1<ValidTransfer, Promise<TransferResult>> withRetry(
        RetryPolicy policy,
        Fn1<ValidTransfer, Promise<TransferResult>> step
    ) {
        return transfer -> retryWithPolicy(policy, () -> step.apply(transfer));
    }

    private static Promise<TransferResult> retryWithPolicy(
        RetryPolicy policy,
        Supplier<Promise<TransferResult>> operation
    ) {
        return operation.get()
            .recover(cause -> {
                if (policy.shouldRetry(cause) && policy.attemptsRemaining() > 0) {
                    return retryWithPolicy(policy.decrementAttempts(), operation);
                }
                return cause.promise();
            });
    }

    // ASPECT: Audit (wraps execution with before/after logging)
    private static Fn1<ValidTransfer, Promise<TransferResult>> withAudit(
        AuditLog auditLog,
        Fn1<ValidTransfer, Promise<TransferResult>> step
    ) {
        return transfer -> {
            var startTime = Instant.now();

            return step.apply(transfer)
                .onResult(result -> logAuditEntry(auditLog, transfer, result, startTime))
```

```
        };
    }

    private static void logAuditEntry(
        AuditLog auditLog,
        ValidTransfer transfer,
        Result<TransferResult> result,
        Instant startTime
    ) {
        var entry = new AuditEntry(
            transfer.requestId(),
            "TRANSFER",
            transfer.initiatedBy(),
            startTime,
            Instant.now(),
            result.isSuccess() ? "SUCCESS" : "FAILURE",
            result.fold(Cause::message, r -> "Transfer completed: " + r.transferId().valu
        );

        // Best-effort: log but don't fail if audit fails
        auditLog.apply(entry)
            .onFailure(e -> { /* log to fallback */ });
    }
}
```

Supporting Types

AuditEntry

```
public record AuditEntry(
    String requestId,
    String operation,
    String initiatedBy,
    Instant startedAt,
    Instant completedAt,
    String outcome,
    String details
) {}
```

RetryPolicy

```
public record RetryPolicy(
    int maxAttempts,
```

```
    int attemptsRemaining,
    Set<Class<? extends Cause>> retryableErrors
) {
    public static RetryPolicy retryPolicy(int maxAttempts, Set<Class<? extends Cause>> re
        return new RetryPolicy(maxAttempts, maxAttempts, retryableErrors);
    }

    public boolean shouldRetry(Cause cause) {
        return retryableErrors.stream()
            .anyMatch(errorClass -> errorClass.isInstance(cause));
    }

    public RetryPolicy decrementAttempts() {
        return new RetryPolicy(maxAttempts, attemptsRemaining - 1, retryableErrors);
    }
}
```

Step Implementations

CheckIdempotency

```
public class IdempotencyChecker implements TransferFunds.CheckIdempotency {
    private final IdempotencyStore store;

    public IdempotencyChecker(IdempotencyStore store) {
        this.store = store;
    }

    @Override
    public Promise<ValidTransfer> apply(ValidTransfer transfer) {
        return Promise.lift(
            _ -> TransferError.General.TRANSFER_FAILED,
            () -> store.exists(transfer.requestId())
        ).flatMap(exists -> exists
            ? TransferError.General.DUPLICATE_REQUEST.promise()
            : Promise.success(transfer));
    }
}
```

CheckBalance

```
public class BalanceChecker implements TransferFunds.CheckBalance {
    private final AccountRepository accounts;
```

```
    public BalanceChecker(AccountRepository accounts) {
        this.accounts = accounts;
    }

    @Override
    public Promise<ValidTransfer> apply(ValidTransfer transfer) {
        return accounts.getBalance(transfer.sourceAccount())
            .flatMap(balance -> balance.isGreaterThan(transfer.amount())
                ? Promise.success(transfer)
                : TransferError.General.INSUFFICIENT_FUNDS.promise());
    }
}
```

ExecuteTransfer

```
public class TransferExecutor implements TransferFunds.ExecuteTransfer {
    private final AccountRepository accounts;
    private final IdempotencyStore idempotencyStore;

    @Override
    public Promise<TransferResult> apply(ValidTransfer transfer) {
        return Promise.lift(
            _ -> TransferError.General.TRANSFER_FAILED,
            () -> executeInTransaction(transfer)
        );
    }

    private TransferResult executeInTransaction(ValidTransfer transfer) {
        // Transaction boundary
        accounts.debit(transfer.sourceAccount(), transfer.amount());
        accounts.credit(transfer.destinationAccount(), transfer.amount());
        idempotencyStore.record(transfer.requestId());

        return new TransferResult(
            new TransferId(UUID.randomUUID().toString()),
            transfer.amount(),
            Instant.now()
        );
    }
}
```

Key Points: Aspects Pattern

What Aspects Are

Aspects are **higher-order functions** that wrap step execution with cross-cutting concerns. They form a composition chain:

```
Request -> [Retry -> [Timeout -> [Audit -> Execute]]]
```

Aspect Composition Order

The order matters. Aspects are applied **outside-in**:

```
var decorated = withRetry(policy,      // 1. Outer: retry wraps everything
    withTimeout(timeout,                // 2. Middle: timeout per attempt
        withAudit(auditLog, execute)    // 3. Inner: audit each attempt
    )
);
```

This means: - Retry wraps timeout (each retry attempt has its own timeout) - Timeout wraps audit (audit happens within timeout) - Audit wraps execute (execution is audited)

Standard Composition Order

For most use cases:

1. **Metrics/Logging** (outermost) - Observe everything
2. **Timeout** - Limit total time
3. **Circuit Breaker** - Fail fast if service unhealthy
4. **Retry** - Retry transient failures
5. **Rate Limit** - Control request rate
6. **Business Logic** (innermost) - Actual operation

Aspect Independence

Each aspect is independent and testable:

```
@Test
void withTimeout_fails_whenTimeoutExceeds() {
    var slowStep = transfer -> Promise.<TransferResult>promise()
        .timeout(TimeSpan.timeSpan(100).millis());  // Never resolves

    var decorated = withTimeout(TimeSpan.timeSpan(50).millis(), slowStep);

    decorated.apply(validTransfer())
        .await()
        .onSuccess(Assertions::fail);  // Should timeout
}
```

Testing

```
class TransferFundsTest {

    @Nested
    class Aspects {

        @Test
        void execute_retriesOnTransientFailure() {
            var attempts = new AtomicInteger(0);

            TransferFunds.ExecuteTransfer failingThenSucceeding = transfer -> {
                if (attempts.incrementAndGet() < 3) {
                    return new TransientError().promise();
                }
                return Promise.success(new TransferResult(
                    new TransferId("tx-123"),
                    transfer.amount(),
                    Instant.now()
                ));
            };

            var policy = RetryPolicy.retryPolicy(3, Set.of(TransientError.class));

            var useCase = TransferFunds.transferFunds(
                t -> Promise.success(t),
                t -> Promise.success(t),
                failingThenSucceeding,
                e -> Promise.success(Unit.INSTANCE),
                TimeSpan.timeSpan(5).seconds(),
                policy
            );

            useCase.execute(validRequest())
                .await()
                .onFailure(Assertions::fail)
                .onSuccess(result -> assertEquals(3, attempts.get()));
        }

        @Test
        void execute_auditsSuccessfulTransfer() {
            var auditedEntries = new ArrayList<AuditEntry>();

            TransferFunds.AuditLog auditLog = entry -> {
```

```
            auditedEntries.add(entry);
            return Promise.success(Unit.INSTANCE);
        };

        var useCase = TransferFunds.transferFunds(
            t -> Promise.success(t),
            t -> Promise.success(t),
            t -> Promise.success(new TransferResult(new TransferId("tx-123"), t.amoun
            auditLog,
            TimeSpan.timeSpan(5).seconds(),
            RetryPolicy.retryPolicy(0, Set.of())
        );

        useCase.execute(validRequest())
            .await()
            .onFailure(Assertions::fail);

        assertEquals(1, auditedEntries.size());
        assertEquals("SUCCESS", auditedEntries.get(0).outcome());
    }

    @Test
    void execute_auditsFailedTransfer() {
        var auditedEntries = new ArrayList<AuditEntry>();

        TransferFunds.AuditLog auditLog = entry -> {
            auditedEntries.add(entry);
            return Promise.success(Unit.INSTANCE);
        };

        var useCase = TransferFunds.transferFunds(
            t -> Promise.success(t),
            t -> TransferError.General.INSUFFICIENT_FUNDS.promise(),
            t -> Promise.success(new TransferResult(new TransferId("tx-123"), t.amoun
            auditLog,
            TimeSpan.timeSpan(5).seconds(),
            RetryPolicy.retryPolicy(0, Set.of())
        );

        useCase.execute(validRequest())
            .await()
            .onSuccess(Assertions::fail);

        // Audit still captured even though transfer failed
        assertEquals(1, auditedEntries.size());
        assertEquals("FAILURE", auditedEntries.get(0).outcome());
```

```
        }

        @Test
        void execute_succeedsEvenIfAuditFails() {
            TransferFunds.AuditLog failingAudit = entry ->
                Causes.cause("Audit service unavailable").promise();

            var useCase = TransferFunds.transferFunds(
                t -> Promise.success(t),
                t -> Promise.success(t),
                t -> Promise.success(new TransferResult(new TransferId("tx-123"), t.amour
                failingAudit,
                TimeSpan.timeSpan(5).seconds(),
                RetryPolicy.retryPolicy(0, Set.of())
            );

            // Transfer should succeed even if audit fails
            useCase.execute(validRequest())
                .await()
                .onFailure(Assertions::fail);
        }
    }

    private TransferRequest validRequest() {
        return new TransferRequest(
            "req-123",
            "acc-source",
            "acc-dest",
            "100.00",
            "user@example.com"
        );
    }

    static class TransientError implements Cause {
        @Override
        public String message() {
            return "Transient error";
        }
    }
}
```

Exercises

1. **Add circuit breaker:** Implement a circuit breaker aspect that fails fast after N consecutive failures.
2. **Add rate limiting:** Implement a rate limit aspect that rejects requests exceeding N per second.
3. **Add metrics:** Create a metrics aspect that records duration and success/failure counts.
4. **Conditional aspects:** Modify the factory to only apply audit logging for transfers above a threshold amount.

Summary

TransferFunds demonstrates:

- **Aspects as higher-order functions** - Wrap step execution with cross-cutting concerns
- **Composition order matters** - Outside-in application (retry wraps timeout wraps audit)
- **Independent and testable** - Each aspect tested in isolation
- **Best-effort operations** - Audit failure doesn't fail the transfer
- **Idempotency pattern** - Prevent duplicate processing
- **Retry with policy** - Configurable retry behavior based on error type

Chapter 15: Project Structure & Framework Integration

What You'll Learn

- Vertical slicing philosophy and package organization
- Module organization for larger systems
- Framework integration with Spring Boot and JOOQ
- Where types go (placement rules)

Prerequisites: Chapter 14: More Examples

Vertical Slicing Philosophy

This technology organizes code around **vertical slices** - each use case is self-contained with its own business logic, validation, and error handling. Unlike architectures that centralize all business logic into one functional core, we **isolate business logic within each use case package**.

Why vertical slicing (by criteria): - **Complexity**: Minimizes coupling between unrelated features (+3) - **Business/Technical Ratio**: Package names reflect domain use cases, not technical layers (+2) - **Mental Overhead**: All related code in one place - less navigation (+2) - **Design Impact**: Forces proper boundaries - business logic cannot leak between use cases (+2)

Package Structure

```
com.example.app/
|-- usecase/
|   |-- registeruser/              # Use case 1 (vertical slice)
|   |   |-- RegisterUser.java      # Use case interface + factory
|   |   |-- RegistrationError.java # Sealed error interface
|   |   |-- [internal types]       # ValidRequest, intermediate records
|   |
|   |-- getuserprofile/            # Use case 2 (vertical slice)
|       |-- GetUserProfile.java
```

```
|       |-- ProfileError.java
|       |-- [internal types]
|
|-- domain/
|   |-- shared/                    # Reusable value objects only
|       |-- Email.java
|       |-- Password.java
|       |-- UserId.java
|
|-- adapter/
|   |-- rest/                      # Inbound adapters (HTTP)
|   |   |-- UserController.java
|   |
|   |-- persistence/               # Outbound adapters (DB)
|       |-- JooqUserRepository.java
|
|-- config/                        # Framework configuration
    |-- UseCaseConfig.java
```

Package Placement Rules

Use Case Packages

`com.example.app.usecase.<usecasename>`: - Use case interface and factory method - Error types specific to this use case (sealed interface) - Step interfaces (nested in use case interface) - Internal validation types (ValidRequest, intermediate records)

Rule: If a type is used only by this use case, it stays here.

Domain Shared

`com.example.app.domain.shared`: - Value objects reused across multiple use cases

Rule: Move here immediately when a second use case needs the same value object.

Anti-pattern: Don't create this upfront - let reuse drive the move.

Adapter Packages

`com.example.app.adapter.*`: - `adapter.rest` - HTTP controllers, request/response DTOs - `adapter.persistence` - Database repositories, ORM entities - `adapter.messaging` - Message queue consumers/producers - `adapter.external` - HTTP clients for external services

Rule: Adapters implement step interfaces from use cases.

Config Package

`com.example.app.config`: - Spring/framework configuration - Bean wiring, dependency injection setup

Rule: No business logic, only infrastructure configuration.

Module Organization (Optional)

For larger systems, split into Gradle/Maven modules:

```
:domain          # Pure Java - value objects
:application     # Use cases and step interfaces
:adapters        # Adapter implementations
:bootstrap       # Main class, configuration
```

When to Use Modules

- Team size > 5 developers
- Multiple deployment units from same codebase
- Enforcing compile-time dependency boundaries
- Independent library publication

When Single Module is Sufficient

- Small to medium teams (< 5 developers)
- Monolithic deployment
- Package conventions provide sufficient structure

Module Dependencies

```
domain         -> (no dependencies)
  |
application    -> domain
  |
adapters       -> application, domain
  |
bootstrap      -> adapters, application, domain
```

Framework Integration

Complete Example: Spring REST -> Use Case -> JOOQ

1. Use Case (Functional Core)

```
public interface GetUserProfile {
    record Request(String userId) {}
    record Response(String userId, String email, String displayName) {
        static Response fromUser(User user) {
            return new Response(user.id().value(),
                                user.email().value(),
                                user.displayName());
        }
    }

    Promise<Response> execute(Request request);

    interface FetchUser {
        Promise<User> apply(UserId userId);
    }

    static GetUserProfile getUserProfile(FetchUser fetchUser) {
        return request -> UserId.userId(request.userId())
                                .async()
                                .flatMap(fetchUser::apply)
                                .map(Response::fromUser);
    }
}
```

Pure business logic. No framework dependencies.

2. REST Controller (Adapter In)

```
@RestController
@RequestMapping("/api/users")
public class UserController {
    private final GetUserProfile getUserProfile;

    public UserController(GetUserProfile getUserProfile) {
        this.getUserProfile = getUserProfile;
    }

    @GetMapping("/{userId}")
    public ResponseEntity<?> getProfile(@PathVariable String userId) {
        var request = new GetUserProfile.Request(userId);
```

```
        return getUserProfile.execute(request)
            .await()
            .fold(this::toErrorResponse,
                  response -> ResponseEntity.ok(response));
    }

    private ResponseEntity<?> toErrorResponse(Cause cause) {
        return switch (cause) {
            case ProfileError.UserNotFound _ ->
                ResponseEntity.status(HttpStatus.NOT_FOUND)
                              .body(Map.of("error", cause.message()));

            case ProfileError.InvalidUserId _ ->
                ResponseEntity.status(HttpStatus.BAD_REQUEST)
                              .body(Map.of("error", cause.message()));

            default ->
                ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
                              .body(Map.of("error", "Internal server error"));
        };
    }
}
```

Thin adapter: HTTP -> Request -> use case -> Response/Cause -> HTTP.

3. JOOQ Repository (Adapter Out)

```
@Repository
public class JooqUserRepository implements GetUserProfile.FetchUser {
    private final DSLContext dsl;

    public JooqUserRepository(DSLContext dsl) {
        this.dsl = dsl;
    }

    public Promise<User> apply(UserId userId) {
        return Promise.lift(ProfileError.DatabaseFailure::cause,
            () -> dsl.selectFrom(USERS)
                     .where(USERS.ID.eq(userId.value()))
                     .fetchOptional()
                     .map(this::toDomain)
                     .orElseThrow(() -> new NotFoundException()));
    }

    private User toDomain(Record record) {
        return new User(
            new UserId(record.get(USERS.ID)),
```

```
            new Email(record.get(USERS.EMAIL)),
            record.get(USERS.DISPLAY_NAME)
        );
    }
}
```

Wraps JOOQ exceptions in domain Causes.

4. Wiring (Spring Config)

```
@Configuration
public class UseCaseConfig {

    @Bean
    public GetUserProfile getUserProfile(JooqUserRepository repository) {
        return GetUserProfile.getUserProfile(repository);
    }
}
```

Key Principles

1. Vertical Slicing

Each use case package is a vertical slice containing everything needed for that feature.

2. Minimal Sharing

Only share value objects when truly reusable. Premature sharing creates coupling.

3. Framework at Edges

Business logic (use cases, domain) has zero framework dependencies. Adapters handle framework integration.

4. Clear Dependencies

- Use cases depend on: domain.shared
- Adapters depend on: use cases (implement step interfaces)
- Config depends on: use cases + adapters (wires them together)
- **Never**: use case depending on adapter

5. Adapter Isolation

All I/O operations live in adapters. Framework swapping (Spring -> Micronaut) affects only adapters.

Where Things Go

Type	Location	Rationale
Use case interface	`usecase.<name>`	Entry point for feature
Step interfaces	Inside use case	Part of use case contract
Errors (sealed)	`usecase.<name>`	Feature-specific
ValidRequest	`usecase.<name>`	Internal validation
Shared value objects	`domain.shared`	Reused across features
Controllers	`adapter.rest`	HTTP handling
Repositories	`adapter.persistence`	Database access
Config	`config`	Bean wiring

Key Takeaways

1. **Vertical slices** - Each use case is self-contained
2. **Move to shared on reuse** - Not upfront
3. **Framework at edges** - Pure business logic in use cases
4. **Adapters implement step interfaces** - Clear contracts
5. **Modules when needed** - Don’t prematurely modularize

Exercises

See Appendix B for exercises on: - Exercise 5.4: Package organization - Exercise 6.2: Module setup

---

What’s Next

Chapter 16 covers systematic application - checkpoints and checklists for writing and reviewing JBCT code.

Chapter 16: Systematic Application Guide

What You'll Learn

- 8 checkpoints for consistent JBCT application
- Quick reference tables for immediate lookup
- Violation -> Fix patterns for common mistakes
- Application order for new code and reviews

Prerequisites: Chapter 15: Project Structure

The Checkpoint Approach

JBCT application works through **checkpoints** - specific moments during coding where you pause and verify compliance. Each checkpoint has: - **Trigger**: When to apply this checkpoint - **Rules**: What to check - **Fixes**: How to correct violations

The goal is 100% compliance through systematic verification, not heroic effort.

CHECKPOINT 1: Before Writing Any Lambda

Trigger: You're about to write `->`

Rules to Check

Rule	Check	Fix
L1	Is method reference possible?	Use `Type::method`
L2	Does lambda have braces `{}`?	Extract to named method
L3	Nested monadic operations inside?	Extract inner operation
L4	Control flow (if/switch/try)?	Extract to named method
L5	Multiple statements?	Extract to named method

Allowed Lambda Forms (Exhaustive)

```
// Method references (ALWAYS PREFERRED)
.map(Email::new)
.flatMap(this::validate)

// Single-value expression (no braces)
.map(value -> expression)
.filter(s -> !s.isBlank())

// Multi-value expression (no braces)
.map((a, b) -> new Pair(a, b))
```

Extraction Pattern

```
// BEFORE (violation)
.map(data -> {
    cache.put(key, data);
    log.info("Cached: {}", data);
    return data.size();
})

// AFTER (compliant)
.map(this::cacheAndCount)

private int cacheAndCount(Data data) {
    cache.put(key, data);
    log.info("Cached: {}", data);
    return data.size();
}
```

CHECKPOINT 2: Choosing Return Type

Trigger: Writing a method signature

Decision Tree

```
Can this operation fail?
|-- NO: Can the value be absent?
|   |-- NO -> return T
|   |-- YES -> return Option<T>
|-- YES: Is it async/IO?
    |-- NO -> return Result<T>
    |-- YES -> return Promise<T>
```

Rules to Check

Rule	Check	Fix
R1	Does Result always succeed?	Change to T
R2	Is Option always present?	Change to T
R3	Using `Promise<Result<T>>`?	Use `Promise<T>` only
R4	Returning Void?	Use Unit
R5	Returning null?	Use `Option<T>`

CHECKPOINT 3: Writing Factory Methods

Trigger: Creating construction logic for a type

Rules to Check

Rule	Check	Fix
F1	Name follows `TypeName.typeName()`?	Rename
F2	Validation happens at construction?	Move to factory
F3	Return type matches validation needs?	Apply Checkpoint 2
F4	Constructor exposed publicly?	Make factory only entry

Pattern

```
public record Email(String value) {
    // Factory with validation -> Result<T>
    public static Result<Email> email(String raw) {
        return Verify.ensure(raw, Verify.Is::notNull)
            .map(String::trim)
            .filter(INVALID_EMAIL, s -> PATTERN.matcher(s).matches())
            .map(Email::new);
    }
}

public record Config(DbUrl url, DbPassword pass) {
    // Factory without validation -> T
    public static Config config(DbUrl url, DbPassword pass) {
        return new Config(url, pass);
    }
}
```

CHECKPOINT 4: Designing a Class/Interface

Trigger: Creating a new type

Rules to Check

Rule	Check	Fix
D1	What zone does this belong to?	Place in correct package
D2	Does it mix I/O with domain logic?	Split into separate types
D3	Are primitives used for domain concepts?	Extract value objects
D4	Does naming match the zone?	Adjust naming style

Zone Placement

```
Zone A (Entry): Controllers, handlers, main
  -> Business action verbs: handleRegistration(), processOrder()

Zone B (Domain): Use cases, value objects, domain services
  -> Domain vocabulary: Email.email(), ValidRequest.validRequest()

Zone C (Infrastructure): DB, external APIs, config loading
  -> Technical names: loadAllGenerations(), saveUser(), readFile()
```

CHECKPOINT 5: Writing Monadic Chains

Trigger: Chaining `.map()/.flatMap()/.filter()` etc.

Rules to Check

Rule	Check	Fix
M1	Single pattern per method?	Extract mixed patterns
M2	Chain length <= 5 steps?	Split into composed methods
M3	Side effects only in terminal ops?	Move to `.onSuccess()/.onFailure()`
M4	Logging mixed with logic?	Move logging to appropriate layer

Pattern Separation

```
// VIOLATION: Mixing Sequencer + Fork-Join
return validate(request)
    .flatMap(req -> Result.all(
        checkInventory(req),
        validatePayment(req)
    ).map((inv, pay) -> proceed(req)));

// FIX: Extract Fork-Join
return validate(request)
    .flatMap(this::validateOrder)
    .flatMap(this::processOrder);

private Result<ValidRequest> validateOrder(ValidRequest req) {
    return Result.all(checkInventory(req), validatePayment(req))
        .map((inv, pay) -> req);
}
```

CHECKPOINT 6: Adding Logging

Trigger: Adding log statements

Rules to Check

Rule	Check	Fix
G1	Is logging conditional on data?	Remove condition
G2	Logger passed as parameter?	Move logging to owner
G3	Logging in pure transformation?	Move to terminal operation
G4	Duplicate logging across layers?	Single layer logs

Ownership Pattern

```
// VIOLATION: Caller logs for callee
cache.refresh()
    .onSuccess(count -> log.debug("Refreshed {}", count));

// FIX: Cache owns its logging
// In GenerationCache:
public Result<Integer> refresh() {
    return doRefresh()
        .onSuccess(count -> log.debug("Refreshed {}", count));
}
```

CHECKPOINT 7: Review Completeness

Trigger: Before submitting code for review

Verification Checklist

- ☐ Every lambda checked against L1-L5
- ☐ Every return type checked against R1-R5
- ☐ Every factory method checked against F1-F4
- ☐ Every new type checked against D1-D4
- ☐ Every monadic chain checked against M1-M4
- ☐ Every log statement checked against G1-G4
- ☐ No FQCNs in code (use imports)
- ☐ Test names follow `methodName_outcome_condition`

CHECKPOINT 8: Implementation Patterns

Trigger: Choosing implementation style

Nested Record vs Lambda Pattern

```
Does implementation need state beyond parameters?
|-- YES -> Use nested record pattern
|-- NO: Is it a functional interface?
    |-- YES -> Use lambda pattern
    |-- NO -> Use nested record pattern
```

Nested record pattern (state needed):

```
static GenerationCache generationCache(WorkConfig config) {
    record generationCache(
        AtomicReference<Instant> lastRefreshTime,
        ConcurrentMap<String, Generation> cache,
        WorkConfig config
    ) implements GenerationCache {
        @Override
        public Result<Unit> refresh() { ... }
    }

    return new generationCache(
        new AtomicReference<>(Instant.EPOCH),
        new ConcurrentHashMap<>(),
        config
```

```
    );
}
```

Lambda pattern (functional interface, no state):

```
static Step validate(Validator validator) {
    return ctx -> validator.validate(ctx.input());
}
```

Quick Reference: Violation -> Fix Patterns

Violation	Detection	Fix
Multi-statement lambda	`{ }` with multiple lines	Extract to method
Nested monadic ops	`.flatMap(x -> y.map(...))`	Extract inner to method
Always-succeeding Result	`Result.success(new X())`	Return X directly
Mixed I/O and domain	File/DB ops in domain class	Split to adapter
Primitive obsession	`String url`, `int poolSize`	Create value object
Conditional logging	`if (x) log.debug()`	Remove condition
Logger as parameter	`method(Logger log)`	Move logging to owner

Application Order

When Implementing New Code

1. **Design phase**: Apply Checkpoint 4 (class design)
2. **Signature phase**: Apply Checkpoint 2 (return types), Checkpoint 3 (factories)
3. **Implementation phase**: Apply Checkpoint 1 (lambdas), Checkpoint 5 (chains), Checkpoint 8 (patterns)
4. **Polish phase**: Apply Checkpoint 6 (logging)
5. **Completion phase**: Apply Checkpoint 7 (review)

When Reviewing Existing Code

1. **Scan for lambdas** (->) - apply Checkpoint 1

2. **Scan for return types** - apply Checkpoint 2
3. **Scan for factories** - apply Checkpoint 3
4. **Scan for mixed responsibilities** - apply Checkpoint 4
5. **Scan for logging** - apply Checkpoint 6
6. **Final verification** - apply Checkpoint 7

Key Takeaways

1. **8 checkpoints** - Cover all aspects of JBCT
2. **Apply systematically** - Violations caught early, not in review
3. **Quick reference tables** - For immediate lookup
4. **Order matters** - Design -> Signature -> Implementation -> Polish
5. **100% compliance** - Achievable through systematic verification

---

Exercises

See Appendix B for exercises on: - Exercise 6.1: Checkpoint application practice - Exercise 6.3: Code review using checkpoints

---

What's Next

Chapter 17 covers migration strategies for adopting JBCT in existing codebases.

Chapter 17: Migration Strategies

This chapter provides a practical playbook for adopting JBCT in existing codebases. Whether you're working with a legacy monolith or a modern Spring Boot application, you'll learn how to migrate incrementally without disrupting ongoing development.

Assessment: Is Your Codebase Ready?

Before migrating, evaluate your starting point.

Readiness Checklist

Factor	Ready	Needs Work
Java version	17+ (records, sealed interfaces)	Upgrade first
Build tool	Maven or Gradle	Any modern build tool
Test coverage	Some tests exist	Add tests before refactoring
Team size	Any	Larger teams need more coordination
Release cycle	Regular releases	Establish before major changes

Code Smell Indicators

These patterns indicate high-value migration targets:

```
// 1. Validation scattered across layers
@PostMapping("/users")
public ResponseEntity<?> createUser(@RequestBody UserDto dto) {
    if (dto.getEmail() == null || dto.getEmail().isEmpty()) {
        return ResponseEntity.badRequest().body("Email required");
    }
    if (!dto.getEmail().contains("@")) {
        return ResponseEntity.badRequest().body("Invalid email");
    }
    // ... more validation
    userService.create(dto);  // Service also validates!
}
```

```java
// 2. Null checks everywhere
public Order processOrder(Order order) {
    if (order == null) return null;
    if (order.getCustomer() == null) return null;
    if (order.getCustomer().getAddress() == null) {
        throw new ValidationException("Address required");
    }
    // ... actual logic buried under null checks
}

// 3. Exception-based control flow
public User findUser(Long id) {
    try {
        return repository.findById(id)
            .orElseThrow(() -> new UserNotFoundException(id));
    } catch (DataAccessException e) {
        throw new ServiceException("Database error", e);
    }
}

// 4. Mixed concerns in services
@Service
public class UserService {
    public User register(UserDto dto) {
        // Validation
        validateEmail(dto.getEmail());
        validatePassword(dto.getPassword());

        // Business logic
        if (userRepository.existsByEmail(dto.getEmail())) {
            throw new EmailExistsException();
        }

        // Infrastructure
        String hashedPassword = passwordEncoder.encode(dto.getPassword());
        User user = new User(dto.getEmail(), hashedPassword);
        return userRepository.save(user);
    }
}
```

Migration Phases

Phase 1: Value Objects Only (Week 1-2)

Goal: Introduce parse-don't-validate without changing existing code structure.

What to do: 1. Add Pragmatica Lite dependency 2. Create value objects for commonly validated fields 3. Use value objects in new code 4. Gradually replace primitives in existing code

Start with these value objects: - Email - Password - UserId / CustomerId / OrderId - Money / Currency - Phone number

Example: Adding Email value object

```
// NEW: Value object
public record Email(String value) {
    private static final Pattern PATTERN = Pattern.compile("^[a-z0-9+_.-]+@[a-z0-9.-]+$")
    private static final Cause INVALID = Causes.cause("Invalid email format");

    public static Result<Email> email(String raw) {
        return Verify.ensure(raw, Verify.Is::notBlank)
            .map(String::trim)
            .map(String::toLowerCase)
            .flatMap(Verify.ensureFn(INVALID, Verify.Is::matches, PATTERN))
            .map(Email::new);
    }
}

// EXISTING: Service method (unchanged initially)
@Service
public class UserService {
    public User register(String email, String password) {
        // Existing validation still here
        if (email == null || !email.contains("@")) {
            throw new ValidationException("Invalid email");
        }
        // ... rest of method
    }
}

// NEW: Parallel method using value object
public Result<User> registerValidated(Email email, Password password) {
    // No validation needed - email and password are already valid
    return checkEmailUnique(email)
        .flatMap(e -> hashPassword(password))
        .map(hashed -> createUser(email, hashed));
}
```

Migration pattern: 1. Create value object 2. Add parallel method accepting value object 3. Update callers one by one 4. Delete old method when all callers migrated

Phase 2: Result in New Code (Week 3-4)

Goal: Stop adding new exception-based code.

Team agreement: - All new methods return `Result<T>` or `Promise<T>` for fallible operations - New validation uses value object factories - Exceptions only at adapter boundaries (wrapping external libraries)

Example: New feature with Result

```
// NEW FEATURE: Apply discount code
public interface ApplyDiscount {
    Result<DiscountedPrice> apply(OrderId orderId, DiscountCode code);
}

// Implementation
public class ApplyDiscountImpl implements ApplyDiscount {
    @Override
    public Result<DiscountedPrice> apply(OrderId orderId, DiscountCode code) {
        return findOrder(orderId)
            .flatMap(order -> validateCode(code, order))
            .map(discount -> calculateDiscountedPrice(order, discount));
    }

    private Result<Order> findOrder(OrderId id) {
        return Option.option(orderRepository.findById(id.value()))
            .toResult(OrderError.NOT_FOUND);
    }

    private Result<Discount> validateCode(DiscountCode code, Order order) {
        return discountRepository.findByCode(code.value())
            .toResult(DiscountError.INVALID_CODE)
            .filter(DiscountError.EXPIRED, d -> !d.isExpired())
            .filter(DiscountError.MIN_ORDER, d -> order.total().isGreaterThan(d.minOrder(
    }
}
```

Bridging old and new:

```
// Controller bridges exception world to Result world
@PostMapping("/orders/{orderId}/discount")
public ResponseEntity<?> applyDiscount(
    @PathVariable String orderId,
    @RequestBody DiscountRequest request
) {
    return Result.all(
        OrderId.orderId(orderId),
```

```
            DiscountCode.discountCode(request.code())
        )
        .flatMap((oid, code) -> applyDiscount.apply(oid, code))
        .fold(
            cause -> toErrorResponse(cause),
            price -> ResponseEntity.ok(price)
        );
}
```

Phase 3: Extract Use Cases (Week 5-8)

Goal: Separate business logic from framework code.

Process: 1. Identify a service method with clear business logic 2. Extract to use case interface 3. Move implementation to use case factory 4. Convert service to thin adapter

Before: Fat service

```
@Service
public class OrderService {
    @Autowired private OrderRepository orderRepository;
    @Autowired private PaymentGateway paymentGateway;
    @Autowired private NotificationService notificationService;

    @Transactional
    public Order placeOrder(OrderDto dto) {
        // Validation
        if (dto.getItems().isEmpty()) {
            throw new ValidationException("Order must have items");
        }

        // Check inventory
        for (ItemDto item : dto.getItems()) {
            if (!inventoryService.hasStock(item.getProductId(), item.getQuantity())) {
                throw new InsufficientStockException(item.getProductId());
            }
        }

        // Process payment
        PaymentResult payment;
        try {
            payment = paymentGateway.charge(dto.getPaymentMethod(), calculateTotal(dto));
        } catch (PaymentException e) {
            throw new OrderException("Payment failed", e);
        }

        // Create order
```

```
        Order order = new Order(dto.getCustomerId(), dto.getItems(), payment.getTransacti
        orderRepository.save(order);

        // Send notification (best effort)
        try {
            notificationService.sendOrderConfirmation(order);
        } catch (Exception e) {
            log.warn("Failed to send notification", e);
        }

        return order;
    }
}
```

After: Use case + thin service

```
// Use case interface
public interface PlaceOrder {
    Promise<OrderConfirmation> execute(OrderRequest request);

    interface CheckInventory {
        Promise<ValidOrder> apply(ValidOrder order);
    }

    interface ProcessPayment {
        Promise<PaymentConfirmation> apply(ValidOrder order);
    }

    interface SaveOrder {
        Promise<OrderId> apply(ValidOrder order, PaymentConfirmation payment);
    }

    interface SendNotification {
        Promise<Unit> apply(OrderId orderId);
    }

    static PlaceOrder placeOrder(
        CheckInventory checkInventory,
        ProcessPayment processPayment,
        SaveOrder saveOrder,
        SendNotification sendNotification
    ) {
        return request -> ValidOrder.validOrder(request)
            .async()
            .flatMap(checkInventory::apply)
            .flatMap(order -> processPayment.apply(order)
                .map(payment -> Pair.of(order, payment)))
```

```
            .flatMap(pair -> saveOrder.apply(pair.first(), pair.second()))
            .onSuccess(orderId -> sendNotification.apply(orderId)
                .onFailure(e -> { /* log but don't fail */ }))
            .map(OrderConfirmation::new);
    }
}

// Thin service (adapter)
@Service
public class OrderService {
    private final PlaceOrder placeOrder;

    public OrderService(
        InventoryChecker inventoryChecker,
        PaymentProcessor paymentProcessor,
        JooqOrderRepository orderRepository,
        EmailNotificationSender notificationSender
    ) {
        this.placeOrder = PlaceOrder.placeOrder(
            inventoryChecker,
            paymentProcessor,
            orderRepository,
            notificationSender
        );
    }

    public Order placeOrder(OrderDto dto) {
        return placeOrder.execute(toRequest(dto))
            .await()
            .fold(
                cause -> { throw toException(cause); },
                confirmation -> toOrder(confirmation)
            );
    }
}
```

Phase 4: Adapter Isolation (Week 9-12)

Goal: All I/O wrapped in adapter leaves with Promise.lift.

Identify adapters: - Database repositories - HTTP clients - Message queue producers/consumers - File system access - Cache clients

Example: Repository adapter

```
// Step interface
public interface SaveUser {
    Promise<UserId> apply(ValidUser user);
```

```
}

// Adapter implementation
@Repository
public class JooqUserRepository implements SaveUser {
    private final DSLContext dsl;

    @Override
    public Promise<UserId> apply(ValidUser user) {
        return Promise.lift(
            RepositoryError.DatabaseFailure::new,
            () -> {
                String id = dsl.insertInto(USERS)
                    .set(USERS.EMAIL, user.email().value())
                    .set(USERS.PASSWORD_HASH, user.passwordHash().value())
                    .returningResult(USERS.ID)
                    .fetchSingle()
                    .value1();
                return new UserId(id);
            }
        );
    }
}
```

Interoperability with Java Standard Library

`Optional<T>` → `Option<T>`

```
// Direct conversion
Optional<User> javaOptional = repository.findById(id);
Option<User> option = Option.from(javaOptional);

// In adapter methods
public Option<User> findById(UserId id) {
    return Option.from(jpaRepository.findById(id.value()));
}

// Back to Optional (for framework integration)
Optional<User> back = option.toOptional();
```

CompletableFuture<T> → Promise<T>

```
// Wrapping CompletableFuture in adapter
public Promise<User> fetchUser(UserId id) {
    var promise = Promise.<User>promise();

    CompletableFuture<User> cf = httpClient.getUser(id.value());
    cf.whenComplete((result, error) -> {
        if (error != null) {
            promise.fail(Causes.fromThrowable(error));
        } else {
            promise.succeed(result);
        }
    });

    return promise;
}

// Helper method for common pattern
public static <T> Promise<T> fromCompletableFuture(
        CompletableFuture<T> cf,
        Fn1<Cause, Throwable> errorMapper) {
    var promise = Promise.<T>promise();
    cf.whenComplete((result, error) -> {
        if (error != null) {
            promise.fail(errorMapper.apply(error));
        } else {
            promise.succeed(result);
        }
    });
    return promise;
}
```

Exceptions → Cause Mapping

In adapters - use Promise.lift:

```
public Promise<User> findUser(UserId id) {
    return Promise.lift(
        DatabaseError::new,  // Exception → Cause
        () -> jdbcTemplate.queryForObject(SQL, mapper, id.value())
    );
}
```

Custom exception mapping:

```
public Promise<Payment> processPayment(PaymentRequest request) {
    return Promise.lift(
```

```
        this::mapPaymentException,  // Custom mapper
        () -> paymentGateway.charge(request)
    );
}

private Cause mapPaymentException(Throwable t) {
    return switch (t) {
        case InsufficientFundsException e -> new PaymentError.InsufficientFunds(e.getMess
        case CardDeclinedException e -> new PaymentError.CardDeclined(e.getMessage());
        case NetworkException e -> new PaymentError.ServiceUnavailable(e.getMessage());
        default -> Causes.fromThrowable(t);
    };
}
```

Transitional Adapter Layer

During migration, create adapter methods that bridge old and new code:

```
// Legacy service (throws exceptions)
public class LegacyUserService {
    public User findUser(String id) throws UserNotFoundException {
        // ... legacy implementation
    }
}

// JBCT adapter wrapping legacy service
public class UserServiceAdapter implements FindUser {
    private final LegacyUserService legacy;

    @Override
    public Promise<User> apply(UserId id) {
        return Promise.lift(
            this::mapLegacyError,
            () -> legacy.findUser(id.value())
        );
    }

    private Cause mapLegacyError(Throwable t) {
        return switch (t) {
            case UserNotFoundException e -> UserError.NotFound.INSTANCE;
            default -> Causes.fromThrowable(t);
        };
    }
}
```

Gradual replacement: 1. Create adapter wrapping legacy service 2. Use adapter in new JBCT code 3. Eventually replace legacy service with JBCT implementation 4.

Remove adapter when migration complete

Team Adoption Strategies

Strategy 1: Champion-Led

One developer becomes JBCT expert, writes initial examples, and reviews others' code.

Pros: Fast start, consistent patterns **Cons:** Bottleneck, bus factor

Strategy 2: New Feature First

All new features use JBCT. Existing code migrated only when touched.

Pros: No big-bang rewrite, natural adoption **Cons:** Mixed codebase for longer, context switching

Strategy 3: Vertical Slice

Pick one use case, migrate completely from controller to database.

Pros: Complete example to learn from, proves value end-to-end **Cons:** Initial investment, may find integration issues late

Recommendation

Combine strategies: 1. Champion writes first vertical slice (Strategy 3) 2. Team reviews and learns from example 3. All new features use JBCT (Strategy 2) 4. Gradual migration of existing code when touched

---

Common Resistance and Responses

"It's too different from what we know"

Response: The patterns are simpler than exception handling. Compare:

```
// Exception-based (hidden control flow)
try {
    User user = findUser(id);
    if (user == null) throw new UserNotFoundException(id);
    Order order = createOrder(user, items);
    if (!paymentService.charge(order)) {
        throw new PaymentException("Failed");
    }
    return order;
```

```
} catch (UserNotFoundException e) {
    return handleUserNotFound(e);
} catch (PaymentException e) {
    return handlePaymentFailed(e);
} catch (Exception e) {
    return handleUnknown(e);
}

// JBCT (explicit flow)
return findUser(id)
    .flatMap(user -> createOrder(user, items))
    .flatMap(order -> chargePayment(order))
    .fold(this::handleError, identity());
```

"We don't have time to rewrite everything"

Response: You don't have to. Start with value objects only (Phase 1). That's one week for immediate benefits. Each phase is incremental.

"Our team doesn't know functional programming"

Response: JBCT isn't about FP. It's about making code predictable. The functional patterns are just the mechanism. Focus on: - Every method declares its failure modes in the return type - Invalid data cannot be constructed - No hidden control flow

"What about debugging?"

Response: Debugging is easier, not harder: - Stack traces show the actual call chain (no exception jumps) - Error causes are typed and carry context - IDE breakpoints work normally in lambdas - Logging can be added at any point in the chain

"Performance overhead?"

Response: Negligible for business logic. Pragmatica Lite types are lightweight wrappers. The overhead is comparable to Optional. If you're writing code where object allocation matters (tight loops, real-time systems), JBCT isn't the right fit - but neither is typical Java web development.

Migration Checklist

Phase 1 Complete When:

- ☐ Pragmatica Lite added to project
- ☐ At least 5 value objects created
- ☐ One service method uses value objects

- ☐ Team has reviewed value object patterns

Phase 2 Complete When:

- ☐ Team agreement: new code uses Result/Promise
- ☐ At least one new feature built with JBCT
- ☐ Controller bridges exception/Result boundary
- ☐ Error types defined for new feature

Phase 3 Complete When:

- ☐ At least one use case extracted
- ☐ Use case has step interfaces
- ☐ Service is thin adapter
- ☐ Integration tests pass

Phase 4 Complete When:

- ☐ All new adapters use Promise.lift
- ☐ Database access wrapped
- ☐ External HTTP calls wrapped
- ☐ Error mapping consistent across adapters

Exercises

1. **Audit your codebase:** Find three methods with scattered validation. Which value objects would consolidate them?
2. **Plan Phase 1:** List the top 5 value objects your codebase needs. Prioritize by how many places validate the same data.
3. **Identify extraction candidates:** Find a service method with more than 50 lines. Can you identify the Sequencer pattern in it?
4. **Adapter inventory:** List all external dependencies (database, HTTP, message queue). Which ones throw checked exceptions?

Summary

Migration to JBCT is incremental:

1. **Phase 1:** Value objects only - immediate validation benefits
2. **Phase 2:** Result in new code - stop adding exceptions
3. **Phase 3:** Extract use cases - separate concerns
4. **Phase 4:** Adapter isolation - complete the boundary

Key principles: - Never big-bang rewrite - Each phase delivers value independently - Mixed codebase is acceptable during transition - Team buy-in through working examples, not mandates

Chapter 18: Comparison with Other Approaches

JBCT doesn't exist in isolation. This chapter compares it to other architectural approaches you may encounter, helping you understand where JBCT fits and what it borrows from or rejects in other methodologies.

Traditional Layered Architecture

The most common Java backend architecture.

Structure

```
Controller Layer     → Receives HTTP requests, validates DTOs
    ↓
Service Layer        → Business logic, transactions, orchestration
    ↓
Repository Layer     → Database access, queries
    ↓
Entity Layer         → JPA entities, data structures
```

Example

```java
@RestController
public class UserController {
    @Autowired private UserService userService;

    @PostMapping("/users")
    public ResponseEntity<UserDto> createUser(@Valid @RequestBody CreateUserDto dto) {
        User user = userService.createUser(dto);
        return ResponseEntity.ok(toDto(user));
    }
}

@Service
public class UserService {
    @Autowired private UserRepository userRepository;
```

```
    @Autowired private PasswordEncoder passwordEncoder;

    @Transactional
    public User createUser(CreateUserDto dto) {
        if (userRepository.existsByEmail(dto.getEmail())) {
            throw new EmailExistsException();
        }
        User user = new User();
        user.setEmail(dto.getEmail());
        user.setPassword(passwordEncoder.encode(dto.getPassword()));
        return userRepository.save(user);
    }
}

@Repository
public interface UserRepository extends JpaRepository<User, Long> {
    boolean existsByEmail(String email);
}
```

What JBCT Changes

Aspect	Layered	JBCT
Error handling	Exceptions	Result/Promise with Cause
Validation	@Valid + manual checks	Parse-don't-validate
Business logic location	Service layer	Use case interface
Data flow	Mutable entities	Immutable value objects
Dependencies	Field injection	Constructor injection via factory

What JBCT Keeps

- Separation of concerns (different responsibility per layer)
- Controllers at the boundary
- Repository pattern for data access

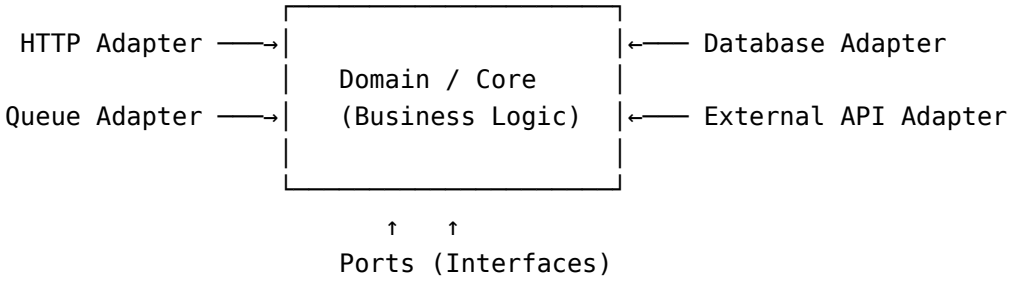
Verdict

JBCT refines layered architecture rather than replacing it. The layers still exist, but with explicit error handling and immutable data flow.

Hexagonal Architecture (Ports and Adapters)

Popularized by Alistair Cockburn. Core idea: business logic at the center, adapters at the edges.

Structure



Example

```java
// Port (interface defined by domain)
public interface UserRepository {
    Optional<User> findByEmail(Email email);
    void save(User user);
}

// Domain service (pure business logic)
public class UserRegistrationService {
    private final UserRepository userRepository;
    private final PasswordHasher passwordHasher;

    public User register(Email email, Password password) {
        if (userRepository.findByEmail(email).isPresent()) {
            throw new EmailAlreadyExistsException(email);
        }
        HashedPassword hashed = passwordHasher.hash(password);
        User user = new User(email, hashed);
        userRepository.save(user);
        return user;
    }
}

// Adapter (implements port)
public class JpaUserRepository implements UserRepository {
    private final JpaUserEntityRepository jpaRepo;

    @Override
    public Optional<User> findByEmail(Email email) {
        return jpaRepo.findByEmail(email.value())
            .map(this::toDomain);
    }
}
```

What JBCT Borrows

- **Ports concept** → Step interfaces
- **Adapters concept** → Adapter leaves
- **Domain at center** → Use cases with pure business logic
- **Dependency inversion** → Steps injected into use case factory

What JBCT Adds

- **Explicit error handling** - Hexagonal doesn't prescribe how to handle errors
- **Functional composition** - Hexagonal uses imperative style
- **Typed failures** - Cause types instead of exceptions
- **Structural patterns** - Leaf, Sequencer, Fork-Join provide composition vocabulary

Key Difference

Hexagonal focuses on **where** code lives (inside vs outside the hexagon). JBCT focuses on **how** code composes (patterns, error handling, data flow).

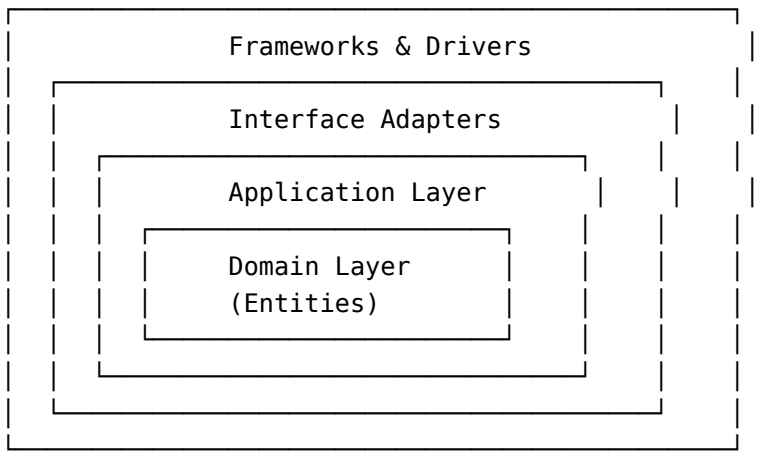
Verdict

JBCT and Hexagonal are complementary. Use Hexagonal for high-level architecture, JBCT for implementation patterns within that architecture.

Clean Architecture

Uncle Bob's architecture with explicit dependency rules.

Structure



Dependencies point inward. Inner layers don't know about outer layers.

Example

```java
// Entity (innermost)
public class User {
    private UserId id;
    private Email email;
    private HashedPassword password;

    public boolean canLogin(Password attempt, PasswordHasher hasher) {
        return hasher.verify(attempt, this.password);
    }
}

// Use Case (application layer)
public class RegisterUserUseCase {
    private final UserGateway userGateway;
    private final PasswordHasher passwordHasher;

    public RegisterUserResponse execute(RegisterUserRequest request) {
        Email email = new Email(request.email);
        if (userGateway.existsByEmail(email)) {
            return RegisterUserResponse.failure("Email exists");
        }
        // ... rest of use case
    }
}

// Gateway interface (application layer, implemented by adapter)
public interface UserGateway {
    boolean existsByEmail(Email email);
    void save(User user);
}
```

What JBCT Borrows

- **Use case as first-class concept** → UseCase interface
- **Dependency rule** → Use cases don't depend on adapters
- **Request/Response models** → Request/ValidRequest/Response records

What JBCT Changes

Aspect	Clean Architecture	JBCT
Use case return	Response objects	Result/Promise
Error handling	Response codes or exceptions	Typed Cause
Entity behavior	Rich domain model	Value objects + pure functions

Aspect	Clean Architecture	JBCT
Composition	Imperative orchestration	Monadic chains

Key Difference

Clean Architecture prescribes **dependency direction** but not **composition style**. JBCT prescribes both.

Verdict

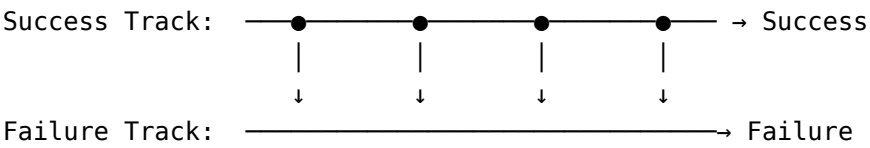
JBCT can be implemented within Clean Architecture. The layers map naturally: - Entities → Value objects - Use Cases → Use case interfaces - Interface Adapters → Adapter leaves - Frameworks → Spring configuration

---

Railway-Oriented Programming

Scott Wlaschin's approach from F#, using "two-track" types for error handling.

Concept



Each function either stays on success track or switches to failure track.

Example (F# style in Java)

```
public Result<User> registerUser(String email, String password) {
    return validateEmail(email)           // Success or Failure
        .flatMap(this::validatePassword)  // Continue or stay failed
        .flatMap(this::checkUniqueness)   // Continue or stay failed
        .flatMap(this::createUser);       // Continue or stay failed
}
```

What JBCT Borrows

- **Two-track model** → `Result<T>` is exactly this
- **flatMap for composition** → Same chaining style
- **Errors as values** → Cause instead of exceptions

What JBCT Adds

- **Four tracks, not two** - T, Option, Result, Promise for different semantics
- **Structural patterns** - Named patterns (Sequencer, Fork-Join) beyond just chaining
- **Aggregation** - Result.all() for parallel validation
- **Async integration** - Promise extends the model to async operations

Key Difference

ROP is a technique for error handling. JBCT is a complete methodology including project structure, testing, and team practices.

Verdict

JBCT incorporates ROP as its error handling model, then builds a full methodology around it.

---

vavr (formerly Javaslang)

Functional programming library for Java.

What vavr Provides

```
import io.vavr.control.Try;
import io.vavr.control.Either;
import io.vavr.control.Option;

// Try - captures exceptions
Try<Integer> result = Try.of(() -> Integer.parseInt(input));

// Either - success or failure with typed error
Either<Error, User> user = findUser(id);

// Option - presence or absence
Option<User> maybeUser = Option.of(nullableUser);

// Pattern matching
String message = Match(result).of(
    Case($Success($()), "Parsed"),
    Case($Failure($()), "Failed")
);
```

Comparison

Feature	vavr	Pragmatica Lite
Option type	`Option<T>`	`Option<T>`
Error type	`Either<L, R>` or `Try<T>`	`Result<T>` with `Cause`
Async type	None (use CompletableFuture)	`Promise<T>`
Collections	Immutable collections	Uses Java collections
Pattern matching	Built-in DSL	Standard switch expressions
Tuples	Tuple1-8	Use records

Key Differences

1. **vavr is a library, JBCT is a methodology** - vavr provides types, JBCT provides patterns, structure, and practices.
2. **Error typing** - vavr's Either has generic left type. Pragmatica Lite's Result always uses Cause, providing consistent error handling.
3. **Async story** - vavr doesn't provide async primitives. JBCT's Promise integrates error handling with async operations.
4. **Simplicity** - vavr includes many FP features (persistent collections, pattern matching DSL, streams). Pragmatica Lite focuses on the minimum needed for JBCT.

When to Use vavr

- You want immutable collections
- You're building a library with FP patterns
- You need persistent data structures
- You want pattern matching DSL

When to Use Pragmatica Lite

- You're building backend services
- You want a complete methodology (not just types)
- You need integrated async support
- You prefer simplicity over features

Verdict

vavr is a more comprehensive FP library. Pragmatica Lite is purpose-built for JBCT. You could use vavr to implement JBCT patterns, but you'd need to add your own Promise type and establish your own structural patterns.

Arrow-kt (Kotlin)

Functional programming library for Kotlin.

What Arrow Provides

```kotlin
// Either for errors
fun divide(a: Int, b: Int): Either<DivisionError, Int> =
    if (b == 0) DivisionError.DivideByZero.left()
    else (a / b).right()

// Validated for accumulating errors
fun validateUser(name: String, age: Int): ValidatedNel<ValidationError, User> =
    ValidatedNel.applicative<ValidationError>().mapN(
        validateName(name),
        validateAge(age)
    ) { (n, a) -> User(n, a) }

// Effect for async
suspend fun fetchUser(id: UserId): Either<Error, User> =
    either { userRepository.findById(id).bind() }
```

Why Mention It?

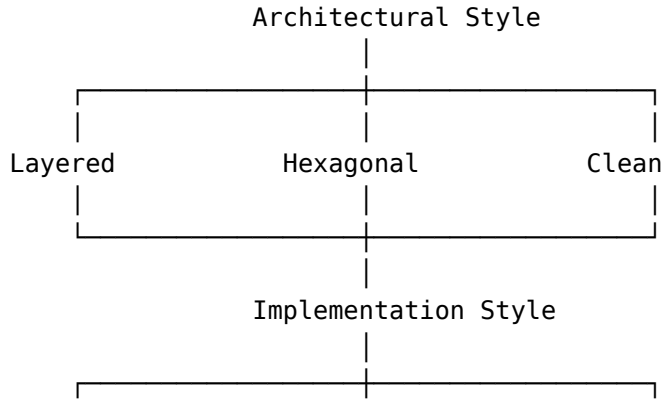
Arrow-kt demonstrates that these patterns work well in a JVM language. Key insights:

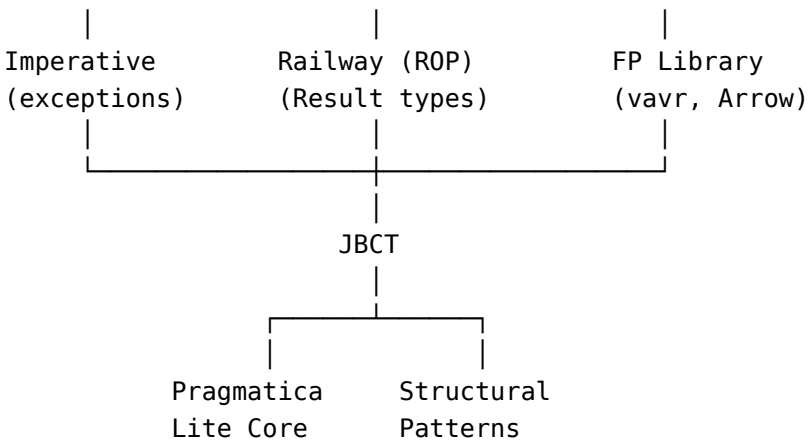
- **Kotlin's coroutines + Either** = Similar to `Promise<T>`
- **Validated type** = Similar to Result.all() accumulation
- **Typed errors** = Same as JBCT's Cause

Verdict

If you're on Kotlin, Arrow-kt provides similar capabilities to JBCT. The concepts transfer, but the syntax differs due to language features (coroutines, extension functions, sealed classes).

Summary: Where JBCT Fits





JBCT: - Works within any architectural style (Layered, Hexagonal, Clean) - Uses railway-oriented error handling - Is simpler than full FP libraries - Provides structural patterns other approaches lack - Includes methodology beyond just types

---

Exercises

1. **Map your architecture:** Does your current project use Layered, Hexagonal, or Clean Architecture? Where would JBCT patterns fit?
2. **Compare error handling:** Take one exception-based method in your codebase. Rewrite it using ROP style with Result. What errors were implicit?
3. **Evaluate vavr:** If you use vavr, identify which features you actually use. Could you replace it with Pragmatica Lite?
4. **Cross-language patterns:** If you have Kotlin services, compare how Arrow-kt's Either compares to JBCT's Result. Are the patterns similar?

Summary

JBCT is not revolutionary - it combines proven ideas:

Idea	Source
Ports and Adapters	Hexagonal Architecture
Use Cases as first-class	Clean Architecture
Errors as values	Railway-Oriented Programming
Functional types	vavr, Arrow-kt, Haskell
Parse don't validate	Type-driven design

What JBCT adds: - **Unified methodology** - Architecture + implementation + testing - **Structural patterns** - Named, composable patterns - **Team practices** - Migration

path, code review guidelines - **AI optimization** - Predictable code for AI collaboration

The goal isn’t originality - it’s unification.

Chapter 19: Troubleshooting & FAQ

This chapter consolidates common issues, debugging techniques, and frequently asked questions about JBCT.

Debugging Monadic Chains

Problem: "I can't see what's happening inside flatMap"

Symptom: You have a chain like this and can't figure out where it fails:

```
return validateRequest(request)
    .flatMap(this::checkPermissions)
    .flatMap(this::processData)
    .flatMap(this::saveResult);
```

Solution 1: Add intermediate logging

```
return validateRequest(request)
    .onSuccess(r -> log.debug("Validated: {}", r))
    .onFailure(c -> log.debug("Validation failed: {}", c.message()))
    .flatMap(this::checkPermissions)
    .onSuccess(r -> log.debug("Permissions OK: {}", r))
    .onFailure(c -> log.debug("Permission denied: {}", c.message()))
    .flatMap(this::processData)
    // ...
```

Solution 2: Use IDE breakpoints

Set breakpoints inside the lambda:

```
.flatMap(data -> {
    return processData(data);  // Breakpoint on this line
})
```

Solution 3: Extract to named methods

```
return validateRequest(request)
    .flatMap(this::checkPermissionsStep)  // Breakpoint in method
    .flatMap(this::processDataStep)
    .flatMap(this::saveResultStep);
```

```
private Result<Permissions> checkPermissionsStep(ValidRequest request) {
    // Breakpoint here, full visibility
    return checkPermissions(request);
}
```

Problem: "My Promise never resolves"

Symptom: Test hangs on `.await()` or production code times out.

Checklist:

1. **Is the Promise created but never resolved?**

```
Promise<User> promise = Promise.promise();  // Unresolved!
// Missing: promise.succeed(user) or promise.fail(cause)
```

2. **Is there a deadlock in async code?**

```
// WRONG: Blocking inside async chain
.flatMap(data -> {
    return otherPromise.await().async();  // Blocks the thread!
})

// CORRECT: Chain without blocking
.flatMap(data -> otherPromise)
```

3. **Is Promise.lift wrapping blocking code?**

```
// This runs async - make sure the thread pool isn't exhausted
Promise.lift(() -> {
    Thread.sleep(10000);  // Blocks a thread
    return result;
});
```

Debug technique: Add timeout to isolate:

```
promise
    .timeout(TimeSpan.timeSpan(5).seconds())
    .await()
    .onFailure(cause -> log.error("Timed out or failed: {}", cause));
```

Problem: "I'm getting CompositeCause but don't know which validations failed"

Symptom: Result.all() fails but you only see "Multiple failures".

Solution: Inspect the CompositeCause:

```
result.onFailure(cause -> {
    if (cause instanceof CompositeCause composite) {
        composite.causes().forEach(c ->
```

```
            log.error("Validation failure: {}", c.message())
        );
    } else {
        log.error("Single failure: {}", cause.message());
    }
});
```

Better solution: Create specific error messages in value objects:

```
public record Email(String value) {
    private static final Fn1<Cause, String> INVALID =
        Causes.forOneValue("Invalid email format: %s");

    public static Result<Email> email(String raw) {
        return Verify.ensure(raw, Verify.Is::notBlank)
            .flatMap(Verify.ensureFn(INVALID, Verify.Is::matches, PATTERN))
            .map(Email::new);
    }
}
```

Now CompositeCause contains "Invalid email format: xyz" instead of generic message.

Common Mistakes and Fixes

Mistake: Nested Result/Promise

```
// WRONG: Returns Result<Result<User>>
public Result<Result<User>> findUser(String id) {
    return UserId.userId(id)
        .map(this::lookupUser);  // lookupUser returns Result<User>
}
```

Fix: Use flatMap instead of map:

```
// CORRECT: Returns Result<User>
public Result<User> findUser(String id) {
    return UserId.userId(id)
        .flatMap(this::lookupUser);
}
```

Mistake: Ignoring the Result

```
// WRONG: Result is created but never used
public void processOrder(Order order) {
    validateOrder(order);  // Returns Result<ValidOrder> but ignored!
```

```
    saveOrder(order);
}
```

Fix: Always handle the Result:

```
// CORRECT: Result flows through
public Result<OrderId> processOrder(Order order) {
    return validateOrder(order)
        .flatMap(this::saveOrder);
}
```

Mistake: Throwing inside monadic chain

```
// WRONG: Exception breaks the chain
.flatMap(data -> {
    if (data.isEmpty()) {
        throw new IllegalStateException("Empty data");  // Escapes!
    }
    return process(data);
})
```

Fix: Return failure instead:

```
// CORRECT: Error stays in the chain
.flatMap(data -> {
    if (data.isEmpty()) {
        return EMPTY_DATA_ERROR.result();
    }
    return process(data);
})

// BETTER: Use filter
.filter(EMPTY_DATA_ERROR, data -> !data.isEmpty())
.flatMap(this::process)
```

Mistake: Multi-statement lambda

```
// WRONG: Violates single level of abstraction
.map(user -> {
    log.info("Processing user: {}", user.id());
    String normalized = user.email().toLowerCase();
    cache.put(user.id(), user);
    return new ProcessedUser(user.id(), normalized);
})
```

Fix: Extract to named method:

```
// CORRECT: Simple lambda, logic in method
.map(this::processUser)

private ProcessedUser processUser(User user) {
    log.info("Processing user: {}", user.id());
    String normalized = user.email().toLowerCase();
    cache.put(user.id(), user);
    return new ProcessedUser(user.id(), normalized);
}
```

Mistake: Using Optional instead of Option

```
// WRONG: Mixing Java Optional with Pragmatica types
public Result<User> findUser(UserId id) {
    Optional<User> optional = repository.findById(id);
    if (optional.isPresent()) {
        return Result.success(optional.get());
    }
    return USER_NOT_FOUND.result();
}
```

Fix: Convert at the boundary:

```
// CORRECT: Wrap immediately
public Result<User> findUser(UserId id) {
    return Option.option(repository.findById(id).orElse(null))
        .toResult(USER_NOT_FOUND);
}

// OR: Use Option.from()
public Result<User> findUser(UserId id) {
    return Option.from(repository.findById(id))
        .toResult(USER_NOT_FOUND);
}
```

Mistake: Bypassing factory method

```
// WRONG: Direct constructor call
Email email = new Email("user@example.com");  // Skips validation!

// CORRECT: Use factory method
Result<Email> email = Email.email("user@example.com");
```

IDE Setup

IntelliJ IDEA

Recommended settings:

1. **Enable parameter hints for lambdas:**
   - Settings → Editor → Inlay Hints → Java → Parameter hints
   - Enable "Show hints for lambdas"
2. **Fold anonymous classes:**
   - Settings → Editor → General → Code Folding
   - Enable "Lambda expressions"
3. **Live templates for JBCT patterns:**

```
// Template: jbctvo (Value Object)
public record $NAME$($TYPE$ value) {
    private static final Cause INVALID = Causes.cause("Invalid $NAME$");

    public static Result<$NAME$> $FACTORY$(String raw) {
        return Verify.ensure(raw, Verify.Is::notBlank)
            .map($NAME$::new);
    }
}

// Template: jbctuc (Use Case)
public interface $NAME$ {
    Promise<Response> execute(Request request);

    record Request($PARAMS$) {}
    record Response($RESULT$) {}

    static $NAME$ $FACTORY$($DEPS$) {
        return request -> ValidRequest.validRequest(request)
            .async()
            .flatMap($STEPS$);
    }
}
```

VS Code

Recommended extensions: - Extension Pack for Java - Java Test Runner

settings.json:

```
{
    "java.completion.chain.enabled": true,
    "java.completion.filteredTypes": [
        "java.util.Optional"  // Suggest Option instead
    ]
```

```
}
```

Testing Troubleshooting

Problem: "Test passes but shouldn't"

Symptom: You expect failure but test passes silently.

```
@Test
void shouldFail() {
    Email.email("invalid");  // Returns Result, but test doesn't check it!
}
```

Fix: Always assert:

```
@Test
void email_fails_forInvalidInput() {
    Email.email("invalid")
        .onSuccess(Assertions::fail);  // Fail if unexpectedly succeeds
}
```

Problem: "Async test is flaky"

Symptom: Test sometimes passes, sometimes fails.

Checklist:

1. **Are you awaiting the Promise?**

```
// WRONG: Test completes before Promise resolves
@Test
void asyncTest() {
    useCase.execute(request);  // Returns Promise, not awaited!
}

// CORRECT: Await before asserting
@Test
void asyncTest() {
    useCase.execute(request)
        .await()
        .onFailure(Assertions::fail);
}
```

2. **Is there shared mutable state between tests?**

```
// WRONG: Shared state
private static List<String> logs = new ArrayList<>();
```

```
// CORRECT: Per-test state
private List<String> logs;

@BeforeEach
void setup() {
    logs = new ArrayList<>();
}
```

3. **Are Promise timeouts too short?**

```
// Increase timeout for CI environments
.await(TimeSpan.timeSpan(30).seconds())
```

Problem: "Can't create stub for step interface"

Symptom: Compilation error when creating lambda stub.

```
// ERROR: Target type must be a functional interface
var stub = req -> Promise.success(req);  // What type is this?
```

Fix: Declare the type explicitly:

```
// CORRECT: Type declaration
CheckEmail checkEmail = req -> Promise.success(req);
```

FAQ

General

Q: Is JBCT only for microservices?

A: No. JBCT works for any backend Java application - monoliths, microservices, serverless functions. The patterns are about code organization, not deployment topology.

Q: Does JBCT require Spring Boot?

A: No. JBCT is framework-agnostic. The examples use Spring Boot because it's common, but the patterns work with Micronaut, Quarkus, plain Java, or any other framework.

Q: Can I use JBCT with Kotlin?

A: Yes, but consider Arrow-kt instead. Kotlin's coroutines and extension functions provide more idiomatic alternatives. The concepts transfer directly.

Q: What about reactive streams (Project Reactor, RxJava)?

A: JBCT's Promise is simpler than reactive streams. Use reactive streams when you need backpressure or complex stream operations. Use Promise for simple async request/response patterns.

Error Handling

Q: When should I use exceptions instead of Result?

A: Use exceptions for: - Programming errors (IllegalArgumentException, NullPointerException) - Unrecoverable system failures - Framework requirements (Spring's @Transactional rollback)

Use Result for: - Business failures (validation, not found, conflict) - Expected error conditions - Anything the caller might want to handle

Q: How do I handle errors from third-party libraries that throw?

A: Wrap at the adapter boundary:

```
public Promise<User> findUser(UserId id) {
    return Promise.lift(
        DatabaseError::new,
        () -> jdbcTemplate.queryForObject(sql, mapper, id.value())
    );
}
```

Q: Should every method return Result?

A: No. Pure functions that cannot fail should return T directly:

```
// Returns T - pure computation, cannot fail
public Money calculateTotal(List<LineItem> items) {
    return items.stream()
        .map(LineItem::subtotal)
        .reduce(Money.ZERO, Money::add);
}

// Returns Result<T> - validation can fail
public Result<Money> parseAmount(String raw) {
    return Money.money(raw);
}
```

Performance

Q: Is there overhead from wrapping everything in Result/Promise?

A: Minimal. The types are thin wrappers. Overhead is comparable to Optional. For typical web applications, this is negligible compared to I/O latency.

Q: Are lambdas slower than methods?

A: No. The JVM inlines simple lambdas. There's no significant performance difference for typical use cases.

Q: Does creating many small objects (value objects) hurt GC?

A: Modern JVMs handle short-lived objects efficiently. Value objects are typically small and short-lived. If profiling shows GC pressure, the issue is usually elsewhere.

Testing

Q: Should I test value object validation separately from use cases?

A: Yes. Value objects get unit tests for each validation rule. Use cases get integration tests with stubbed steps.

Q: How many tests should a use case have?

A: At minimum: - 1 happy path test - 1 test per step failure - 1 test per validation rule in ValidRequest

Q: Should I use mocks or stubs?

A: Prefer stubs (simple lambda implementations). Mocks add complexity and verification overhead. JBCT's functional style makes stubbing easy.

Architecture

Q: Where do DTOs belong?

A: At adapter boundaries: - Request DTOs in controller (convert to domain types immediately) - Response DTOs from use case Response records - No DTOs in domain layer

Q: Can use cases call other use cases?

A: Generally no. If use case A needs functionality from use case B, extract that functionality to a shared step interface.

Q: How do I handle transactions?

A: At the adapter layer:

```
@Repository
public class JooqOrderRepository implements SaveOrder {
    @Transactional
    @Override
    public Promise<OrderId> apply(ValidOrder order) {
        return Promise.lift(
            DatabaseError::new,
            () -> saveInTransaction(order)
        );
    }
}
```

Error Message Reference

"Cannot infer functional interface type"

Cause: Lambda target type is ambiguous.

Fix: Add explicit type declaration:

```
CheckEmail checkEmail = req -> Promise.success(req);
```

"Incompatible types: `Result<X>` cannot be converted to `Result<Y>`"

Cause: Using map when flatMap is needed.

Fix: Check if the mapping function returns a Result. If so, use flatMap:

```
// WRONG: validate returns Result<ValidEmail>
.map(this::validate)  // Results in Result<Result<ValidEmail>>

// CORRECT
.flatMap(this::validate)  // Results in Result<ValidEmail>
```

"No instances of type variable(s) T exist"

Cause: Type inference failure, often with Cause.result() or Cause.promise().

Fix: Add type witness or assign to typed variable:

```
// Option 1: Type witness
return ERROR.<User>result();

// Option 2: Assign to variable
Result<User> failure = ERROR.result();
return failure;
```

"Promise.await() blocks forever"

Cause: Promise never resolved.

Fix: Ensure all code paths resolve the Promise:

```
Promise<User> promise = Promise.promise();

// Ensure both success and failure paths resolve
if (condition) {
    promise.succeed(user);
} else {
    promise.fail(cause);  // Don't forget this!
}
```

Summary

When debugging JBCT code:

1. **Add logging at chain points** - Use onSuccess/onFailure for visibility

2. **Extract to named methods** - Easier breakpoints and stack traces
3. **Check for common mistakes** - Nested results, ignored returns, thrown exceptions
4. **Use explicit types** - Helps IDE and compiler catch errors
5. **Test both success and failure** - Always assert on Result/Promise

The functional style may feel unfamiliar initially, but debugging becomes easier once you internalize the patterns. The explicit error handling means fewer surprises at runtime.

Appendix A: Pragmatica Lite Core API Reference

Library Information

Maven:

```xml
<dependency>
   <groupId>org.pragmatica-lite</groupId>
   <artifactId>core</artifactId>
   <version>0.8.4</version>
</dependency>
```

Gradle:

```
implementation 'org.pragmatica-lite:core:0.8.4'
```

Documentation: https://central.sonatype.com/artifact/org.pragmatica-lite/core

---

Type Conversions

Option Conversions

From	To	Method
`Option<T>`	`Result<T>`	`.toResult(Cause cause)` or `.await(Cause cause)`
`Option<T>`	`Result<T>`	`.toResult()` (uses CoreError.emptyOption)
`Option<T>`	`Promise<T>`	`.async(Cause cause)`
`Option<T>`	`Promise<T>`	`.async()` (uses CoreError.emptyOption)
`Option<T>`	`Optional<T>`	`.toOptional()`

Result Conversions

From	To	Method
`Result<T>`	`Option<T>`	`.option()` (loses error)
`Result<T>`	`Promise<T>`	`.async()`

Promise Conversions

From	To	Method
`Promise<T>`	`Promise<T>`	`.async()` (identity)
`Promise<T>`	`Result<T>`	`.await()` (blocks)
`Promise<T>`	`Result<T>`	`.await(TimeSpan timeout)`

Cause Conversions

From	To	Method
`Cause`	`Result<T>`	`.result()`
`Cause`	`Promise<T>`	`.promise()`

Creating Instances

Option

```
Option.some(value)          // Create present option
Option.present(value)       // Alias for some()
Option.none()               // Create empty option
Option.empty()              // Alias for none()
Option.option(nullable)     // null -> none, value -> some
Option.from(Optional<T>)    // Convert Java Optional
```

Result

```
Result.success(value)       // Create success
Result.ok(value)            // Alias for success()
Result.unitResult()         // Success with Unit value
Result.failure(cause)       // Create failure (prefer cause.result())
Result.err(cause)           // Alias for failure()
```

Promise

```
Promise.success(value)                    // Resolved success
Promise.ok(value)                         // Alias for success()
Promise.unitPromise()                     // Resolved with Unit
Promise.failure(cause)                    // Resolved failure (prefer cause.promise())
Promise.resolved(Result<T> result)        // Resolved from result
Promise.promise()                         // Unresolved promise
Promise.promise(Consumer<Promise<T>>)     // Unresolved, runs consumer async
Promise.promise(Supplier<Result<T>>)      // Async execution of supplier
```

Exception Handling (lift methods)

Result.lift

```
// Basic lift
Result.lift(ThrowingFn0<U> supplier)
Result.lift(Fn1<Cause, Throwable> mapper, ThrowingFn0<U> supplier)
Result.lift(ThrowingRunnable runnable)  // Returns Result<Unit>
Result.lift(Cause cause, ThrowingFn0<U> supplier)  // Fixed cause

// Direct invocation (lift + apply)
Result.lift1(ThrowingFn1<R, T1> fn, T1 value)
Result.lift2(ThrowingFn2<R, T1, T2> fn, T1 v1, T2 v2)
Result.lift3(ThrowingFn3<R, T1, T2, T3> fn, T1 v1, T2 v2, T3 v3)

// Function factories (returns wrapped function)
Result.liftFn1(ThrowingFn1<R, T1> fn)  // Returns Fn1<Result<R>, T1>
Result.liftFn2(ThrowingFn2<R, T1, T2> fn)
Result.liftFn3(ThrowingFn3<R, T1, T2, T3> fn)
```

Promise.lift

```
// Basic lift (async execution)
Promise.lift(ThrowingFn0<U> supplier)
Promise.lift(Fn1<Cause, Throwable> mapper, ThrowingFn0<U> supplier)
Promise.lift(ThrowingRunnable runnable)  // Returns Promise<Unit>

// Direct invocation (async)
Promise.lift1(ThrowingFn1<R, T1> fn, T1 value)
Promise.lift2(ThrowingFn2<R, T1, T2> fn, T1 v1, T2 v2)
Promise.lift3(ThrowingFn3<R, T1, T2, T3> fn, T1 v1, T2 v2, T3 v3)

// Function factories
Promise.liftFn1(ThrowingFn1<R, T1> fn)  // Returns Fn1<Promise<R>, T1>
```

Aggregation (all/any/allOf)

Result.all (accumulates failures)

```
Result.all(Result<T1>)                    // -> Mapper1<T1>
Result.all(Result<T1>, Result<T2>)        // -> Mapper2<T1, T2>
// ... up to Mapper9

Result.allOf(Result<T>...)                // -> Result<List<T>>
Result.allOf(List<Result<T>>)             // -> Result<List<T>>
```

Promise.all (fail-fast)

```
Promise.all(Promise<T1>)                  // -> Mapper1<T1>
Promise.all(Promise<T1>, Promise<T2>)     // -> Mapper2<T1, T2>
// ... up to Mapper9

Promise.allOf(Collection<Promise<T>>)     // -> Promise<List<Result<T>>>
```

any Methods

```
Option.any(Option<T>...)   // First present option
Result.any(Result<T>...)   // First success result
Promise.any(Promise<T>...) // First success, cancels others
```

Common Methods

map/flatMap

```
// All types
.map(Fn1<U, T> mapper)           // Transform value
.map(Supplier<U> supplier)        // Replace value
.flatMap(Fn1<M<U>, T> mapper)     // Chain monadic operations
.flatMap(Supplier<M<U>> supplier) // Replace with monadic value

// Result/Promise only
.flatMap2(Fn2<M<U>, T, I> mapper, I param)  // flatMap with extra param
.mapToUnit()                                 // Transform to Unit
```

filter (Result and Promise)

```
.filter(Cause cause, Predicate<T> predicate)
.filter(Fn1<Cause, T> causeMapper, Predicate<T> predicate)
```

Callback Methods

```
// Option
.onPresent(Consumer<T>)
.onEmpty(Runnable)
.apply(Consumer<T>, Runnable)  // Bifurcation

// Result
.onSuccess(Consumer<T>)
.onFailure(Consumer<Cause>)
.onResult(Consumer<Result<T>>)
.apply(Consumer<T>, Consumer<Cause>)  // Bifurcation

// Promise (same as Result, plus async variants)
.onSuccessAsync(Consumer<T>)
.onFailureAsync(Consumer<Cause>)
.withSuccess(Consumer<T>)  // Returns self for chaining
```

fold

```
// Option
.fold(Supplier<R> empty, Fn1<R, T> present)

// Result/Promise
.fold(Fn1<R, Cause> failure, Fn1<R, T> success)
```

Recovery

```
.or(T replacement)                    // Fallback value
.or(Supplier<T> supplier)             // Lazy fallback
.orElse(M<T> replacement)             // Fallback monadic value
.recover(Fn1<T, Cause> mapper)        // Result/Promise: recover from failure
```

Verify.Is Predicates

```
// Null check
Verify.Is::notNull
```

```
// String checks
Verify.Is::empty
Verify.Is::notEmpty
Verify.Is::blank
Verify.Is::notBlank
Verify.Is::lenBetween        // lenBetween(8, 128)
Verify.Is::contains
Verify.Is::notContains
Verify.Is::matches           // matches(Pattern)

// Numeric checks
Verify.Is::positive
Verify.Is::negative
Verify.Is::nonNegative
Verify.Is::nonPositive
Verify.Is::greaterThan
Verify.Is::lessThan
Verify.Is::between           // between(0, 150)
Verify.Is::equalTo
Verify.Is::notEqualTo
```

Usage:

```
Verify.ensure(password, Verify.Is::lenBetween, 8, 128)
Verify.ensure(age, Verify.Is::between, 0, 150)
```

Parse Subpackage

Number Parsing

```
Number.parseInt(String)           // -> Result<Integer>
Number.parseLong(String)          // -> Result<Long>
Number.parseDouble(String)        // -> Result<Double>
Number.parseBigDecimal(String)    // -> Result<BigDecimal>
```

DateTime Parsing

```
DateTime.parseLocalDate(String)       // -> Result<LocalDate>
DateTime.parseLocalTime(String)       // -> Result<LocalTime>
DateTime.parseLocalDateTime(String)   // -> Result<LocalDateTime>
DateTime.parseInstant(String)         // -> Result<Instant>
```

Network Parsing

```
Network.parseUUID(String)        // -> Result<UUID>
Network.parseURL(String)         // -> Result<URL>
Network.parseURI(String)         // -> Result<URI>
```

Causes Utilities

```
Causes.cause(String message)                          // Simple cause
Causes.cause(String message, Option<Cause> source)    // Cause with source
Causes.fromThrowable(Throwable)                       // Convert exception
Causes.forOneValue(String template)                   // Fn1<Cause, T> factory
Causes.forTwoValues(String template)                  // Fn2<Cause, T1, T2>
Causes.forThreeValues(String template)                // Fn3<Cause, T1, T2, T3>
Causes.composite(Result<?>...)                        // Composite from results
```

Template syntax (uses String.format):

```
Causes.forOneValue("Invalid email: %s")           // CORRECT
Causes.forTwoValues("Range error: %s to %s")      // CORRECT
```

Promise-Specific Operations

```
// Resolution
.resolve(Result<T>)     // Resolve unresolved promise
.succeed(T value)       // Resolve with success
.fail(Cause cause)      // Resolve with failure
.cancel()               // Cancel execution

// Query
.isResolved()           // Check if resolved

// Timeout
.timeout(TimeSpan duration)  // Add timeout

// Result manipulation
.mapResult(Fn1<Result<U>, Result<T>>)  // Transform result
.trace(Fn1<Cause, Cause>)              // Transform error (alias: mapError)
```

Example Usage Patterns

Adapter with exception handling

```
public Promise<User> findUser(UserId id) {
    return Promise.lift(
        CoreError::database,
        () -> jdbcTemplate.queryForObject(
            "SELECT * FROM users WHERE id = ?",
            new Object[]{id.value()},
            this::mapUser)
    );
}
```

Lifting sync Result to async Promise

```
public Promise<Response> execute(Request request) {
    return ValidRequest.validRequest(request)  // Result<ValidRequest>
                       .async()                // Promise<ValidRequest>
                       .flatMap(step1::apply)
                       .flatMap(step2::apply);
}
```

Testing with functional assertions

```
@Test
void validation_fails_forInvalidInput() {
    ValidRequest.validRequest(invalidRequest)
                .onSuccess(Assertions::fail);
}

@Test
void validation_succeeds_forValidInput() {
    ValidRequest.validRequest(validRequest)
                .onFailure(Assertions::fail)
                .onSuccess(valid -> {
                    assertEquals("expected", valid.field());
                });
}
```

Appendix B: Exercises and Solutions

This appendix contains exercises for each part of the book, organized by difficulty level. Each exercise includes a solution with explanation.

Difficulty Levels: - [Beginner] Beginner - Direct application of concepts - [Intermediate] Intermediate - Combining multiple concepts - [Advanced] Advanced - Design decisions and trade-offs

---

Part I: Foundations (Chapters 1-3)

Exercise 1.1 [Beginner] - Return Type Selection

For each scenario, identify the correct return type (T, Option, Result, Promise):

1. A method that calculates the sum of two integers
2. A method that finds a user's middle name (not all users have one)
3. A method that parses a date string
4. A method that fetches a user profile from a remote API
5. A method that validates an email format
6. A method that looks up a value in an in-memory cache

Solution

1. **T** - Pure calculation, cannot fail
2. **Option** - Absence is normal, not an error
3. **Result** - Parsing can fail with a reason
4. **Promise** - Remote I/O, can fail asynchronously
5. **Result** - Validation can fail with a reason
6. **Option** - Cache miss is normal, not an error

Exercise 1.2 [Beginner] - Option vs Result

Explain why each of these uses the wrong type and provide the correct signature:

```
// A
public Optional<User> authenticate(String email, String password) {
    // Returns empty if credentials invalid
```

```
}

// B
public Result<Config> getOptionalConfig(String key) {
    // Returns failure if config key doesn't exist
}
```

Solution

A is wrong: Authentication failure is an error with a reason (wrong password, account locked, user not found). Should be:

```
public Result<User> authenticate(String email, String password)
```

B is wrong: "Optional config" implies absence is expected, not an error. Should be:

```
public Option<Config> getOptionalConfig(String key)
```

Exercise 1.3 [Intermediate] - Type Lifting

Complete the following code to properly lift between types:

```
public Promise<UserProfile> loadProfile(String userId) {
    // 1. Parse userId to UserId value object (Result<UserId>)
    // 2. Look up in cache (Option<UserProfile>)
    // 3. If not in cache, fetch from database (Promise<UserProfile>)

    Result<UserId> id = UserId.userId(userId);
    // TODO: Complete the chain
}
```

Solution

```
public Promise<UserProfile> loadProfile(String userId) {
    return UserId.userId(userId)
        .async()  // Result -> Promise
        .flatMap(id -> cache.get(id)
            .async(CACHE_MISS)  // Option -> Promise (with cause for empty)
            .recover(cause -> database.fetchProfile(id)));
}

// Alternative: if cache miss should fall through silently
public Promise<UserProfile> loadProfile(String userId) {
    return UserId.userId(userId)
        .async()
        .flatMap(id -> cache.get(id)
            .map(Promise::success)
            .or(() -> database.fetchProfile(id)));
}
```

Exercise 1.4 [Beginner] - Cause Creation

Create appropriate Cause definitions for a user registration system:

1. Email already exists
2. Password too weak (should include the specific weakness)
3. Username contains invalid characters (should show which characters)

Solution

```
public sealed interface RegistrationError extends Cause {

    record EmailExists(String email) implements RegistrationError {
        private static final Fn1<Cause, String> FACTORY =
            Causes.forOneValue("Email already registered: %s");

        @Override
        public String message() {
            return FACTORY.apply(email).message();
        }
    }

    record WeakPassword(String weakness) implements RegistrationError {
        private static final Fn1<Cause, String> FACTORY =
            Causes.forOneValue("Password too weak: %s");

        @Override
        public String message() {
            return FACTORY.apply(weakness).message();
        }
    }

    record InvalidUsername(String invalidChars) implements RegistrationError {
        private static final Fn1<Cause, String> FACTORY =
            Causes.forOneValue("Username contains invalid characters: %s");

        @Override
        public String message() {
            return FACTORY.apply(invalidChars).message();
        }
    }
}
```

Exercise 1.5 [Intermediate] - Pragmatica Lite Operations

What does each expression evaluate to?

```
// Given:
Result<Integer> success = Result.success(10);
Result<Integer> failure = Causes.cause("error").result();

// 1.
success.map(x -> x * 2).or(0)

// 2.
failure.map(x -> x * 2).or(0)

// 3.
success.filter(Causes.cause("not positive"), x -> x > 0)

// 4.
success.filter(Causes.cause("not negative"), x -> x < 0)

// 5.
Result.all(success, failure).map((a, b) -> a + b)
```

Solution

1. `20` - maps 10 to 20, or() returns value
2. `0` - failure skips map, or() returns fallback
3. `Result.success(10)` - predicate passes
4. `Result.failure(Cause("not negative"))` - predicate fails
5. `Result.failure(Cause("error"))` - all() accumulates failure from second

Part II: Core Principles (Chapters 4-6)

Exercise 2.1 [Beginner] - Value Object Creation

Create a `PhoneNumber` value object that: - Accepts strings in format “+1-555-123-4567” or “5551234567” - Normalizes to digits only - Must be 10-15 digits

Solution

```
public record PhoneNumber(String value) {
    private static final Pattern DIGITS_ONLY = Pattern.compile("\\d+");
    private static final Cause INVALID_FORMAT =
        Causes.cause("Phone number must contain only digits, dashes, and optional leading
    private static final Cause INVALID_LENGTH =
        Causes.cause("Phone number must be 10-15 digits");

    public static Result<PhoneNumber> phoneNumber(String raw) {
```

```
        return Verify.ensure(raw, Verify.Is::notBlank)
            .map(PhoneNumber::normalize)
            .flatMap(Verify.ensureFn(INVALID_FORMAT, PhoneNumber::isValidFormat))
            .flatMap(Verify.ensureFn(INVALID_LENGTH, Verify.Is::lenBetween, 10, 15))
            .map(PhoneNumber::new);
    }

    private static String normalize(String input) {
        return input.replaceAll("[^0-9]", "");
    }

    private static boolean isValidFormat(String normalized) {
        return DIGITS_ONLY.matcher(normalized).matches();
    }
}
```

Exercise 2.2 [Intermediate] - Aggregate Validation

Create a `DateRange` value object with `from` and `to` LocalDate fields where: - Both dates are required - `from` must be before or equal to `to` - Neither date can be in the past

Solution

```
public record DateRange(LocalDate from, LocalDate to) {
    private static final Cause FROM_AFTER_TO =
        Causes.cause("Start date must be before or equal to end date");
    private static final Cause FROM_IN_PAST =
        Causes.cause("Start date cannot be in the past");
    private static final Cause TO_IN_PAST =
        Causes.cause("End date cannot be in the past");

    public static Result<DateRange> dateRange(String fromRaw, String toRaw) {
        var today = LocalDate.now();

        return Result.all(parseAndValidate(fromRaw, today, FROM_IN_PAST),
                          parseAndValidate(toRaw, today, TO_IN_PAST))
                     .flatMap(DateRange::validateOrder);
    }

    private static Result<LocalDate> parseAndValidate(String raw, LocalDate today, Cause
        return DateTime.parseLocalDate(raw)
            .flatMap(Verify.ensureFn(pastError, date -> !date.isBefore(today)));
    }

    private static Result<DateRange> validateOrder(LocalDate from, LocalDate to) {
        if (from.isAfter(to)) {
```

```
            return FROM_AFTER_TO.result();
        }
        return Result.success(new DateRange(from, to));
    }
}
```

Exercise 2.3 [Intermediate] - Error Accumulation vs Short-Circuit

Explain the difference in behavior and output:

```
// Version A
Result<ValidForm> validateA(Form form) {
    return Email.email(form.email())
        .flatMap(email -> Password.password(form.password())
            .map(password -> new ValidForm(email, password)));
}

// Version B
Result<ValidForm> validateB(Form form) {
    return Result.all(Email.email(form.email()),
                      Password.password(form.password()))
                 .map(ValidForm::new);
}
```

Given input with both invalid email AND invalid password, what does each return?

Solution

Version A (short-circuit): Returns failure with ONLY the email error. flatMap stops at first failure.

Version B (accumulation): Returns failure with BOTH errors in a CompositeCause. Result.all() collects all failures.

When to use each: - Use flatMap chain when errors are dependent (later validations need earlier values) - Use Result.all() when validations are independent and you want to show all errors to user

---

Exercise 2.4 [Advanced] - Null Handling Strategy

Refactor this method to eliminate null handling:

```
public User processUser(UserDto dto) {
    if (dto == null) {
        throw new IllegalArgumentException("DTO cannot be null");
    }
```

```
    String email = dto.getEmail();
    if (email == null || email.isBlank()) {
        throw new ValidationException("Email required");
    }

    String name = dto.getName();  // Optional field
    String normalizedName = name != null ? name.trim() : null;

    Address address = null;
    if (dto.getAddress() != null) {
        address = processAddress(dto.getAddress());
    }

    return new User(email.toLowerCase(), normalizedName, address);
}
```

Solution

```
public Result<User> processUser(UserDto dto) {
    return Result.all(Email.email(dto.email()),
                      processOptionalName(dto.name()),
                      processOptionalAddress(dto.address()))
                 .map(User::new);
}

private Result<Option<Name>> processOptionalName(String raw) {
    return Option.option(raw)
        .map(String::trim)
        .filter(s -> !s.isBlank())
        .map(Name::new)
        .fold(
            () -> Result.success(Option.none()),
            name -> Result.success(Option.some(name))
        );
}

private Result<Option<Address>> processOptionalAddress(AddressDto dto) {
    return Option.option(dto)
        .map(this::validateAddress)
        .fold(
            () -> Result.success(Option.none()),
            result -> result.map(Option::some)
        );
}

// User record accepts Option types
public record User(Email email, Option<Name> name, Option<Address> address) {}
```

Exercise 2.5 [Beginner] - Recovery Patterns

Complete the recovery logic:

```
public Promise<Config> loadConfig() {
    return loadFromDatabase()
        // TODO: If database fails with ConnectionError, try file
        // TODO: If file fails with FileNotFound, use defaults
        // TODO: Any other error should propagate
        ;
}

private Promise<Config> loadFromDatabase() { /* ... */ }
private Promise<Config> loadFromFile() { /* ... */ }
private Config defaultConfig() { /* ... */ }
```

Solution

```
public Promise<Config> loadConfig() {
    return loadFromDatabase()
        .recover(this::recoverFromDatabaseError);
}

private Promise<Config> recoverFromDatabaseError(Cause cause) {
    return switch (cause) {
        case ConnectionError ignored -> loadFromFile()
            .recover(this::recoverFromFileError);
        default -> cause.promise();
    };
}

private Promise<Config> recoverFromFileError(Cause cause) {
    return switch (cause) {
        case FileNotFound ignored -> Promise.success(defaultConfig());
        default -> cause.promise();
    };
}
```

Part III: Patterns (Chapters 7-9)

Exercise 3.1 [Beginner] - Pattern Identification

Identify the JBCT pattern used in each code snippet:

```
// A
return validateOrder(request)
    .flatMap(this::checkInventory)
    .flatMap(this::processPayment)
    .flatMap(this::createShipment);

// B
return Promise.all(fetchUserProfile(userId),
                   fetchUserOrders(userId),
                   fetchUserPreferences(userId))
              .map(Dashboard::new);

// C
return switch (user.tier()) {
    case PREMIUM -> premiumProcessor.process(order);
    case STANDARD -> standardProcessor.process(order);
    case BASIC -> basicProcessor.process(order);
};

// D
return Promise.lift(DatabaseError::new,
                    () -> jdbcTemplate.query(sql, mapper));
```

Solution

- **A: Sequencer** - Linear chain of dependent steps
- **B: Fork-Join** - Parallel independent operations combined
- **C: Condition** - Routing based on discriminator
- **D: Leaf** - Adapter wrapping external I/O

Exercise 3.2 [Intermediate] - Implement Fork-Join

Implement a dashboard loader that fetches three pieces of data in parallel: - User profile (required) - Recent orders (required) - Recommendations (optional - use empty list on failure)

```
public interface LoadDashboard {
    Promise<Dashboard> load(UserId userId);

    record Dashboard(UserProfile profile, List<Order> orders, List<Product> recommendatio
}
```

Solution

```
public interface LoadDashboard {
    Promise<Dashboard> load(UserId userId);
```

```
    record Dashboard(UserProfile profile, List<Order> orders, List<Product> recommendatio

    interface FetchProfile {
        Promise<UserProfile> apply(UserId userId);
    }

    interface FetchOrders {
        Promise<List<Order>> apply(UserId userId);
    }

    interface FetchRecommendations {
        Promise<List<Product>> apply(UserId userId);
    }

    static LoadDashboard create(FetchProfile fetchProfile,
                                FetchOrders fetchOrders,
                                FetchRecommendations fetchRecommendations) {
        return userId -> Promise.all(fetchProfile.apply(userId),
                                     fetchOrders.apply(userId),
                                     fetchRecommendations.apply(userId)
                                         .recover(cause -> Promise.success(List.of())))
                                .map(Dashboard::new);
    }
}
```

Exercise 3.3 [Intermediate] - Implement Condition Pattern

Implement a notification sender that routes based on user preference:

```
public interface SendNotification {
    Promise<Unit> send(UserId userId, Message message);
}

public enum NotificationPreference {
    EMAIL, SMS, PUSH, NONE
}
```

Solution

```
public interface SendNotification {
    Promise<Unit> send(UserId userId, Message message);

    interface GetPreference {
        Promise<NotificationPreference> apply(UserId userId);
    }
```

```
    interface SendEmail {
        Promise<Unit> apply(UserId userId, Message message);
    }

    interface SendSms {
        Promise<Unit> apply(UserId userId, Message message);
    }

    interface SendPush {
        Promise<Unit> apply(UserId userId, Message message);
    }

    static SendNotification create(GetPreference getPreference,
                                   SendEmail sendEmail,
                                   SendSms sendSms,
                                   SendPush sendPush) {
        return (userId, message) -> getPreference.apply(userId)
            .flatMap(pref -> routeByPreference(pref, userId, message,
                                               sendEmail, sendSms, sendPush));
    }

    private static Promise<Unit> routeByPreference(NotificationPreference pref,
                                                   UserId userId,
                                                   Message message,
                                                   SendEmail sendEmail,
                                                   SendSms sendSms,
                                                   SendPush sendPush) {
        return switch (pref) {
            case EMAIL -> sendEmail.apply(userId, message);
            case SMS -> sendSms.apply(userId, message);
            case PUSH -> sendPush.apply(userId, message);
            case NONE -> Promise.unitPromise();
        };
    }
}
```

Exercise 3.4 [Advanced] - Implement Aspects

Add retry and timeout aspects to an existing step:

```
public interface FetchData {
    Promise<Data> apply(DataId id);
}

// Requirements:
```

```
// - Retry up to 3 times on TransientError
// - Timeout after 5 seconds
// - Log each attempt
```

Solution

```
public interface FetchData {
    Promise<Data> apply(DataId id);

    static FetchData withAspects(FetchData base, Logger log) {
        return id -> withRetry(withTimeout(base, id), id, log, 3);
    }

    private static Promise<Data> withTimeout(FetchData base, DataId id) {
        return base.apply(id)
            .timeout(TimeSpan.timeSpan(5).seconds());
    }

    private static Promise<Data> withRetry(Promise<Data> operation,
                                           DataId id,
                                           Logger log,
                                           int remainingAttempts) {
        return operation.recover(cause -> {
            log.warn("Fetch failed for {}: {}", id, cause.message());

            if (remainingAttempts <= 0) {
                return cause.promise();
            }

            return switch (cause) {
                case TransientError ignored -> {
                    log.info("Retrying fetch for {}, {} attempts remaining",
                             id, remainingAttempts);
                    yield withRetry(operation, id, log, remainingAttempts - 1);
                }
                default -> cause.promise();
            };
        });
    }
}
```

Exercise 3.5 [Intermediate] - Thread Safety Analysis

Identify the thread safety issue and fix it:

```
public class OrderProcessor {
    private List<Order> processedOrders = new ArrayList<>();
    private int totalAmount = 0;

    public Promise<OrderResult> process(Order order) {
        return validateOrder(order)
            .flatMap(this::chargePayment)
            .onSuccess(result -> {
                processedOrders.add(order);
                totalAmount += order.amount();
            })
            .map(OrderResult::new);
    }
}
```

Solution

Issues: 1. `processedOrders` is mutable and accessed from multiple threads (Promise callbacks) 2. `totalAmount` is a primitive with non-atomic read-modify-write 3. State mutation in callbacks violates JBCT principles

Fix: Remove shared mutable state, return immutable results

```
public class OrderProcessor {
    public Promise<OrderResult> process(Order order) {
        return validateOrder(order)
            .flatMap(this::chargePayment)
            .map(payment -> new OrderResult(order, payment));
    }
}

// Track processed orders externally with thread-safe collection if needed
public record OrderResult(Order order, Payment payment) {
    public int amount() {
        return order.amount();
    }
}

// If aggregation is needed, do it at a higher level:
public Promise<BatchResult> processBatch(List<Order> orders) {
    return Promise.allOf(orders.stream()
                               .map(this::process)
                               .toList())
                  .map(this::aggregateResults);
}

private BatchResult aggregateResults(List<Result<OrderResult>> results) {
    // Immutable aggregation
```

```
    var successful = results.stream()
        .filter(Result::isSuccess)
        .map(r -> r.fold(cause -> null, identity()))
        .filter(Objects::nonNull)
        .toList();

    int total = successful.stream()
        .mapToInt(OrderResult::amount)
        .sum();

    return new BatchResult(successful, total);
}
```

Part IV: Testing (Chapters 10-11)

Exercise 4.1 [Beginner] - Test Structure

Write tests for this value object:

```
public record Age(int value) {
    private static final Cause TOO_YOUNG = Causes.cause("Must be at least 0");
    private static final Cause TOO_OLD = Causes.cause("Must be at most 150");

    public static Result<Age> age(int value) {
        return Verify.ensure(value, Verify.Is::nonNegative)
            .flatMap(Verify.ensureFn(TOO_OLD, Verify.Is::lessThanOrEqualTo, 150))
            .map(Age::new);
    }
}
```

Solution

```
class AgeTest {

    @Test
    void age_succeeds_forValidValue() {
        Age.age(25)
           .onFailure(Assertions::fail)
           .onSuccess(age -> assertEquals(25, age.value()));
    }

    @Test
    void age_succeeds_forZero() {
        Age.age(0)
           .onFailure(Assertions::fail)
           .onSuccess(age -> assertEquals(0, age.value()));
```

```
    }

    @Test
    void age_succeeds_forMaximum() {
        Age.age(150)
           .onFailure(Assertions::fail)
           .onSuccess(age -> assertEquals(150, age.value()));
    }

    @Test
    void age_fails_forNegative() {
        Age.age(-1)
           .onSuccess(Assertions::fail);
    }

    @Test
    void age_fails_forTooOld() {
        Age.age(151)
           .onSuccess(Assertions::fail);
    }
}
```

Exercise 4.2 [Intermediate] - Stub Implementation

Create test stubs for this use case:

```
public interface TransferFunds {
    Promise<TransferResult> execute(TransferRequest request);

    interface ValidateAccounts {
        Promise<ValidatedAccounts> apply(AccountId from, AccountId to);
    }

    interface ExecuteTransfer {
        Promise<TransferId> apply(ValidatedAccounts accounts, Money amount);
    }
}
```

Write stubs for: success case, source account not found, insufficient funds.

Solution

```
class TransferFundsTest {

    // Success stubs
    private static final ValidateAccounts VALID_ACCOUNTS =
        (from, to) -> Promise.success(new ValidatedAccounts(
```

```
            new Account(from, Money.of(1000)),
            new Account(to, Money.of(500))
        ));

    private static final ExecuteTransfer TRANSFER_SUCCESS =
        (accounts, amount) -> Promise.success(new TransferId("TXN-001"));

    // Failure stubs
    private static final ValidateAccounts SOURCE_NOT_FOUND =
        (from, to) -> TransferError.SOURCE_NOT_FOUND.promise();

    private static final ValidateAccounts INSUFFICIENT_FUNDS =
        (from, to) -> TransferError.INSUFFICIENT_FUNDS.promise();

    @Test
    void transfer_succeeds_forValidAccounts() {
        var useCase = TransferFunds.create(VALID_ACCOUNTS, TRANSFER_SUCCESS);
        var request = new TransferRequest(
            new AccountId("ACC-001"),
            new AccountId("ACC-002"),
            Money.of(100)
        );

        useCase.execute(request)
               .await()
               .onFailure(Assertions::fail)
               .onSuccess(result ->
                   assertEquals("TXN-001", result.transferId().value()));
    }

    @Test
    void transfer_fails_whenSourceNotFound() {
        var useCase = TransferFunds.create(SOURCE_NOT_FOUND, TRANSFER_SUCCESS);
        var request = new TransferRequest(
            new AccountId("INVALID"),
            new AccountId("ACC-002"),
            Money.of(100)
        );

        useCase.execute(request)
               .await()
               .onSuccess(Assertions::fail);
    }
}
```

Exercise 4.3 [Intermediate] - Testing Async Behavior

Test that this use case properly times out:

```
public interface SlowService {
    Promise<Data> fetch(DataId id);

    static SlowService withTimeout(SlowService base, TimeSpan timeout) {
        return id -> base.fetch(id).timeout(timeout);
    }
}
```

Solution

```
class SlowServiceTest {

    @Test
    void fetch_timesOut_whenServiceTooSlow() {
        // Stub that never resolves
        SlowService neverResolves = id -> {
            Promise<Data> promise = Promise.promise();
            // Never call promise.succeed() or promise.fail()
            return promise;
        };

        var withTimeout = SlowService.withTimeout(
            neverResolves,
            TimeSpan.timeSpan(100).millis()
        );

        withTimeout.fetch(new DataId("test"))
                   .await(TimeSpan.timeSpan(500).millis())
                   .onSuccess(Assertions::fail)
                   .onFailure(cause ->
                       assertTrue(cause instanceof TimeoutError));
    }

    @Test
    void fetch_succeeds_whenServiceFastEnough() {
        SlowService fast = id -> Promise.success(new Data("result"));

        var withTimeout = SlowService.withTimeout(
            fast,
            TimeSpan.timeSpan(1).seconds()
        );

        withTimeout.fetch(new DataId("test"))
                   .await()
                   .onFailure(Assertions::fail)
```

```
                    .onSuccess(data ->
                        assertEquals("result", data.value()));
    }
}
```

Part V: Production Systems (Chapters 12-15)

Exercise 5.1 [Intermediate] - Complete Use Case Design

Design a use case interface for "Cancel Order" with these requirements: - Validate order exists and belongs to user - Only pending orders can be cancelled - Refund payment if already charged - Send cancellation notification

Solution

```
public interface CancelOrder {
    Promise<CancellationResult> execute(CancelRequest request);

    record CancelRequest(UserId userId, OrderId orderId) {}
    record CancellationResult(OrderId orderId, Option<RefundId> refundId) {}

    // Step interfaces
    interface FindOrder {
        Promise<Order> apply(OrderId orderId);
    }

    interface ValidateOwnership {
        Promise<Order> apply(Order order, UserId userId);
    }

    interface ValidateCancellable {
        Promise<Order> apply(Order order);
    }

    interface ProcessRefund {
        Promise<Option<RefundId>> apply(Order order);
    }

    interface UpdateOrderStatus {
        Promise<Unit> apply(OrderId orderId, OrderStatus status);
    }

    interface SendNotification {
        Promise<Unit> apply(UserId userId, OrderId orderId);
    }
```

```
    static CancelOrder create(FindOrder findOrder,
                              ValidateOwnership validateOwnership,
                              ValidateCancellable validateCancellable,
                              ProcessRefund processRefund,
                              UpdateOrderStatus updateStatus,
                              SendNotification sendNotification) {
        return request -> findOrder.apply(request.orderId())
            .flatMap(order -> validateOwnership.apply(order, request.userId()))
            .flatMap(validateCancellable::apply)
            .flatMap(order -> processRefund.apply(order)
                .flatMap(refundId -> updateStatus.apply(order.id(), OrderStatus.CANCELLED
                    .map(unit -> new CancellationResult(order.id(), refundId))))
            .onSuccess(result -> sendNotification.apply(request.userId(), request.orderId
                .onFailure(cause -> log.warn("Notification failed", cause)));
    }
}
```

Exercise 5.2 [Advanced] - Compensation Pattern

Implement a booking system where if any step fails, previous steps are rolled back:

```
// Steps: Reserve seat → Charge payment → Send confirmation
// If payment fails, release the seat
// If confirmation fails, refund payment and release seat
```

Solution

```
public interface BookSeat {
    Promise<Booking> execute(BookingRequest request);

    interface ReserveSeat {
        Promise<ReservationId> apply(SeatId seatId);
    }

    interface ReleaseSeat {
        Promise<Unit> apply(ReservationId reservationId);
    }

    interface ChargePayment {
        Promise<PaymentId> apply(UserId userId, Money amount);
    }

    interface RefundPayment {
        Promise<Unit> apply(PaymentId paymentId);
    }
```

```
    interface SendConfirmation {
        Promise<Unit> apply(UserId userId, ReservationId reservationId);
    }

    static BookSeat create(ReserveSeat reserveSeat,
                           ReleaseSeat releaseSeat,
                           ChargePayment chargePayment,
                           RefundPayment refundPayment,
                           SendConfirmation sendConfirmation) {
        return request -> reserveSeat.apply(request.seatId())
            .flatMap(reservationId ->
                chargePayment.apply(request.userId(), request.amount())
                    .recover(cause -> compensateReservation(releaseSeat, reservationId, c
                    .flatMap(paymentId ->
                        sendConfirmation.apply(request.userId(), reservationId)
                            .recover(cause -> compensatePayment(
                                releaseSeat, refundPayment,
                                reservationId, paymentId, cause))
                            .map(unit -> new Booking(reservationId, paymentId))));
    }

    private static Promise<PaymentId> compensateReservation(ReleaseSeat releaseSeat,
                                                            ReservationId reservationId,
                                                            Cause originalCause) {
        return releaseSeat.apply(reservationId)
            .flatMap(unit -> originalCause.promise());
    }

    private static Promise<Unit> compensatePayment(ReleaseSeat releaseSeat,
                                                   RefundPayment refundPayment,
                                                   ReservationId reservationId,
                                                   PaymentId paymentId,
                                                   Cause originalCause) {
        return Promise.all(releaseSeat.apply(reservationId),
                           refundPayment.apply(paymentId))
                      .flatMap((u1, u2) -> originalCause.promise());
    }
}
```

Exercise 5.3 [Intermediate] - Project Structure

Given these classes, organize them into the correct package structure:

```
RegisterUser (use case interface)
RegisterUserImpl (use case implementation - if needed)
```

```
ValidRegistration (validated request)
Email, Password, Username (value objects)
UserRepository (step interface)
JpaUserRepository (repository implementation)
RegistrationError (error types)
UserController (REST controller)
UserDto (API DTO)
```

Solution

```
src/main/java/com/example/user/
├── domain/
│   ├── Email.java              # Value object
│   ├── Password.java           # Value object
│   ├── Username.java           # Value object
│   └── RegistrationError.java  # Domain errors
├── usecase/
│   └── RegisterUser.java       # Use case with:
│                               #   - Request record
│                               #   - ValidRegistration record
│                               #   - Response record
│                     #   - Step interfaces (SaveUser, CheckEmailUnique, etc.)
│                               #   - Factory method
├── adapter/
│   ├── persistence/
│   │   └── JpaUserRepository.java  # Implements step interface
│   └── web/
│       ├── UserController.java     # REST controller
│       └── UserDto.java            # API DTO
└── config/
    └── UserConfiguration.java      # Spring wiring
```

Notes: - `RegisterUserImpl` is not needed - factory method in interface creates implementation - `ValidRegistration` is nested in `RegisterUser` as `ValidRequest` record - `UserRepository` doesn't exist separately - it's a step interface inside use case

---

Part VI: Adoption (Chapters 16-19)

Exercise 6.1 [Beginner] - Code Review Checklist

Review this code for JBCT violations:

```
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;
```

```
    public User createUser(String email, String password) {
        if (email == null || !email.contains("@")) {
            throw new ValidationException("Invalid email");
        }

        if (userRepository.findByEmail(email).isPresent()) {
            throw new EmailExistsException();
        }

        User user = new User();
        user.setEmail(email);
        user.setPassword(passwordEncoder.encode(password));

        return userRepository.save(user);
    }
}
```

Solution

Violations found:

1. **Primitive obsession** - Using raw `String` for email and password instead of value objects
2. **Validation in wrong place** - Controller or service validating instead of value object factory
3. **Exceptions for business errors** - EmailExistsException should be a Cause in Result
4. **Mutable entity** - Using setters instead of immutable record
5. **Field injection** - Using @Autowired instead of constructor injection
6. **Missing use case interface** - Business logic directly in service

Refactored:

```
public interface CreateUser {
    Promise<UserId> execute(CreateUserRequest request);

    record CreateUserRequest(String email, String password) {}
    record ValidRequest(Email email, Password password) {}

    interface CheckEmailUnique {
        Promise<Email> apply(Email email);
    }

    interface SaveUser {
        Promise<UserId> apply(ValidRequest request);
    }

    static CreateUser create(CheckEmailUnique checkEmail, SaveUser saveUser) {
```

```
        return request -> ValidRequest.validRequest(request)
            .async()
            .flatMap(valid -> checkEmail.apply(valid.email())
                .map(email -> valid))
            .flatMap(saveUser::apply);
    }
}
```

Exercise 6.2 [Intermediate] - Migration Planning

You have this legacy service. Plan a phase-by-phase migration:

```
@Service
public class OrderService {
    public Order placeOrder(OrderDto dto) {
        // 50 lines of validation
        // 30 lines of inventory check
        // 40 lines of payment processing
        // 20 lines of order creation
        // 15 lines of notification
    }
}
```

Solution

Phase 1: Value Objects (Week 1) - Create: OrderId, ProductId, Quantity, Money, CustomerId - Parallel method: `placeOrderValidated(ValidOrderRequest request)` - Keep original method, call validated version internally

Phase 2: Result in New Code (Week 2) - Extract validation to `ValidOrderRequest.validRequest(OrderDto)` - Return `Result<Order>` instead of throwing - Create `OrderError` sealed interface

Phase 3: Extract Use Case (Week 3-4) - Create `PlaceOrder` interface with step interfaces: - `CheckInventory` - `ProcessPayment` - `CreateOrder` - `SendNotification` - Move business logic to factory method - Service becomes thin wrapper calling use case

Phase 4: Adapter Isolation (Week 5) - Wrap inventory check in `Promise.lift()` - Wrap payment gateway in `Promise.lift()` - Wrap repository in `Promise.lift()` - Email service as fire-and-forget Promise

Migration commits: 1. Add value objects 2. Add ValidOrderRequest 3. Add OrderError types 4. Extract PlaceOrder interface 5. Implement step interfaces as adapters 6. Convert service to use case wrapper 7. Remove legacy method

---

Exercise 6.3 [Beginner] - Debugging Practice

This chain fails silently. Add debugging to find where:

```
public Promise<Report> generateReport(ReportRequest request) {
    return validateRequest(request)
        .flatMap(this::fetchData)
        .flatMap(this::transformData)
        .flatMap(this::formatReport);
}
```

Solution

```
public Promise<Report> generateReport(ReportRequest request) {
    return validateRequest(request)
        .onSuccess(r -> log.debug("Validated: {}", r))
        .onFailure(c -> log.error("Validation failed: {}", c.message()))
        .flatMap(this::fetchData)
        .onSuccess(d -> log.debug("Fetched {} records", d.size()))
        .onFailure(c -> log.error("Fetch failed: {}", c.message()))
        .flatMap(this::transformData)
        .onSuccess(t -> log.debug("Transformed to {} items", t.size()))
        .onFailure(c -> log.error("Transform failed: {}", c.message()))
        .flatMap(this::formatReport)
        .onSuccess(r -> log.debug("Report generated: {} pages", r.pageCount()))
        .onFailure(c -> log.error("Format failed: {}", c.message()));
}
```

Alternative: Extract to named methods for breakpoints:

```
public Promise<Report> generateReport(ReportRequest request) {
    return validateRequestStep(request)
        .flatMap(this::fetchDataStep)
        .flatMap(this::transformDataStep)
        .flatMap(this::formatReportStep);
}

private Promise<ValidRequest> validateRequestStep(ReportRequest request) {
    log.debug("Validating request");
    return validateRequest(request);  // Breakpoint here
}

// ... similar for other steps
```

Summary

These exercises cover the full JBCT learning path:

Part	Focus	Key Skills
I	Foundations	Type selection, Cause creation, basic operations
II	Principles	Value objects, error accumulation, null handling
III	Patterns	Pattern identification, implementation, thread safety
IV	Testing	Test structure, stubs, async testing
V	Production	Use case design, compensation, project structure
VI	Adoption	Code review, migration, debugging

Practice these exercises to build muscle memory for JBCT patterns. The goal is to make these patterns your default way of thinking about code.

Appendix C: Glossary

This glossary defines key terms used throughout the book.

A

Adapter A component that bridges between external systems and the domain. Adapters implement step interfaces and wrap external I/O in Promise types. Examples: repositories, HTTP clients, message queue producers.

Adapter Zone The architectural layer containing adapters. Converts between external representations (JSON, SQL) and domain types. The only place where Promise.lift() should appear.

Aggregation Combining multiple Result or Promise values into one. `Result.all()` accumulates all failures. `Promise.all()` fails fast on first failure.

Aspects Pattern A structural pattern for cross-cutting concerns like retry, timeout, logging, and audit. Implemented as wrapper functions that compose around a core operation.

B

Bifurcation Handling both success and failure cases. Methods like `fold()` and `apply()` accept two handlers - one for each outcome.

C

Cause A typed error value in Pragmatica Lite. Represents why an operation failed. Defined as sealed interfaces for exhaustive handling. Preferred over exceptions for business errors.

Chain A sequence of monadic operations connected by `flatMap()`. Each step receives the previous step's success value. Failures short-circuit the chain.

Clean Architecture Uncle Bob's architectural style with concentric layers and the dependency rule. JBCT can be implemented within Clean Architecture.

CompositeCause A Cause containing multiple sub-causes. Created by `Result.all()` when multiple validations fail. Allows collecting all errors instead of stopping at first.

Condition Pattern A structural pattern for routing based on a discriminator value. Implemented with switch expressions. Each branch returns the same type.

D

Dependency Inversion Dependencies point toward abstractions, not implementations. In JBCT, use cases depend on step interfaces (abstractions), adapters implement them.

Domain Zone The architectural layer containing pure business logic. Contains use cases, value objects, and step interfaces. No framework dependencies.

E

Error Accumulation Collecting all errors instead of stopping at the first. `Result.all()` accumulates failures into CompositeCause.

Exception Wrapping Converting exceptions to Cause values at adapter boundaries. Done with `Promise.lift()` or `Result.lift()`.

External Zone Everything outside your application: databases, external APIs, message queues, file systems.

F

Factory Method A static method that creates instances of value objects or use cases. Value object factories return `Result<T>` to enforce validation.

Fail-Fast Stopping at the first failure. `flatMap()` chains and `Promise.all()` are fail-fast.

flatMap The monadic bind operation. Chains dependent operations. If input is failure, skips the operation. Prevents nested monads (`Result<Result<T>>`).

Fn1, Fn2, Fn3 Functional interfaces in Pragmatica Lite for functions with 1, 2, or 3 parameters. Similar to Java's Function but with better composition.

Fork-Join Pattern A structural pattern for parallel independent operations. Uses `Promise.all()` to run operations concurrently and combine results.

G

Guard Clause Early return on invalid input. In JBCT, replaced by parse-don't-validate: validation happens in value object factories, not guard clauses.

H

Happy Path The execution path when all operations succeed. In JBCT, the happy path is the default - failures are handled explicitly.

Hexagonal Architecture Ports and Adapters architecture. Domain at center, adapters at edges. JBCT's step interfaces are similar to ports.

I

Immutability Objects that cannot be modified after creation. Java records are immutable by default. Essential for thread safety in JBCT.

Iteration Pattern A structural pattern for processing collections. Uses `Promise.allOf()` for parallel processing or stream operations for sequential.

J

JBCT Java Backend Coding Technology. A methodology for writing maintainable backend Java code using functional composition and explicit error handling.

L

Leaf Pattern A structural pattern for atomic operations with no dependencies. Typically adapters wrapping external I/O with `Promise.lift()`.

Lift Converting a value or operation to a monadic context. `Result.lift()` wraps throwing code in Result. `result.async()` lifts Result to Promise.

M

map Transform the success value while preserving the container. Does not flatten nested containers - use `flatMap()` when the mapper returns a monadic type.

Monad A design pattern for chaining operations that may have side effects. Option, Result, and Promise are all monads. The key operation is `flatMap()`.

O

Option A type representing a value that may be absent. Use when absence is normal, not an error. Convert to Result when absence should be an error.

P

Parse, Don't Validate A principle: instead of validating data and proceeding with raw types, parse data into validated types that make invalid states unrepresentable.

Port In Hexagonal Architecture, an interface defining how the domain interacts with the outside world. Similar to JBCT's step interfaces.

Pragmatica Lite A minimal Java library providing Option, Result, Promise, and Cause types. The foundation for JBCT patterns.

Promise A type representing an asynchronous operation that may fail. Combines async execution with explicit error handling.

Pure Function A function with no side effects that always returns the same output for the same input. Value object factories should be pure.

R

Railway-Oriented Programming (ROP) An error handling approach using "two-track" types. Success track and failure track. JBCT uses this model via Result and Promise.

Record Java's immutable data class (since Java 14). Used for value objects, requests, responses, and error types in JBCT.

recover Handle failures and potentially convert to success. Takes a function `Cause -> M<T>` that can either provide a fallback value or re-throw.

Result A type representing a synchronous operation that may fail. Contains either a success value or a Cause.

Result.all Combines multiple Results, accumulating all failures into CompositeCause. Use for independent validations.

S

Sealed Interface Java's restricted inheritance (since Java 17). All implementations must be known at compile time. Used for Cause types and exhaustive switch expressions.

Sequencer Pattern A structural pattern for linear chains of dependent operations. Each step depends on the previous step's output. Uses `flatMap()` chains.

Short-Circuit Stopping execution at the first failure. `flatMap()` chains short-circuit. `Result.all()` does not short-circuit.

Step Interface A functional interface defining one operation in a use case. Declared inside the use case interface. Implemented by adapters.

Stub A test double that returns predetermined values. In JBCT, typically lambdas implementing step interfaces.

T

Three Zones JBCT's architectural model: External (outside systems), Adapter (boundary translation), Domain (pure business logic).

TimeSpan Pragmatica Lite's duration type. Created with fluent API: `TimeSpan.timeSpan(5).seconds()`.

Type Lifting Converting from a lower type to a higher one: `T` → `Option` → `Result` → `Promise`. Safe direction that adds information.

U

Unit A type with a single value, representing "no meaningful result". Used instead of `Void` for operations that succeed but return nothing.

Use Case A single business operation exposed as a functional interface. Contains Request/ValidRequest/Response records and step interfaces.

V

Validation Checking that data meets requirements. In JBCT, validation happens in value object factories, returning `Result<T>`.

Value Object An immutable object representing a domain concept. Created through factory methods that enforce validation. Examples: Email, Money, UserId.

Verify Pragmatica Lite utility class for validation predicates. `Verify.ensure()` creates Result from predicate. `Verify.Is` provides common predicates.

W

Wrapper A function that adds behavior around another function. Used in the Aspects pattern for cross-cutting concerns.

Quick Reference

Term	One-Line Definition
Adapter	Bridges external systems to domain
Cause	Typed error value
flatMap	Chain dependent operations
Fork-Join	Parallel independent operations
Leaf	Atomic adapter operation
map	Transform success value
Option	Value that may be absent
Parse, Don’t Validate	Create validated types, not checked raw data
Promise	Async operation that may fail
Result	Sync operation that may fail
Sequencer	Linear chain of dependent steps
Step Interface	One operation in a use case
Unit	Type for “no result”
Use Case	Single business operation
Value Object	Immutable validated domain concept